ITSB CORPORATION · TECHNICAL DATA ANALYTICS LEARNING CAMP

BUKU PRAKTIKUM

Interactive Dashboard Development with Streamlit & Dash

Dari EDA dan visualisasi interaktif hingga integrasi final Machine Learning model ke dashboard

Durasi	Visualisasi	Aplikasi	Model ML
16 jam	**40+**	**11 app**	**1 final**
8 sesi · 2 hari	pie hingga gauge	Streamlit & Dash	inference siap pakai

Informasi	Keterangan
Program	2nd On-the-Job Training (OJT) — Technical Data Analytics
Penyelenggara	ITSB Corporation
Durasi	2 hari · 16 jam · 8 sesi · Learning Camp
Platform / Tools	Python, Streamlit, Dash, Pandas, Plotly, Matplotlib, scikit-learn
Instruktur	Bakti Siregar, M.Sc., CDSS.
Studi kasus	Customer Churn Dashboard dengan final trained ML model

Edisi 22-23 Oktober 2026

Edisi Pertama: Bakti Siregar, M.Sc., CDSS.

Daftar Isi

Pendahuluan **1**
Cara memakai buku ini 1
Prasyarat peserta 1
Persiapan environment 1
Struktur folder proyek 2
Studi kasus terintegrasi: Customer Churn 2
Model ML final yang dipakai 3
Alur end-to-end pelatihan 3

Sesi 1 — Streamlit Fundamentals (90 menit) **5**
Tujuan sesi 5
1.1 Apa itu Streamlit 5
1.2 Model eksekusi Streamlit 5
1.3 Praktikum A — Aplikasi pertama 5
1.4 Struktur aplikasi Streamlit yang disarankan 6
1.5 Praktikum B — Tur komponen dasar 6
1.6 Praktikum C — Aplikasi sederhana: kalkulator tagihan 8
Latihan mandiri Sesi 1 9
Kesalahan umum Sesi 1 9

Sesi 2 — EDA & Basic Visualization (90 menit) **11**
Tujuan sesi 11
2.1 Alur EDA yang dipakai 11
2.2 Lima tujuan visualisasi dan grafik yang tepat 11
2.3 Praktikum A — Persiapan, profiling, dan statistik 12
2.4 Galeri visualisasi EDA 14
2.5 Praktikum B — Dashboard EDA interaktif di Streamlit 25
2.6 Menulis insight dari EDA 33
Latihan mandiri Sesi 2 33
Kesalahan umum Sesi 2 33

Sesi 3 — Dynamic Layout with Streamlit (120 menit) **35**
Tujuan sesi 35
3.1 Komponen layout Streamlit 35
3.2 Pola desain dashboard 35
3.3 Praktikum — Dashboard Streamlit dengan layout dinamis 36
3.4 Teknik tambahan 40
Latihan mandiri Sesi 3 40
Kesalahan umum Sesi 3 40

Sesi 4 — ML Model Inference & Dashboard Integration (105 menit) **42**
Tujuan sesi 42
4.1 Posisi sesi ini dalam siklus ML 42
4.2 Kontrak input model 42
4.3 Praktikum A — Modul inference yang dapat dipakai ulang 43
4.4 Praktikum B — Uji inference di terminal 44
4.5 Praktikum C — Dashboard prediksi di Streamlit 45
4.6 Menyajikan prediction output secara bertanggung jawab 49
Latihan mandiri Sesi 4 49
Kesalahan umum Sesi 4 49

Edisi Pertama: Bakti Siregar, M.Sc., CDSS.

Sesi 5 — Dash Fundamentals (90 menit) 51
Tujuan sesi 51
5.1 Perbedaan cara berpikir Streamlit dan Dash 51
5.2 Struktur minimal aplikasi Dash 51
5.3 Tiga keluarga komponen 51
5.4 Praktikum — Dashboard Dash statis 52
5.5 Pola dasar tata letak dashboard di Dash 54
Latihan mandiri Sesi 5 54
Kesalahan umum Sesi 5 55

Sesi 6 — Interactive Visualization with Plotly (90 menit) 56
Tujuan sesi 56
6.1 Plotly Express vs Graph Objects 56
6.2 Interaksi bawaan Plotly 56
6.3 Praktikum A — Galeri chart Plotly 56
6.4 Praktikum B — Chart Plotly di aplikasi Dash 64
6.5 Dari chart ke insight 65
Latihan mandiri Sesi 6 65
Kesalahan umum Sesi 6 66

Sesi 7 — Dash Callback System (120 menit) 67
Tujuan sesi 67
7.1 Anatomi callback 67
7.2 Contoh paling sederhana 68
7.3 Praktikum — Dashboard dengan sistem callback lengkap 68
7.4 Penjelasan empat callback 71
7.5 Checkpoint 73
7.6 Debugging callback 73
Latihan mandiri Sesi 7 73
Kesalahan umum Sesi 7 73

Sesi 8 — Final Dashboard Project (105 menit) 75
Tujuan sesi 75
8.1 Alur kerja proyek akhir (105 menit) 75
8.2 Arsitektur dashboard 75
8.3 Kode lengkap final dashboard 76
8.4 Penjelasan keputusan desain 81
8.5 Pemenuhan Dashboard Minimum Requirements 81
8.6 Checkpoint 82
8.7 Contoh insight dan rekomendasi 82
8.8 Struktur presentasi 5–7 menit 83
8.9 Template final project 83
8.10 Checklist final submission 83
8.11 Rubrik penilaian 84
8.12 Variasi studi kasus 84

Lampiran 85
A. Kunci jawaban latihan mandiri 85
B. Cheat sheet 87
C. Troubleshooting umum 88
D. Isi paket praktikum 89
E. Referensi 89

Daftar Gambar

Gambar P.1 Alur end-to-end pelatihan dan pemetaannya ke sesi 4
Gambar 1.1 Tur komponen dasar Streamlit 8
Gambar 1.2 Kalkulator tagihan setelah tombol Hitung diklik 9
Gambar 2.1 KPI cards hasil Matplotlib 14
Gambar 2.2 Pie chart status churn 15
Gambar 2.3 Donut chart komposisi kontrak 16
Gambar 2.4 Bar chart churn rate per kontrak 17
Gambar 2.5 Bar chart horizontal churn per region 17
Gambar 2.6 Grouped bar churn rate per segmen dan kontrak 18
Gambar 2.7 Stacked bar 100% status per metode pembayaran 19
Gambar 2.8 Histogram tenure per status 19
Gambar 2.9 Box plot tagihan per status 20
Gambar 2.10 Violin plot tenure per kontrak 21
Gambar 2.11 Line chart pelanggan baru per bulan 21
Gambar 2.12 Line chart churn rate per kelompok tenure 22
Gambar 2.13 Area chart pelanggan baru per kuartal per segmen 23
Gambar 2.14 Scatter plot tenure vs tagihan 23
Gambar 2.15 Heatmap churn rate region x segmen 24
Gambar 2.16 Heatmap korelasi variabel numerik 25
Gambar 2.17 Tab Komposisi: KPI card, pie, donut, dan treemap 28
Gambar 2.18 Tab Perbandingan: bar terurut dengan tabel berwarna, grouped bar, dan stacked bar 100% 29
Gambar 2.19 Tab Distribusi: histogram dengan box marginal dan violin plot 30
Gambar 2.20 Tab Tren: line chart dan area chart 30
Gambar 2.21 Tab Hubungan: scatter plot, heatmap korelasi, dan heatmap pivot 31
Gambar 2.22 Tab Data: profil kolom, statistik deskriptif, dan tabel data 32
Gambar 3.1 Dashboard Streamlit Sesi 3 — tab Ringkasan 38
Gambar 3.2 Dashboard Streamlit Sesi 3 — tab Segmentasi dengan tabel berwarna 39
Gambar 4.1 Prediksi satu pelanggan di Streamlit 47
Gambar 4.2 Prediksi batch: KPI, donut kategori risiko, bar probabilitas, dan tabel progres 48
Gambar 5.1 Dashboard Dash statis Sesi 5 54
Gambar 6.1 Bar chart berkelompok dengan hover kustom 59
Gambar 6.2 Pie chart proporsi status 60
Gambar 6.3 Donut chart dengan angka total di tengah 60
Gambar 6.4 Histogram overlay per status 60
Gambar 6.5 Scatter dengan facet per jenis kontrak (small multiples) 61
Gambar 6.6 Line chart dengan range slider 61
Gambar 6.7 Box plot tagihan per jumlah keluhan 61
Gambar 6.8 Bar Graph Objects dengan label dan garis rata-rata 62
Gambar 6.9 Heatmap churn rate region × segmen 62
Gambar 6.10 Treemap region → segmen 62
Gambar 6.11 Sunburst kontrak → segmen 63
Gambar 6.12 Funnel penyempitan kelompok berisiko 63
Gambar 6.13 Gauge/indicator churn rate terhadap target 63
Gambar 6.14 Aplikasi Dash Sesi 6 dengan chart Plotly 65
Gambar 7.1 Dashboard Dash dengan sistem callback 71

Edisi Pertama: Bakti Siregar, M.Sc., CDSS.

Gambar 7.2 Graf callback sesi7/app_dash_callback.py 72
Gambar 8.1 Final dashboard — tab Overview & Risiko 80
Gambar 8.2 Final dashboard — tab Simulasi Prediksi 81

Daftar Tabel

Tabel P.1 Studi kasus terintegrasi: Customer Churn 3
Tabel P.2 Model ML final yang dipakai 3
Tabel 1.1 Apa itu Streamlit 5
Tabel 1.2 Ringkasan komponen yang dipakai 7
Tabel 1.3 Kesalahan umum Sesi 1 9
Tabel 2.1 Lima tujuan visualisasi dan grafik yang tepat 11
Tabel 2.2 Praktikum A — Persiapan, profiling, dan statistik 13
Tabel 2.3 Statistik deskriptif variabel numerik 13
Tabel 2.4 Hasil filtering dan grouping yang dicetak 13
Tabel 2.5 Menulis insight dari EDA 33
Tabel 2.6 Kesalahan umum Sesi 2 33
Tabel 3.1 Komponen layout Streamlit 35
Tabel 3.2 Kesalahan umum Sesi 3 41
Tabel 4.1 Posisi sesi ini dalam siklus ML 42
Tabel 4.2 Kesalahan umum Sesi 4 49
Tabel 5.1 Tiga keluarga komponen 52
Tabel 5.2 Kesalahan umum Sesi 5 55
Tabel 6.1 Plotly Express vs Graph Objects 56
Tabel 6.2 Kesalahan umum Sesi 6 66
Tabel 7.1 Anatomi callback 67
Tabel 7.2 Kesalahan umum Sesi 7 73
Tabel 8.1 Alur kerja proyek akhir 75
Tabel 8.2 Pemenuhan Dashboard Minimum Requirements 82
Tabel 8.3 Template final project 83
Tabel 8.4 Rubrik penilaian 84
Tabel 8.5 Variasi studi kasus 84
Tabel L.1 Cheat sheet (bagian 1) 87
Tabel L.2 Cheat sheet (bagian 2) 88
Tabel L.3 Cheat sheet (bagian 3) 88
Tabel L.4 Troubleshooting umum 88
Tabel L.5 Isi paket praktikum 89

Pendahuluan

Buku ini memandu peserta membangun dashboard interaktif berbasis Python dengan Streamlit dan Dash dalam 8 sesi (2 hari, 16 jam), dari aplikasi pertama sampai dashboard final yang menampilkan output prediksi dari model Machine Learning final. Seluruh praktikum memakai satu studi kasus yang sama, Customer Churn, sehingga setiap sesi menambah satu lapisan pada dashboard yang sedang dibangun.

Pelatihan ini adalah bagian dari 2nd On-the-Job Training (OJT) Technical Data Analytics, ITSB Corporation, dengan instruktur Bakti Siregar, M.Sc. Pendekatannya practice-first: konsep singkat, demonstrasi, hands-on, integrasi, proyek akhir, lalu interpretasi hasil.

Cara memakai buku ini

Setiap bab sesi memiliki struktur yang sama:

1. **Tujuan sesi** — kemampuan yang harus dikuasai di akhir sesi.
2. **Konsep inti** — penjelasan singkat sebelum menulis kode.
3. **Praktikum terpandu** — langkah bernomor, kode lengkap, dan perintah untuk menjalankannya.
4. **Penjelasan kode** — bagian penting dari kode dibahas baris demi baris.
5. **Checkpoint** — hasil yang harus terlihat di layar sebelum lanjut.
6. **Latihan mandiri** — tugas untuk dikerjakan sendiri; kunci jawaban ada di Lampiran.
7. **Kesalahan umum** — pesan error yang sering muncul dan cara mengatasinya.

Semua kode di buku ini sudah dijalankan dan diuji dengan Python 3.13, Streamlit 1.65, Dash 4.4, Plotly 7.1, pandas 3.0, dan scikit-learn 1.9.1. Kode juga kompatibel dengan versi yang lebih lama sesuai `requirements.txt` (Streamlit ≥ 1.40, Dash ≥ 2.17, Plotly ≥ 5.22, pandas ≥ 2.2).

Prasyarat peserta

- Dasar Python: variabel, list, dictionary, fungsi, `if`, dan `for`.
- Dasar pandas: membaca CSV, memilih kolom, dan filter baris.
- Laptop dengan Python 3.10 atau lebih baru, editor kode (VS Code disarankan), dan koneksi internet.
- Materi ini bukan pengantar Python dari nol.

Persiapan environment

Lakukan langkah berikut sebelum Sesi 1. Perintah ditulis untuk Windows (PowerShell/CMD) dan macOS/Linux (Terminal).

1. Ekstrak paket praktikum `dashboard_course.zip` ke folder kerja, misalnya `D:\pelatihan\dashboard_course`.
2. Buka terminal di folder tersebut.
3. Buat virtual environment agar library pelatihan terpisah dari instalasi Python lain.

```bash
# Windows
python -m venv .venv
.venv\Scripts\activate
```

```
# macOS / Linux
python3 -m venv .venv
source .venv/bin/activate
```

1. Instal seluruh library dari `requirements.txt`.

```bash
pip install -r requirements.txt
```

1. Verifikasi instalasi. Perintah berikut harus mencetak lima nomor versi tanpa error.

```bash
python -c "import streamlit, dash, plotly, pandas, sklearn; print(streamlit.__version__, dash.__version__,
plotly.__version__, pandas.__version__, sklearn.__version__)"
```

Isi `requirements.txt`:

```text
streamlit>=1.40
dash>=2.17
plotly>=5.22
pandas>=2.2
numpy>=1.26
matplotlib>=3.8
scikit-learn==1.9.1   # samakan dengan versi saat model difinalisasi (lihat model_metadata.json)
joblib>=1.4
```

Versi scikit-learn dikunci karena model yang disimpan dengan `joblib` paling aman dimuat dengan versi yang sama dengan saat model difinalisasi. Versi berbeda dapat memunculkan `InconsistentVersionWarning` atau error saat inference.

Struktur folder proyek

Semua perintah di buku ini dijalankan dari folder akar `dashboard_course/`.

```text
dashboard_course/
├── data/
│   ├── generate_data.py          # pembuat dataset latihan (sudah dijalankan)
│   ├── customer_churn.csv        # 3.000 pelanggan, 13 kolom
│   └── new_customers.csv         # 200 pelanggan baru tanpa label (untuk inference)
├── models/
│   ├── prepare_final_model.py    # script fasilitator, BUKAN bagian sesi
│   ├── churn_model.joblib        # final trained / selected best-performing model
│   └── model_metadata.json       # kontrak input model
├── utils/
│   └── model_utils.py            # fungsi load, validasi, dan prediksi
├── sesi1/ … sesi8/               # kode praktikum per sesi
├── assets/                       # screenshot hasil akhir
└── requirements.txt
```

Studi kasus terintegrasi: Customer Churn

Sebuah penyedia layanan internet ingin mengetahui pelanggan mana yang berisiko berhenti berlangganan (churn) dan tindakan apa yang perlu diambil. Pertanyaan bisnis utama: **siapa pelanggan yang berisiko churn, di segmen mana risiko terkonsentrasi, dan apa rekomendasi tindakannya?**

Dataset `customer_churn.csv` adalah data sintetis untuk latihan, dibuat dengan seed tetap agar hasil setiap peserta sama.

Rinciannya disajikan pada Tabel P.1.

Tabel P.1. *Studi kasus terintegrasi: Customer Churn*

Kolom	Tipe	Keterangan
customer_id	teks	ID unik pelanggan, contoh `CUST-00001`
signup_date	tanggal	Tanggal mulai berlangganan, Januari 2023 sampai Desember 2025
region	kategori	Jawa, Sumatera, Kalimantan, Sulawesi, Bali-Nusa Tenggara
segment	kategori	Retail, UMKM, Korporat
contract_type	kategori	Bulanan, Tahunan, Dua Tahun
payment_method	kategori	Transfer Bank, Kartu Kredit, E-Wallet, Virtual Account
internet_service	kategori	Fiber, DSL, Mobile Broadband
tenure_months	angka	Lama berlangganan dalam bulan (1–71)
monthly_charges	angka	Tagihan bulanan dalam Rupiah
total_charges	angka	Akumulasi tagihan; 45 baris sengaja kosong untuk latihan data quality
num_complaints	angka	Jumlah keluhan yang tercatat
support_calls	angka	Jumlah panggilan ke customer service
churn	0/1	Target: 1 = pelanggan berhenti berlangganan

Ringkasan data: 3.000 baris, churn rate keseluruhan 34,9%.

Model ML final yang dipakai

Model sudah dilatih dan dipilih oleh fasilitator sebelum pelatihan. Peserta **tidak** melatih ulang, tidak melakukan tuning, dan tidak membandingkan model. Tugas peserta: memuat model, menyiapkan input, menjalankan inference, dan menampilkan prediction output.

Rinciannya disajikan pada Tabel P.2.

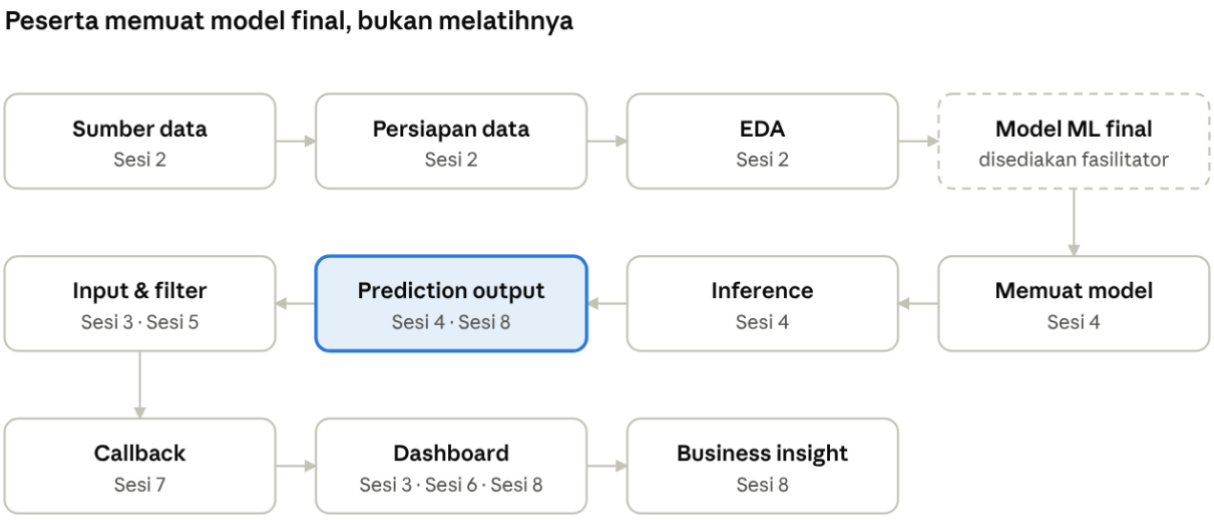
Tabel P.2. *Model ML final yang dipakai*

Atribut	Nilai
File model	`models/churn_model.joblib`
Jenis	scikit-learn `Pipeline`: preprocessing (imputasi median, standarisasi, one-hot encoding) + `GradientBoostingClassifier`
Input	10 fitur: 5 numerik (`tenure_months`, `monthly_charges`, `total_charges`, `num_complaints`, `support_calls`) dan 5 kategorik (`region`, `segment`, `contract_type`, `payment_method`, `internet_service`)
Output	probabilitas churn (0–1), kelas prediksi (threshold 0,5), kategori risiko
Kategori risiko	Rendah < 0,30 · Sedang 0,30–0,60 · Tinggi ≥ 0,60
Kinerja holdout (informasi saja)	ROC-AUC 0,789 · akurasi 0,718 · F1 0,579

Karena preprocessing berada di dalam `Pipeline`, data mentah dapat langsung dikirim ke `model.predict_proba()` asalkan nama kolom dan nilai kategorinya sama dengan saat model difinalisasi. File `model_metadata.json` mencatat "kontrak" tersebut.

Alur end-to-end pelatihan

Seluruh sesi mengikuti satu alur yang sama: data mentah diolah, dieksplorasi, dikirim ke model final, lalu hasilnya disajikan melalui komponen interaktif sampai menjadi insight bisnis (lihat Gambar P.1).



Gambar P.1. *Alur end-to-end pelatihan dan pemetaannya ke sesi*

Kotak bergaris putus-putus (model ML final) adalah satu-satunya tahap di luar pelatihan; kotak berwarna (prediction output) adalah hasil yang wajib tampil di final dashboard.

Sesi 1 — Streamlit Fundamentals (90 menit)

Di akhir sesi ini peserta sudah menjalankan tiga aplikasi Streamlit: aplikasi sapaan, tur komponen dasar, dan kalkulator tagihan sebagai aplikasi sederhana pertama.

Tujuan sesi

- Menjelaskan apa itu Streamlit dan kapan memakainya.
- Menginstal dan menjalankan aplikasi Streamlit dari terminal.
- Memahami model eksekusi Streamlit (script dijalankan ulang dari atas setiap ada interaksi).
- Menggunakan komponen teks, data, input, tombol, pesan status, dan grafik bawaan.
- Membuat aplikasi sederhana yang menerima input dan menampilkan hasil perhitungan.

1.1 Apa itu Streamlit

Streamlit adalah library Python open-source untuk mengubah script analisis data menjadi aplikasi web interaktif tanpa menulis HTML, CSS, atau JavaScript. Satu file `.py` sudah cukup untuk satu aplikasi.

Rinciannya disajikan pada Tabel 1.1.

Tabel 1.1. *Apa itu Streamlit*

Aspek	Streamlit	Dash (Sesi 5–8)
Gaya penulisan	Script berurutan dari atas ke bawah	Layout + fungsi callback terpisah
Interaktivitas	Seluruh script dijalankan ulang saat input berubah	Hanya callback terkait yang dijalankan
Kurva belajar	Sangat landai, cocok untuk prototipe cepat	Sedikit lebih curam, kontrol lebih detail
Cocok untuk	Prototipe, tool internal, demo model	Dashboard produksi dengan banyak interaksi antarkomponen

1.2 Model eksekusi Streamlit

Konsep terpenting di Streamlit: **setiap kali pengguna mengubah widget (slider, tombol, dropdown), seluruh script dijalankan ulang dari baris pertama sampai terakhir.** Nilai widget terbaru dikembalikan sebagai variabel Python biasa. Karena itu:

- Proses berat seperti membaca file besar atau memuat model harus di-cache (`@st.cache_data`, `@st.cache_resource`) agar tidak diulang setiap interaksi. Ini dibahas di Sesi 2 dan 4.
- Variabel biasa akan kembali ke nilai awal setiap rerun. Untuk menyimpan nilai antar-rerun gunakan `st.session_state`.

1.3 Praktikum A — Aplikasi pertama

1. Pastikan virtual environment aktif (nama `.venv` muncul di awal baris terminal).
2. Buat file `sesi1/app_hello.py` dengan isi berikut.

```python
# sesi1/app_hello.py — aplikasi Streamlit pertama
```

```
import streamlit as st

st.title("Halo, Streamlit! 👋")
st.write("Ini aplikasi data interaktif pertama saya.")

nama = st.text_input("Siapa nama Anda?")
if nama:
    st.success(f"Selamat datang di pelatihan dashboard, {nama}!")
```

1. Jalankan aplikasi dari folder akar proyek.

```
BASH
streamlit run sesi1/app_hello.py
```

1. Browser akan terbuka otomatis di `http://localhost:8501`. Jika tidak, salin alamat yang muncul di terminal ke browser.
2. Ketik nama Anda di kotak input lalu tekan Enter.
3. Untuk menghentikan aplikasi, kembali ke terminal dan tekan `Ctrl + C`.

Penjelasan kode

- `import streamlit as st` — alias `st` adalah konvensi umum.
- `st.title()` — judul besar di halaman.
- `st.write()` — fungsi serbaguna: dapat menampilkan teks, angka, DataFrame, atau grafik.
- `st.text_input()` mengembalikan string yang diketik pengguna; awalnya string kosong sehingga blok `if nama:` belum dijalankan.
- Saat pengguna menekan Enter, script dijalankan ulang, `nama` kini berisi teks, dan `st.success()` muncul.

Checkpoint: halaman menampilkan judul, kalimat pengantar, kotak input, dan kotak hijau berisi sapaan setelah nama diisi.

Tips: aktifkan opsi **Always rerun** di pojok kanan atas aplikasi (atau tekan `R`) agar perubahan kode langsung terlihat setelah file disimpan.

1.4 Struktur aplikasi Streamlit yang disarankan

Aplikasi kecil cukup ditulis berurutan. Untuk aplikasi yang akan berkembang menjadi dashboard, gunakan urutan berikut agar mudah dibaca:

1. Import library.
2. `st.set_page_config()` — harus menjadi perintah Streamlit pertama.
3. Konstanta dan fungsi bantu (termasuk fungsi load data yang di-cache).
4. Input pengguna (biasanya di sidebar).
5. Pemrosesan data berdasarkan input.
6. Output: KPI, grafik, tabel.

1.5 Praktikum B — Tur komponen dasar

Buat file `sesi1/app_komponen.py`, lalu jalankan dengan `streamlit run sesi1/app_komponen.py`.

PYTHON

```python
# sesi1/app_komponen.py — tur komponen dasar Streamlit
import datetime as dt
import numpy as np
import pandas as pd
import streamlit as st

st.set_page_config(page_title="Komponen Dasar", page_icon="🧩", layout="centered")

# 1. Teks
st.title("Komponen Dasar Streamlit")
st.header("1. Menampilkan teks")
st.subheader("Subjudul")
st.markdown("Teks **tebal**, *miring*, dan `kode` dengan Markdown.")
st.caption("Caption untuk keterangan kecil.")
st.code("print('Halo')", language="python")

# 2. Data
st.header("2. Menampilkan data")
df = pd.DataFrame({"produk": ["A", "B", "C"], "penjualan": [120, 95, 143]})
st.dataframe(df, width="stretch")
st.metric(label="Total penjualan", value=f"{df['penjualan'].sum()} unit", delta="+12 vs bulan lalu")

# 3. Input widget
st.header("3. Input pengguna")
umur = st.slider("Umur", min_value=17, max_value=70, value=30)
kota = st.selectbox("Kota", ["Jakarta", "Makassar", "Medan", "Surabaya"])
minat = st.multiselect("Minat", ["Data", "Desain", "Bisnis"], default=["Data"])
tanggal = st.date_input("Tanggal mulai", dt.date(2026, 1, 1))
setuju = st.checkbox("Saya setuju dengan syarat & ketentuan")
st.write(f"Umur: {umur} | Kota: {kota} | Minat: {', '.join(minat)} | Mulai: {tanggal}")

# 4. Tombol dan status
st.header("4. Tombol & pesan status")
if st.button("Kirim"):
    if setuju:
        st.success("Data terkirim.")
    else:
        st.warning("Centang persetujuan terlebih dahulu.")
st.info("st.info untuk informasi netral.")

# 5. Grafik bawaan
st.header("5. Grafik bawaan")
rng = np.random.default_rng(0)
tren = pd.DataFrame({"hari": range(1, 31), "pengunjung": rng.integers(80, 160, 30)}).set_index("hari")
st.line_chart(tren)
```

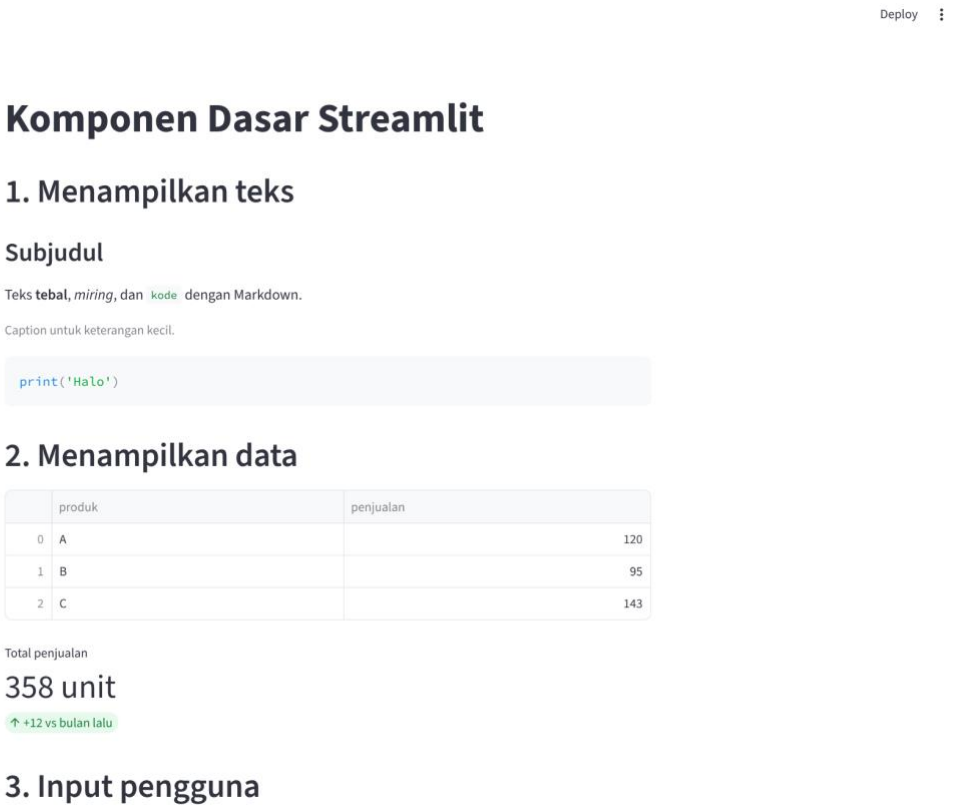
Ringkasan komponen yang dipakai (Tabel 1.2):

Tabel 1.2. *Ringkasan komponen yang dipakai*

Kelompok	Fungsi	Nilai kembali
Teks	`st.title`, `st.header`, `st.subheader`, `st.markdown`, `st.caption`, `st.code`	—
Data	`st.dataframe` (interaktif, bisa diurutkan), `st.table` (statis), `st.metric` (angka KPI + delta)	—
Input	`st.text_input`, `st.number_input`, `st.slider`, `st.selectbox`, `st.multiselect`, `st.radio`, `st.checkbox`, `st.date_input`	nilai pilihan pengguna
Aksi	`st.button`, `st.download_button`, `st.form_submit_button`	`True` hanya pada rerun saat diklik
Status	`st.success`, `st.info`, `st.warning`, `st.error`, `st.progress`, `st.spinner`	—
Grafik	`st.line_chart`, `st.bar_chart`, `st.area_chart`, `st.scatter_chart`, `st.pyplot`, `st.plotly_chart`	—

Catatan versi: parameter `width="stretch"` dipakai pada Streamlit 1.40 ke atas. Pada versi lama, gunakan `use_container_width=True`.

Checkpoint: geser slider umur dan perhatikan baris ringkasan berubah seketika. Klik **Kirim** tanpa mencentang persetujuan; kotak kuning peringatan harus muncul (lihat Gambar 1.1).



Gambar 1.1. *Tur komponen dasar Streamlit*

1.6 Praktikum C — Aplikasi sederhana: kalkulator tagihan

Aplikasi ini menggabungkan input, logika perhitungan, tombol, dan output KPI. Buat `sesi1/app_kalkulator_tagihan.py`:

```python
# sesi1/app_kalkulator_tagihan.py — PRAKTIKUM Sesi 1: aplikasi sederhana
import streamlit as st

st.set_page_config(page_title="Kalkulator Tagihan", page_icon="🧾")
st.title("🧾 Kalkulator Estimasi Tagihan Internet")
st.write("Masukkan paket dan durasi berlangganan untuk melihat estimasi biaya.")

HARGA = {"DSL": 260_000, "Fiber": 420_000, "Mobile Broadband": 310_000}
DISKON = {"Bulanan": 0.00, "Tahunan": 0.10, "Dua Tahun": 0.18}

layanan = st.selectbox("Jenis layanan", list(HARGA))
kontrak = st.radio("Jenis kontrak", list(DISKON), horizontal=True)
bulan = st.number_input("Durasi (bulan)", min_value=1, max_value=72, value=12, step=1)
ppn = st.checkbox("Tambahkan PPN 11%", value=True)

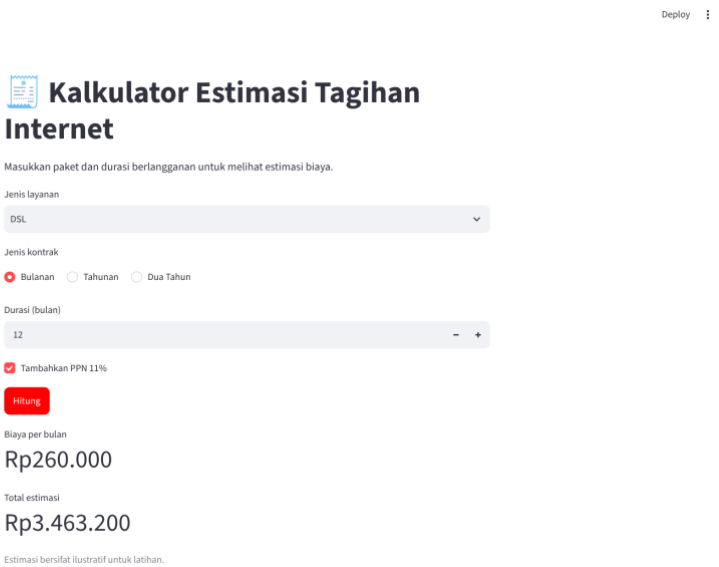
if st.button("Hitung", type="primary"):
    per_bulan = HARGA[layanan] * (1 - DISKON[kontrak])
    total = per_bulan * bulan * (1.11 if ppn else 1)
    st.metric("Biaya per bulan", f"Rp{per_bulan:,.0f}".replace(",", "."))
    st.metric("Total estimasi", f"Rp{total:,.0f}".replace(",", "."),
              delta=f"hemat {DISKON[kontrak]:.0%}" if DISKON[kontrak] else None)
    st.caption("Estimasi bersifat ilustratif untuk latihan.")
```

Jalankan dengan `streamlit run sesi1/app_kalkulator_tagihan.py`.

Penjelasan kode

- Dictionary `HARGA` dan `DISKON` memisahkan data dari logika, sehingga menambah paket cukup dengan menambah satu baris.
- `list(HARGA)` menghasilkan daftar kunci dictionary sebagai pilihan dropdown.
- `f"Rp{x:,.0f}".replace(",", ".")` mengubah 3463200 menjadi `Rp3.463.200` sesuai format Indonesia.
- Output hanya muncul setelah tombol diklik karena `st.button()` bernilai `True` pada rerun saat tombol ditekan.

Checkpoint: dengan pilihan DSL, Bulanan, 12 bulan, dan PPN dicentang, hasilnya adalah **Biaya per bulan Rp260.000** dan **Total estimasi Rp3.463.200** (lihat Gambar 1.2).



Gambar 1.2. *Kalkulator tagihan setelah tombol Hitung diklik*

Latihan mandiri Sesi 1

1. Tambahkan layanan baru "Satelit" dengan harga Rp550.000 ke kalkulator.
2. Tambahkan `st.slider` untuk diskon promosi tambahan 0–20% dan terapkan pada perhitungan.
3. Buat aplikasi "Profil Peserta" yang meminta nama, divisi (selectbox), dan tahun pengalaman (slider), lalu menampilkan ringkasan dalam `st.info`.
4. Tantangan: gunakan `st.session_state` untuk menghitung berapa kali tombol **Hitung** sudah diklik.

Kesalahan umum Sesi 1

Rinciannya disajikan pada Tabel 1.3.

Tabel 1.3. *Kesalahan umum Sesi 1*

Gejala	Penyebab	Solusi
`streamlit: command not found`	Virtual environment tidak aktif	Aktifkan `.venv`, atau jalankan `python -m streamlit run file.py`
`StreamlitAPIException: set_page_config() can only be called once`	`set_page_config` bukan perintah Streamlit pertama atau dipanggil dua kali	Letakkan tepat setelah import
Aplikasi dijalankan dengan `python app.py` dan tidak terjadi apa-apa	Streamlit harus dijalankan lewat perintah `streamlit run`	Gunakan `streamlit run sesi1/app_hello.py`
Port 8501 sudah dipakai	Aplikasi lain masih berjalan	Hentikan dengan `Ctrl + C` atau gunakan `--server.port 8502`

Gejala	Penyebab	Solusi
`DuplicateWidgetID`	Dua widget memiliki label sama	Tambahkan argumen `key="unik"` pada salah satu widget

Sesi 2 — EDA & Basic Visualization (90 menit)

Sesi ini menghasilkan analisis EDA lengkap dengan 16 visualisasi: kartu KPI, pie, donut, bar vertikal, bar horizontal, grouped bar, stacked bar 100%, histogram, box plot, violin plot, dua line chart, area chart, scatter plot, heatmap pivot, dan heatmap korelasi. Setelah itu, hasil EDA disajikan ulang sebagai aplikasi Streamlit interaktif dengan enam tab. Temuan utamanya: churn rate kontrak Bulanan 54,6%, jauh di atas Tahunan 16,8% dan Dua Tahun 9,7%.

Tujuan sesi

- Memuat data dengan pandas dan melakukan data profiling (ukuran, tipe, missing value, duplikat).
- Menghitung statistik deskriptif dan menyajikannya sebagai tabel.
- Melakukan filtering, grouping, pivot table, dan binning (`pd.cut`).
- Membuat visualisasi EDA dengan Matplotlib untuk lima tujuan: komposisi, perbandingan, distribusi, tren, dan hubungan.
- Menulis judul grafik yang menyatakan temuan, bukan sekadar nama variabel.
- Menampilkan EDA di Streamlit dengan KPI card, tab, dan chart Plotly interaktif.

2.1 Alur EDA yang dipakai

1. **Load** — baca data, pastikan kolom tanggal bertipe datetime.
2. **Profiling** — cek `shape`, `dtypes`, missing value, nilai unik, duplikat.
3. **Statistik deskriptif** — rata-rata, median, sebaran, distribusi kategori.
4. **Cleaning ringan** — tangani missing value secara eksplisit dan buat kolom bantu.
5. **Filtering & grouping** — bandingkan kelompok untuk menemukan pola.
6. **Visualisasi** — pilih jenis grafik sesuai pertanyaan.
7. **Catat temuan** — tulis insight dalam satu kalimat dengan angka.

2.2 Lima tujuan visualisasi dan grafik yang tepat

Sebelum menggambar, tentukan dulu pertanyaannya. Tabel 2.1 menjadi peta seluruh grafik di sesi ini.

Tabel 2.1. *Lima tujuan visualisasi dan grafik yang tepat*

Tujuan	Pertanyaan	Grafik yang dipakai di sesi ini
Ringkasan	Berapa angka utama saat ini?	KPI card, tabel
Komposisi	Bagaimana keseluruhan terbagi menjadi bagian?	Pie, donut, stacked bar 100%, stacked area
Perbandingan	Kelompok mana yang lebih tinggi/rendah?	Bar vertikal, bar horizontal terurut, grouped bar
Distribusi	Bagaimana sebaran nilai satu variabel?	Histogram, box plot, violin plot
Tren	Bagaimana nilai berubah dari waktu/urutan?	Line chart, area chart
Hubungan	Apakah dua variabel saling berkaitan?	Scatter plot, heatmap korelasi, heatmap pivot

> **TIPS**
> Aturan pie/donut: gunakan hanya untuk 2–5 kategori yang menjumlah 100%. Untuk lebih dari 5 kategori atau untuk membandingkan nilai yang mirip, bar chart terurut lebih mudah dibaca.

2.3 Praktikum A — Persiapan, profiling, dan statistik

Buat file `sesi2/eda_churn.py`. Bagian pertama berisi pengaturan gaya visual dan langkah 1–6 EDA. Gaya visual diatur sekali lewat `plt.rcParams` dan konstanta warna, sehingga ke-16 grafik tampil seragam.

PYTHON

```python
# sesi2/eda_churn.py — EDA lengkap dengan Pandas + Matplotlib (jalankan: python sesi2/eda_churn.py)
from pathlib import Path
import matplotlib
matplotlib.use("Agg")            # agar bisa jalan tanpa layar; hapus baris ini di Jupyter
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / "sesi2" / "output"
OUT.mkdir(exist_ok=True)

# ---------- Gaya visual konsisten untuk semua grafik ----------
BIRU, ORANYE, HIJAU, KUNING, UNGU = "#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#4a3aa7"
TEKS, TEKS2, GRID = "#0b0b0b", "#52514e", "#e6e5e0"
STATUS = {"Bertahan": BIRU, "Churn": ORANYE}
plt.rcParams.update({
    "figure.dpi": 110, "savefig.dpi": 200, "font.size": 10, "axes.titlesize": 12,
    "axes.titleweight": "bold", "axes.titlelocation": "left", "axes.spines.top": False,
    "axes.spines.right": False, "axes.edgecolor": "#b9b8b2", "axes.grid": True,
    "grid.color": GRID, "grid.linewidth": 0.8, "axes.axisbelow": True,
    "axes.labelcolor": TEKS2, "xtick.color": TEKS2, "ytick.color": TEKS2,
})
KONTRAK = ["Bulanan", "Tahunan", "Dua Tahun"]

def simpan(fig, nama):
    fig.tight_layout()
    fig.savefig(OUT / nama, bbox_inches="tight", facecolor="white")
    plt.close(fig)

# =============== 1. LOAD DATA ===============
df = pd.read_csv(ROOT / "data" / "customer_churn.csv", parse_dates=["signup_date"])
df["status"] = df["churn"].map({0: "Bertahan", 1: "Churn"})

# =============== 2. PROFILING ===============
print("Shape:", df.shape)
profil = pd.DataFrame({"tipe": df.dtypes.astype(str), "missing": df.isna().sum(),
                       "unik": df.nunique()})
print(profil)
print("Duplikat customer_id:", df["customer_id"].duplicated().sum())

# =============== 3. STATISTIK DESKRIPTIF ===============
num_cols = ["tenure_months", "monthly_charges", "total_charges", "num_complaints", "support_calls"]
print(df[num_cols].describe().T.round(1))
print("Churn rate keseluruhan: {:.1%}".format(df["churn"].mean()))

# =============== 4. CLEANING RINGAN ===============
df["total_charges"] = df["total_charges"].fillna(df["total_charges"].median())
df["kelompok_tenure"] = pd.cut(df["tenure_months"], bins=[0, 12, 24, 48, 72],
                               labels=["0-12", "13-24", "25-48", "49-72"])

# =============== 5. FILTERING ===============
berisiko = df[(df["contract_type"] == "Bulanan") & (df["num_complaints"] >= 3)]
print(f"Bulanan & keluhan>=3: {len(berisiko)} pelanggan, churn {berisiko['churn'].mean():.1%}")
print("Fiber di Jawa:", len(df.query("region == 'Jawa' and internet_service == 'Fiber'")))

# =============== 6. GROUPING ===============
churn_kontrak = (df.groupby("contract_type")
                   .agg(pelanggan=("customer_id", "count"), churn_rate=("churn", "mean"),
                        rata2_tagihan=("monthly_charges", "mean"))
                   .reindex(KONTRAK))
print(churn_kontrak.round(3))
pivot = pd.pivot_table(df, values="churn", index="region", columns="segment", aggfunc="mean")
print(pivot.round(3))
```

Jalankan dengan `python sesi2/eda_churn.py`. Hasil profiling yang dicetak di terminal dirangkum dalam Tabel 2.2.

Tabel 2.2. *Praktikum A — Persiapan, profiling, dan statistik*

Kolom	Tipe	Missing	Nilai unik
customer_id	teks	0	3.000
signup_date	datetime	0	1.017
region	teks	0	5
segment	teks	0	3
contract_type	teks	0	3
payment_method	teks	0	4
internet_service	teks	0	3
tenure_months	integer	0	71
monthly_charges	float	0	652
total_charges	float	**45**	2.770
num_complaints	integer	0	8
support_calls	integer	0	13
churn	integer (0/1)	0	2

Statistik deskriptif variabel numerik (Tabel 2.3):

Tabel 2.3. *Statistik deskriptif variabel numerik*

Variabel	Rata-rata	Std	Min	Q1	Median	Q3	Maks
tenure_months (bulan)	26,5	16,3	1	13	25	35	71
monthly_charges (ribu Rp)	424,0	164,9	160	304	393	497	1.353
total_charges (juta Rp)	11,3	8,7	0,2	5,1	9,2	15,2	59,5
num_complaints	1,1	1,1	0	0	1	2	7
support_calls	2,4	1,8	0	1	2	3	13

Hasil filtering dan grouping yang dicetak (Tabel 2.4):

Tabel 2.4. *Hasil filtering dan grouping yang dicetak*

Analisis	Hasil
Kontrak Bulanan dengan keluhan ≥ 3	171 pelanggan, churn rate 71,9%
Pelanggan Fiber di Jawa	607 pelanggan
Churn rate kontrak Bulanan	54,6% (1.541 pelanggan)
Churn rate kontrak Tahunan	16,8% (923 pelanggan)
Churn rate kontrak Dua Tahun	9,7% (536 pelanggan)

Penjelasan kode

- `Path(__file__).resolve().parents[1]` menunjuk ke folder akar proyek, sehingga script menemukan `data/` dari mana pun ia dijalankan.
- `plt.rcParams.update({...})` mengatur gaya global: judul rata kiri dan tebal, tanpa garis bingkai atas/kanan, grid tipis di belakang data. Satu pengaturan ini membuat semua grafik terlihat profesional tanpa mengulang kode.

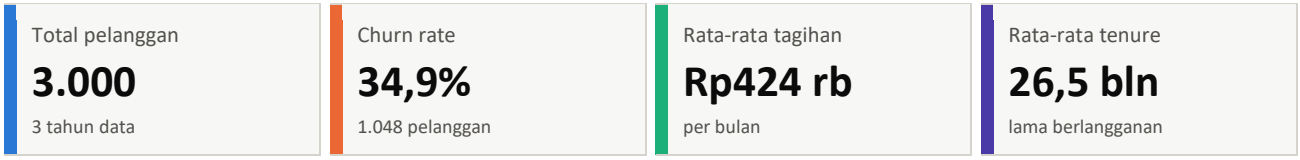
- Konstanta `BIRU`, `ORANYE`, dan seterusnya, serta kamus `STATUS`, mengunci warna: Bertahan selalu biru dan Churn selalu oranye di semua grafik.
- Fungsi `simpan()` menyimpan grafik dengan resolusi 200 dpi (tajam saat dicetak) lalu menutup figure agar memori tidak menumpuk.
- `profil` adalah DataFrame kecil yang menggabungkan tipe, jumlah missing, dan jumlah nilai unik per kolom; cara cepat melihat kualitas data dalam satu tabel.
- `pd.cut()` mengubah tenure numerik menjadi kelompok (0–12, 13–24, 25–48, 49–72 bulan) yang dipakai di line chart 2.4.12.
- `.reindex(KONTRAK)` mengurutkan hasil groupby sesuai urutan logis (Bulanan → Tahunan → Dua Tahun), bukan urutan abjad.

2.4 Galeri visualisasi EDA

Setiap subbab 2.4.x berikut menampilkan potongan kode bagian 7.x yang bernomor sama di `eda_churn.py`, grafik yang dihasilkan, dan insight yang dapat ditarik. Semua grafik tersimpan di `sesi2/output/`.

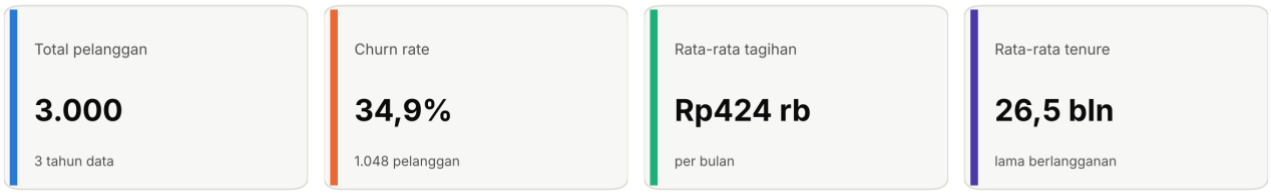
2.4.1 KPI cards — ringkasan angka utama

KPI card menjawab "berapa angka utamanya?" dalam satu pandangan. Di laporan, letakkan KPI card di bagian paling atas.




```python
# 7.1 KPI cards — ringkasan angka utama
kpi = [("Total pelanggan", f"{len(df):,}".replace(",", "."), "3 tahun data"),
       ("Churn rate", f"{df.churn.mean():.1%}".replace(".", ","), f"{df.churn.sum():,}
pelanggan".replace(",", ".")),
       ("Rata-rata tagihan", f"Rp{df.monthly_charges.mean()/1000:,.0f} rb", "per bulan"),
       ("Rata-rata tenure", f"{df.tenure_months.mean():.1f} bln".replace(".", ","), "lama berlangganan")]
fig, axes = plt.subplots(1, 4, figsize=(12, 2.1))
for ax, (judul, nilai, sub), warna in zip(axes, kpi, [BIRU, ORANYE, HIJAU, UNGU]):
    ax.axis("off")
    ax.add_patch(FancyBboxPatch((0.02, 0.05), 0.96, 0.9, boxstyle="round,pad=0.02,rounding_size=0.06",
                                transform=ax.transAxes, facecolor="#f7f7f5", edgecolor="#dcdbd5"))
    ax.add_patch(FancyBboxPatch((0.02, 0.05), 0.025, 0.9, boxstyle="square,pad=0",
                                transform=ax.transAxes, facecolor=warna, edgecolor="none"))
    ax.text(0.1, 0.72, judul, fontsize=10, color=TEKS2, transform=ax.transAxes)
    ax.text(0.1, 0.38, nilai, fontsize=20, fontweight="bold", color=TEKS, transform=ax.transAxes)
    ax.text(0.1, 0.15, sub, fontsize=9, color=TEKS2, transform=ax.transAxes)
simpan(fig, "01_kpi_cards.png")
```

Hasilnya ditunjukkan pada Gambar 2.1.



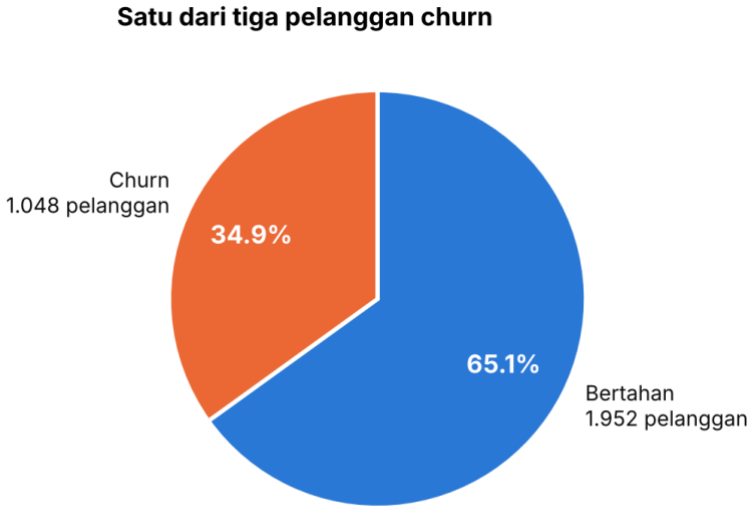
Gambar 2.1. *KPI cards hasil Matplotlib*

`FancyBboxPatch` menggambar kotak bersudut tumpul; garis tipis berwarna di sisi kiri memberi identitas tiap kartu. Teks ditempatkan dengan koordinat relatif sumbu (`transform=ax.transAxes`, 0–1), sehingga posisi tidak bergantung pada data.

2.4.2 Pie chart — proporsi status churn

```
PYTHON
# 7.2 Pie chart — proporsi status churn
cnt = df["status"].value_counts().reindex(["Bertahan", "Churn"])
fig, ax = plt.subplots(figsize=(5, 4))
wedges, labels, persen = ax.pie(
    cnt, labels=[f"{k}\n{v:,} pelanggan".replace(",", ".") for k, v in cnt.items()],
    colors=[STATUS[k] for k in cnt.index], autopct="%1.1f%%", startangle=90,
    counterclock=False, pctdistance=0.68, labeldistance=1.12,
    wedgeprops=dict(edgecolor="white", linewidth=2), textprops=dict(color=TEKS))
for t in persen:                        # angka persen di dalam irisan: putih & tebal
    t.set_color("white"); t.set_fontweight("bold"); t.set_fontsize(12)
ax.set_title("Satu dari tiga pelanggan churn")
simpan(fig, "02_pie_status_churn.png")
```

Hasilnya ditunjukkan pada Gambar 2.2.



Gambar 2.2. *Pie chart status churn*

Insight: 34,9% pelanggan (1.048 orang) churn. Satu dari tiga pelanggan berhenti berlangganan, angka yang tinggi untuk bisnis berlangganan.

`autopct="%1.1f%%"` menampilkan persentase di dalam irisan; `ax.pie()` mengembalikan tiga objek (irisan, label, persen) sehingga teks persen dapat diberi warna putih tebal agar kontras.

2.4.3 Donut chart — komposisi pelanggan per kontrak

```
PYTHON
# 7.3 Donut chart — komposisi pelanggan per jenis kontrak
kc = df["contract_type"].value_counts().reindex(KONTRAK)
fig, ax = plt.subplots(figsize=(5, 4))
ax.pie(kc, colors=[BIRU, HIJAU, KUNING], startangle=90, counterclock=False,
       wedgeprops=dict(width=0.38, edgecolor="white", linewidth=2))
ax.text(0, 0.06, f"{len(df):,}".replace(",", "."), ha="center", fontsize=18, fontweight="bold")
ax.text(0, -0.16, "pelanggan", ha="center", color=TEKS2)
ax.legend([f"{k}  {v/len(df):.0%}" for k, v in kc.items()], loc="center left",
          bbox_to_anchor=(0.95, 0.5), frameon=False)
ax.set_title("Separuh pelanggan berkontrak bulanan")
simpan(fig, "03_donut_kontrak.png")
```

Hasilnya ditunjukkan pada Gambar 2.3.



Gambar 2.3. *Donut chart komposisi kontrak*

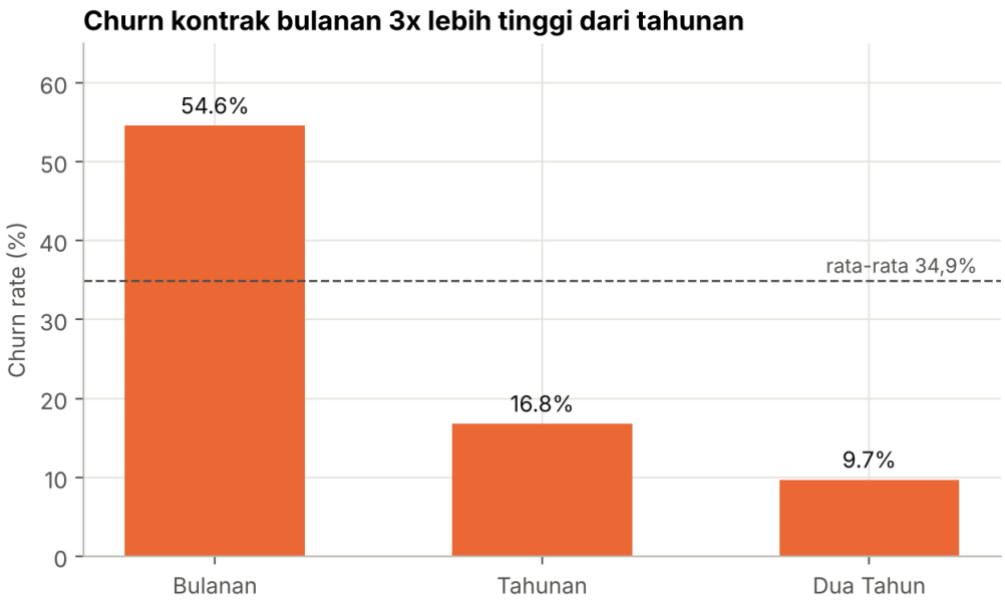
Insight: 51% pelanggan berkontrak Bulanan, 31% Tahunan, dan 18% Dua Tahun. Karena kontrak Bulanan paling berisiko (lihat 2.4.4), separuh basis pelanggan berada di kelompok rawan.

Donut adalah pie dengan lubang (`wedgeprops=dict(width=0.38)`). Ruang di tengah dipakai untuk angka total, sehingga satu grafik menyampaikan komposisi sekaligus jumlah.

2.4.4 Bar chart vertikal dengan garis rata-rata

```python
# 7.4 Bar chart vertikal + garis rata-rata
fig, ax = plt.subplots(figsize=(6.5, 4))
bars = ax.bar(churn_kontrak.index, churn_kontrak["churn_rate"] * 100, color=ORANYE, width=0.55)
ax.bar_label(bars, fmt="%.1f%%", padding=3, color=TEKS)
ax.axhline(df.churn.mean() * 100, color=TEKS2, linestyle="--", linewidth=1)
ax.text(2.33, df.churn.mean() * 100 + 1, "rata-rata 34,9%", ha="right", color=TEKS2, fontsize=9)
ax.set(ylabel="Churn rate (%)", ylim=(0, 65))
ax.set_title("Churn kontrak bulanan 3x lebih tinggi dari tahunan")
simpan(fig, "04_bar_churn_kontrak.png")
```

Hasilnya ditunjukkan pada Gambar 2.4.



Gambar 2.4. *Bar chart churn rate per kontrak*

Insight: churn kontrak Bulanan 54,6%, lebih dari 3 kali kontrak Tahunan (16,8%) dan hampir 6 kali kontrak Dua Tahun (9,7%). Hanya kontrak Bulanan yang berada di atas rata-rata 34,9%.

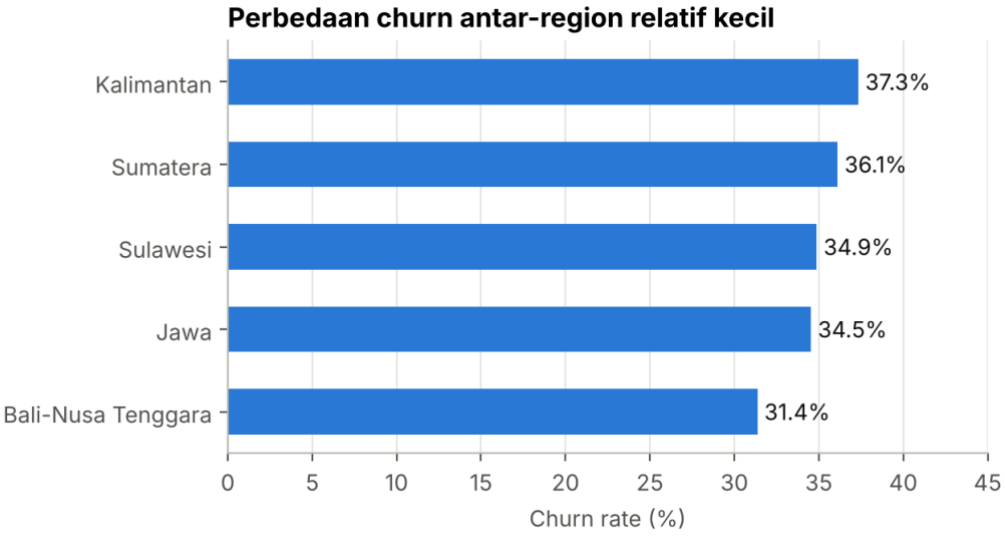
`ax.bar_label()` menulis nilai di atas batang; `ax.axhline()` menambah garis referensi rata-rata. Keduanya membuat pembaca tidak perlu menebak angka dari sumbu.

2.4.5 Bar chart horizontal terurut — region

PYTHON

```
# 7.5 Bar chart horizontal terurut — region
reg = df.groupby("region")["churn"].mean().sort_values() * 100
fig, ax = plt.subplots(figsize=(6.5, 3.6))
bars = ax.barh(reg.index, reg.values, color=BIRU, height=0.55)
ax.bar_label(bars, fmt="%.1f%%", padding=3, color=TEKS)
ax.set(xlabel="Churn rate (%)", xlim=(0, 45))
ax.grid(axis="y", visible=False)
ax.set_title("Perbedaan churn antar-region relatif kecil")
simpan(fig, "05_barh_churn_region.png")
```

Hasilnya ditunjukkan pada Gambar 2.5.



Gambar 2.5. *Bar chart horizontal churn per region*

Insight: churn per region hanya berkisar 31,4% (Bali-Nusa Tenggara) sampai 37,3% (Kalimantan). Region bukan pembeda utama; perhatian lebih baik difokuskan pada jenis kontrak.

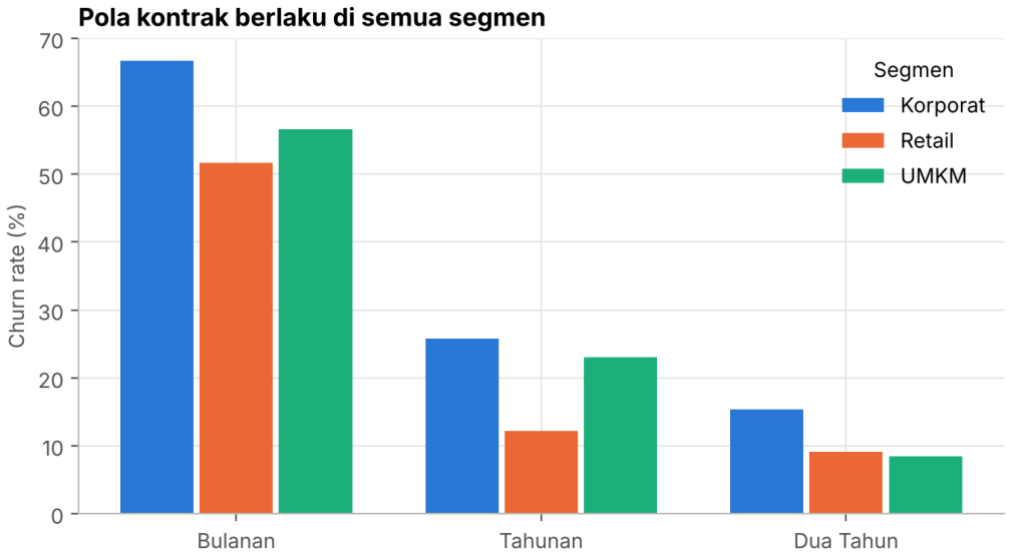
Bar horizontal cocok untuk label kategori yang panjang. Selalu urutkan (`sort_values()`) agar peringkat langsung terbaca.

2.4.6 Grouped bar — segmen × kontrak

PYTHON

```
# 7.6 Grouped bar — segmen x kontrak
gb = df.pivot_table(values="churn", index="contract_type", columns="segment",
aggfunc="mean").reindex(KONTRAK) * 100
fig, ax = plt.subplots(figsize=(7, 4))
x = np.arange(len(gb.index)); w = 0.26
for i, (seg, warna) in enumerate(zip(gb.columns, [BIRU, ORANYE, HIJAU])):
    ax.bar(x + (i - 1) * w, gb[seg], width=w - 0.02, color=warna, label=seg)
ax.set_xticks(x, gb.index); ax.set_ylabel("Churn rate (%)")
ax.legend(title="Segmen", frameon=False)
ax.set_title("Pola kontrak berlaku di semua segmen")
simpan(fig, "06_grouped_bar_segmen_kontrak.png")
```

Hasilnya ditunjukkan pada Gambar 2.6.



Gambar 2.6. *Grouped bar churn rate per segmen dan kontrak*

Insight: di ketiga segmen, kontrak Bulanan selalu paling tinggi. Segmen Korporat memiliki churn tertinggi di setiap jenis kontrak, sejalan dengan tagihannya yang lebih besar.

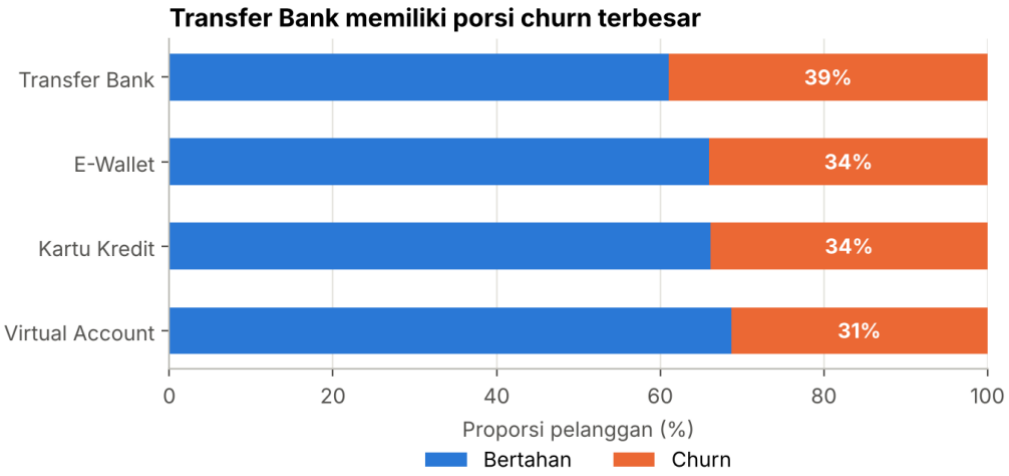
Grouped bar dibuat dengan menggeser posisi x setiap seri (`x + (i - 1) * w`). Jumlah seri sebaiknya tidak lebih dari 3–4 agar tetap terbaca.

2.4.7 Stacked bar 100% — komposisi status per metode pembayaran

PYTHON

```
# 7.7 Stacked bar 100% — komposisi status per metode pembayaran
sb = pd.crosstab(df["payment_method"], df["status"], normalize="index").sort_values("Churn") * 100
fig, ax = plt.subplots(figsize=(7, 3.6))
ax.barh(sb.index, sb["Bertahan"], color=BIRU, label="Bertahan", height=0.55)
ax.barh(sb.index, sb["Churn"], left=sb["Bertahan"], color=ORANYE, label="Churn", height=0.55)
for i, v in enumerate(sb["Churn"]):
    ax.text(100 - v / 2, i, f"{v:.0f}%", ha="center", va="center", color="white", fontweight="bold")
ax.set(xlabel="Proporsi pelanggan (%)", xlim=(0, 100)); ax.grid(axis="y", visible=False)
ax.legend(ncol=2, loc="upper center", bbox_to_anchor=(0.5, -0.18), frameon=False)
ax.set_title("Transfer Bank memiliki porsi churn terbesar")
simpan(fig, "07_stacked_bar_pembayaran.png")
```

Hasilnya ditunjukkan pada Gambar 2.7.



Gambar 2.7. *Stacked bar 100% status per metode pembayaran*

Insight: porsi churn pada Transfer Bank 39%, tertinggi dibanding E-Wallet dan Kartu Kredit (34%) serta Virtual Account (31%). Pembayaran manual kemungkinan menambah friksi.

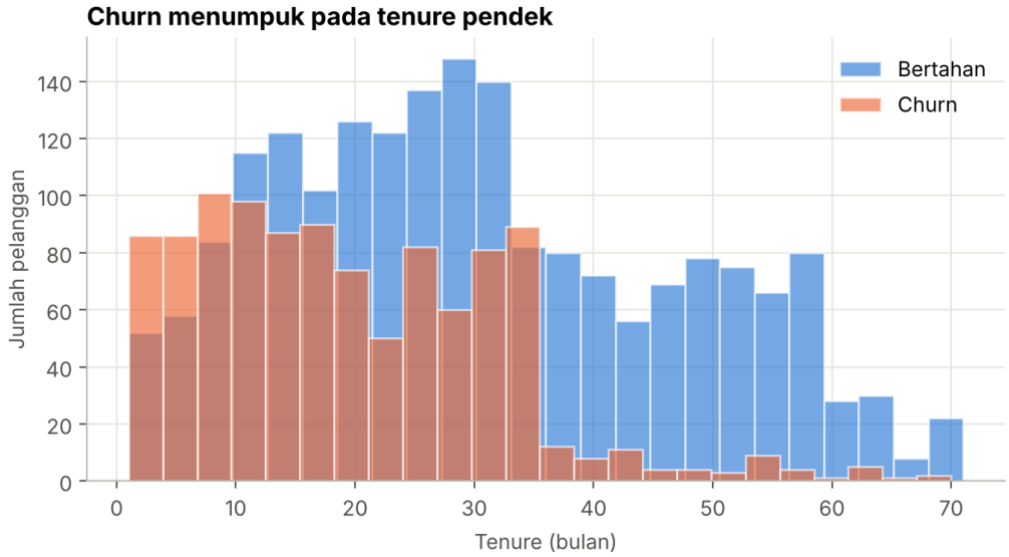
`pd.crosstab(..., normalize="index")` menghasilkan proporsi per baris yang menjumlah 1. Batang kedua digambar mulai dari ujung batang pertama dengan argumen `left=`.

2.4.8 Histogram overlay — distribusi tenure per status

PYTHON

```
# 7.8 Histogram overlay — distribusi tenure per status
fig, ax = plt.subplots(figsize=(7, 4))
for s in ["Bertahan", "Churn"]:
    ax.hist(df.loc[df.status == s, "tenure_months"], bins=24, alpha=0.65, color=STATUS[s],
            label=s, edgecolor="white")
ax.set(xlabel="Tenure (bulan)", ylabel="Jumlah pelanggan"); ax.legend(frameon=False)
ax.set_title("Churn menumpuk pada tenure pendek")
simpan(fig, "08_hist_tenure_status.png")
```

Hasilnya ditunjukkan pada Gambar 2.8.



Gambar 2.8. *Histogram tenure per status*

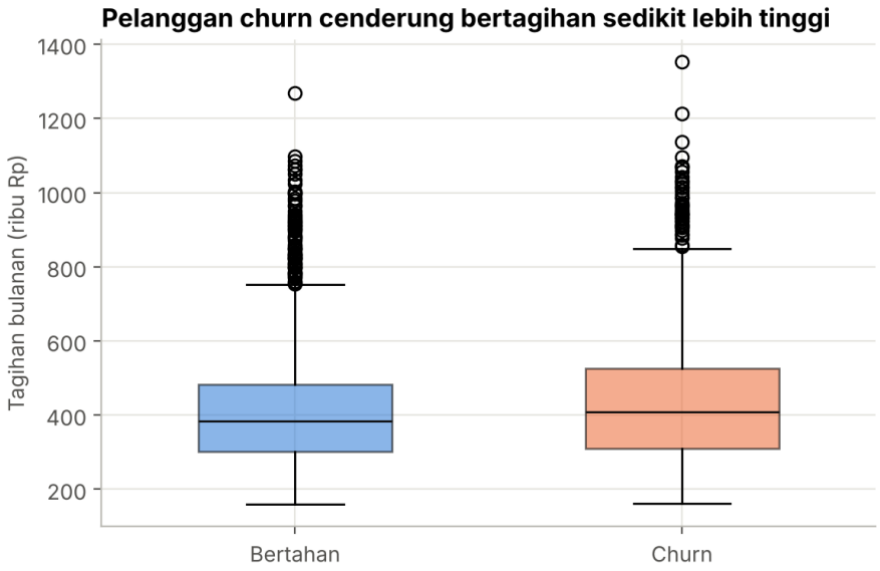
Insight: pelanggan churn menumpuk pada tenure di bawah 36 bulan; setelah itu jumlahnya sangat sedikit. Masa awal berlangganan adalah periode paling rawan.

Dua histogram ditumpuk dengan transparansi (`alpha=0.65`) agar keduanya terlihat. Jumlah `bins` memengaruhi bentuk; coba 12, 24, dan 48 untuk melihat perbedaannya.

2.4.9 Box plot — tagihan per status

PYTHON
```
# 7.9 Box plot — tagihan per status
fig, ax = plt.subplots(figsize=(6, 4))
data_box = [df.loc[df.status == s, "monthly_charges"] / 1000 for s in ["Bertahan", "Churn"]]
bp = ax.boxplot(data_box, tick_labels=["Bertahan", "Churn"], patch_artist=True, widths=0.5,
                medianprops=dict(color=TEKS))
for patch, s in zip(bp["boxes"], ["Bertahan", "Churn"]):
    patch.set_facecolor(STATUS[s]); patch.set_alpha(0.55)
ax.set(ylabel="Tagihan bulanan (ribu Rp)")
ax.set_title("Pelanggan churn cenderung bertagihan sedikit lebih tinggi")
simpan(fig, "09_box_tagihan_status.png")
```

Hasilnya ditunjukkan pada Gambar 2.9.



Gambar 2.9. *Box plot tagihan per status*

Insight: median tagihan pelanggan churn Rp409 ribu, sedikit di atas pelanggan bertahan (Rp385 ribu). Titik di atas kumis adalah outlier: pelanggan bertagihan sangat tinggi, umumnya segmen Korporat.

Cara membaca box plot: garis tengah = median, kotak = kuartil 1 sampai kuartil 3 (50% data di tengah), kumis = rentang data normal, titik = outlier.

2.4.10 Violin plot — sebaran tenure per kontrak

PYTHON
```
# 7.10 Violin plot — sebaran tenure per kontrak
fig, ax = plt.subplots(figsize=(6.5, 4))
vp = ax.violinplot([df.loc[df.contract_type == k, "tenure_months"] for k in KONTRAK],
                   showmedians=True)
for body, warna in zip(vp["bodies"], [BIRU, HIJAU, KUNING]):
    body.set_facecolor(warna); body.set_alpha(0.6)
for key in ("cmedians", "cmins", "cmaxes", "cbars"):
    vp[key].set_color(TEKS2)
ax.set_xticks([1, 2, 3], KONTRAK); ax.set_ylabel("Tenure (bulan)")
ax.set_title("Kontrak panjang identik dengan tenure panjang")
simpan(fig, "10_violin_tenure_kontrak.png")
```

Hasilnya ditunjukkan pada Gambar 2.10.



Gambar 2.10. *Violin plot tenure per kontrak*

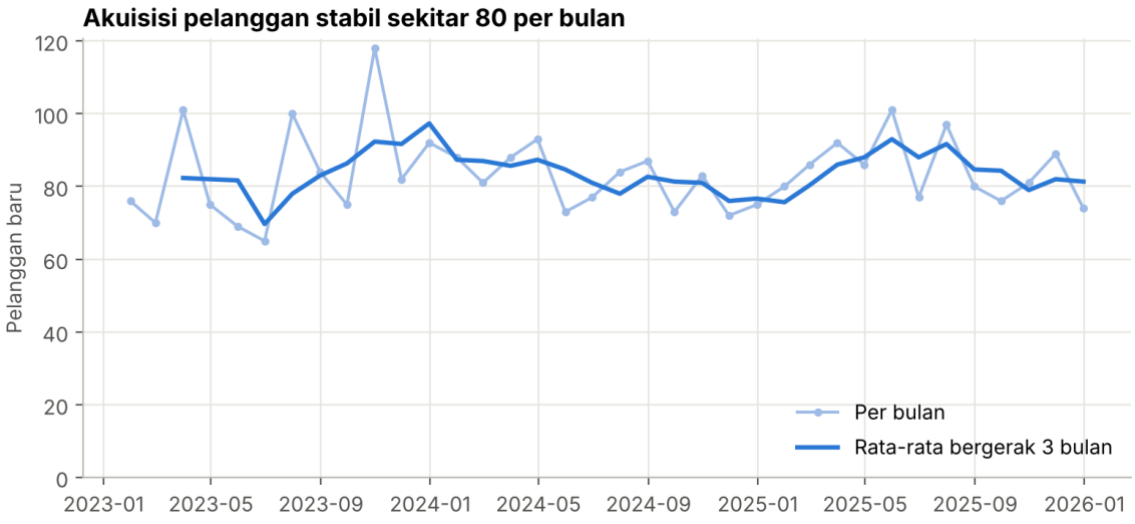
Insight: kontrak Bulanan berisi pelanggan bertenure 1–35 bulan, sedangkan kontrak Dua Tahun bertenure 12–71 bulan. Violin memperlihatkan bentuk sebaran yang tidak terlihat di box plot.

Violin plot adalah gabungan box plot dan kurva kepadatan: semakin lebar bagian violin, semakin banyak data pada nilai tersebut.

2.4.11 Line chart — pelanggan baru per bulan

```python
# 7.11 Line chart — pelanggan baru per bulan + rata-rata bergerak 3 bulan
bulanan = df.set_index("signup_date").resample("ME")["customer_id"].count()
fig, ax = plt.subplots(figsize=(8, 3.8))
ax.plot(bulanan.index, bulanan.values, color="#9dbbe6", linewidth=1.5, marker="o", markersize=3,
        label="Per bulan")
ax.plot(bulanan.index, bulanan.rolling(3).mean(), color=BIRU, linewidth=2.2,
        label="Rata-rata bergerak 3 bulan")
ax.set(ylabel="Pelanggan baru", ylim=(0, None)); ax.legend(frameon=False, loc="lower right")
ax.set_title("Akuisisi pelanggan stabil sekitar 80 per bulan")
simpan(fig, "11_line_signup.png")
```

Hasilnya ditunjukkan pada Gambar 2.11.



Gambar 2.11. *Line chart pelanggan baru per bulan*

Insight: akuisisi stabil sekitar 80 pelanggan per bulan tanpa tren naik atau turun yang jelas. Lonjakan sesaat (lebih dari 110 pelanggan) terjadi pada akhir 2023.

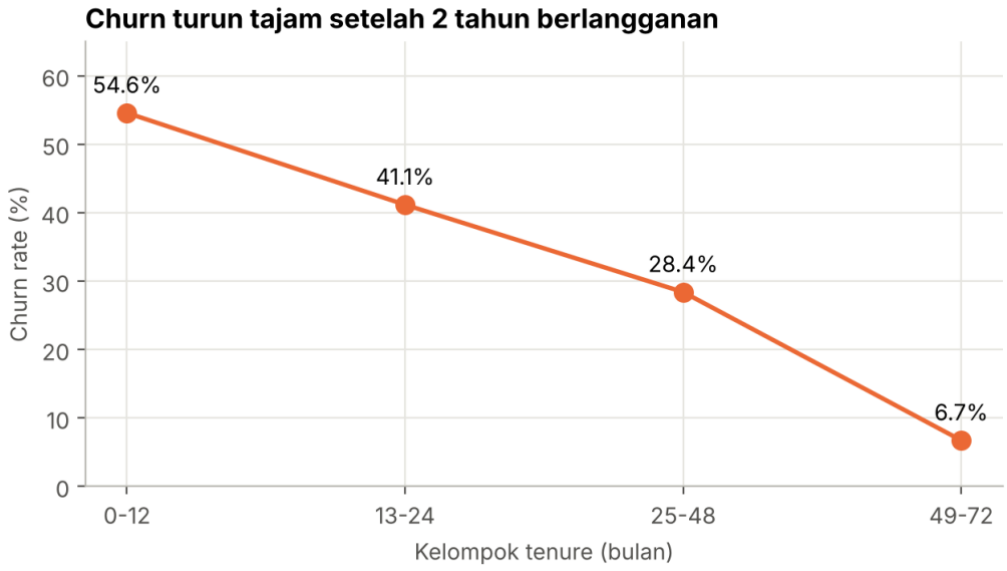
`resample("ME")` menghitung jumlah per akhir bulan. `rolling(3).mean()` menghaluskan fluktuasi sehingga tren lebih mudah dibaca; garis asli tetap ditampilkan tipis sebagai konteks.

2.4.12 Line chart — churn rate per kelompok tenure

PYTHON

```
# 7.12 Line chart — churn rate per kelompok tenure
kt = df.groupby("kelompok_tenure", observed=True)["churn"].mean() * 100
fig, ax = plt.subplots(figsize=(6.5, 3.8))
ax.plot(kt.index.astype(str), kt.values, color=ORANYE, marker="o", markersize=8, linewidth=2)
for xi, yi in zip(kt.index.astype(str), kt.values):
    ax.annotate(f"{yi:.1f}%", (xi, yi), textcoords="offset points", xytext=(0, 9), ha="center")
ax.set(xlabel="Kelompok tenure (bulan)", ylabel="Churn rate (%)", ylim=(0, 65))
ax.set_title("Churn turun tajam setelah 2 tahun berlangganan")
simpan(fig, "12_line_churn_tenure.png")
```

Hasilnya ditunjukkan pada Gambar 2.12.



Gambar 2.12. *Line chart churn rate per kelompok tenure*

Insight: churn turun dari 54,6% (0–12 bulan) menjadi 41,1%, 28,4%, lalu 6,7% (49–72 bulan). Pelanggan yang bertahan lebih dari 2 tahun hampir tidak pernah churn.

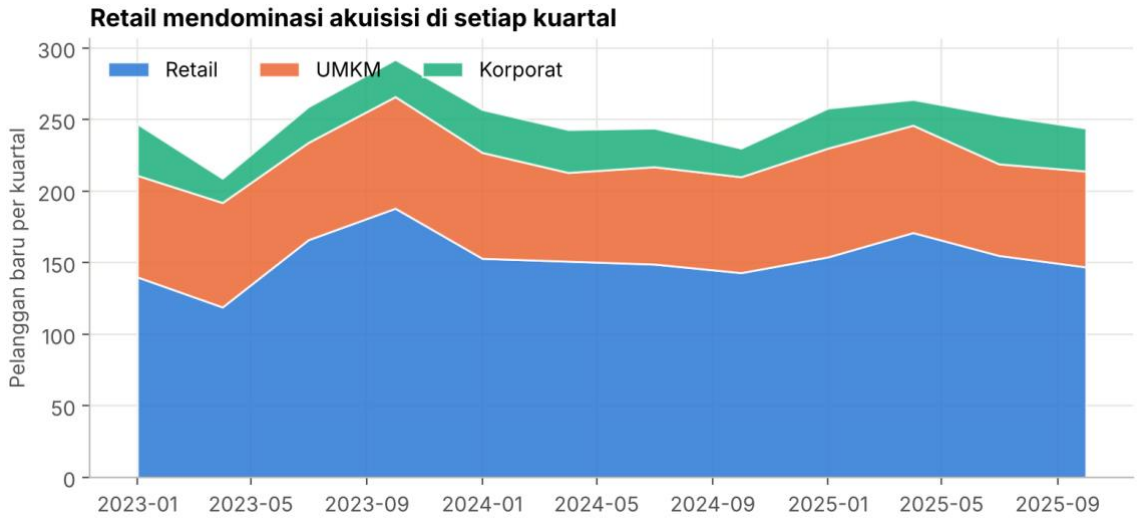
`ax.annotate()` menulis nilai di atas setiap titik. Untuk data berurutan (kelompok tenure), line chart memperlihatkan arah perubahan lebih jelas daripada bar.

2.4.13 Stacked area — pelanggan baru per kuartal per segmen

PYTHON

```
# 7.13 Stacked area — pelanggan baru per kuartal per segmen
area = (df.assign(kuartal=df.signup_date.dt.to_period("Q").dt.to_timestamp())
          .pivot_table(index="kuartal", columns="segment", values="customer_id", aggfunc="count"))
area = area[["Retail", "UMKM", "Korporat"]]
fig, ax = plt.subplots(figsize=(8, 3.8))
ax.stackplot(area.index, area.T.values, labels=area.columns, colors=[BIRU, ORANYE, HIJAU],
             alpha=0.85, edgecolor="white", linewidth=1)
ax.set(ylabel="Pelanggan baru per kuartal"); ax.legend(loc="upper left", ncol=3, frameon=False)
ax.set_title("Retail mendominasi akuisisi di setiap kuartal")
simpan(fig, "13_area_segmen.png")
```

Hasilnya ditunjukkan pada Gambar 2.13.



Gambar 2.13. *Area chart pelanggan baru per kuartal per segmen*

Insight: Retail menyumbang sekitar 60% pelanggan baru di setiap kuartal; Korporat hanya sekitar 11%. Komposisi akuisisi stabil sepanjang 2023–2025.

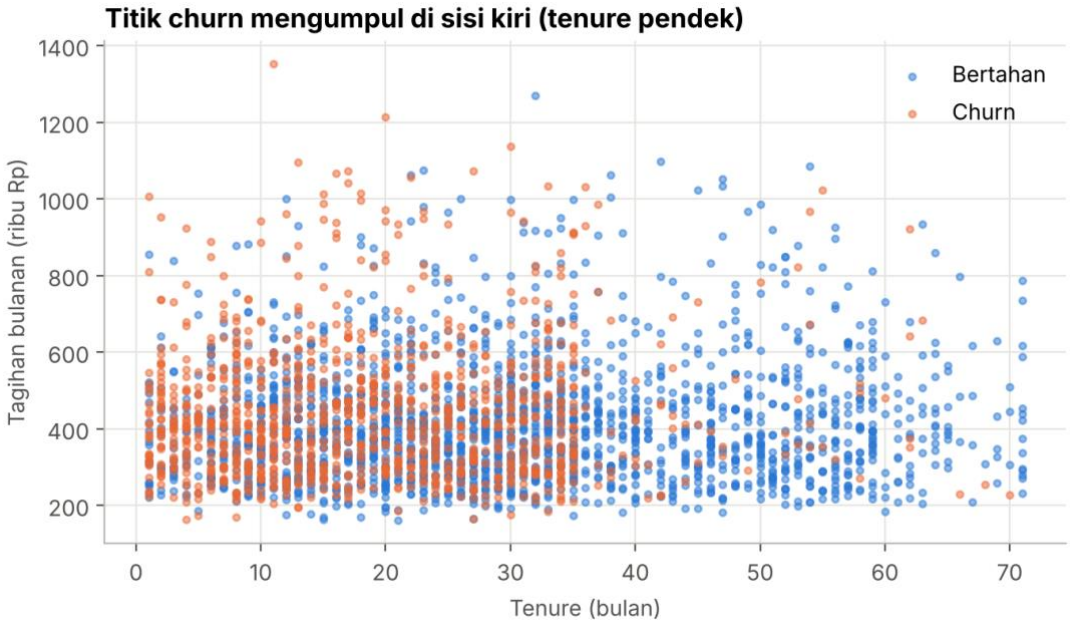
`ax.stackplot()` menumpuk beberapa seri sehingga tinggi total = jumlah semua segmen, dan ketebalan tiap lapisan = kontribusi segmen.

2.4.14 Scatter plot — tenure vs tagihan

PYTHON

```
# 7.14 Scatter — tenure vs tagihan, warna = status
fig, ax = plt.subplots(figsize=(7, 4.2))
for s in ["Bertahan", "Churn"]:
    d = df[df.status == s]
    ax.scatter(d["tenure_months"], d["monthly_charges"] / 1000, s=9, alpha=0.5,
               color=STATUS[s], label=s)
ax.set(xlabel="Tenure (bulan)", ylabel="Tagihan bulanan (ribu Rp)"); ax.legend(frameon=False)
ax.set_title("Titik churn mengumpul di sisi kiri (tenure pendek)")
simpan(fig, "14_scatter_tenure_tagihan.png")
```

Hasilnya ditunjukkan pada Gambar 2.14.



Gambar 2.14. *Scatter plot tenure vs tagihan*

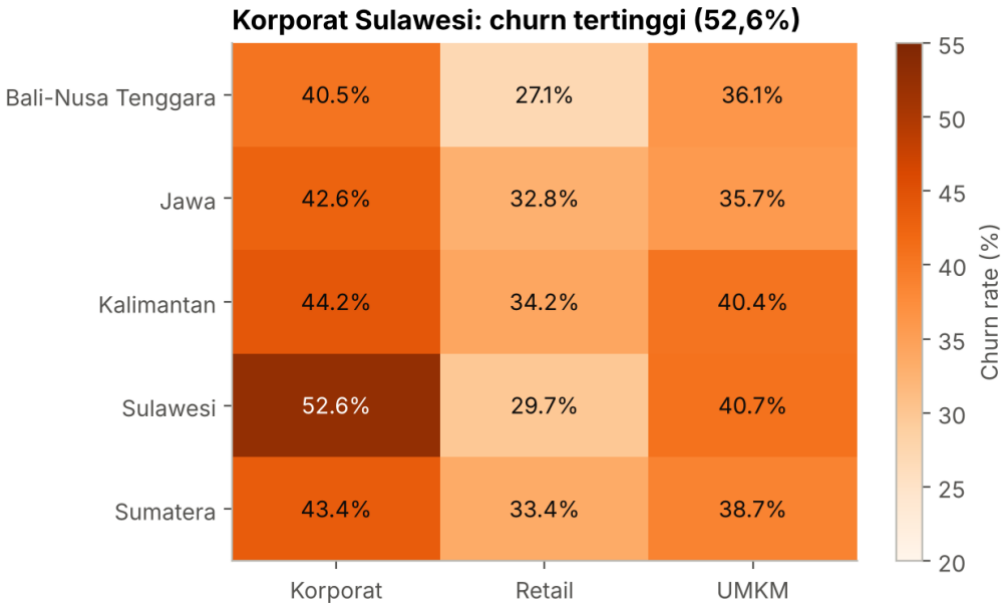
Insight: titik oranye (churn) mengumpul di sisi kiri (tenure < 36 bulan) di semua tingkat tagihan. Tenure lebih membedakan churn daripada besar tagihan.

Gunakan titik kecil (`s=9`) dan transparansi (`alpha=0.5`) agar 3.000 titik yang saling menumpuk tetap terbaca.

2.4.15 Heatmap — churn rate region × segmen

```python
# 7.15 Heatmap — churn rate region x segmen
fig, ax = plt.subplots(figsize=(6.5, 4))
im = ax.imshow(pivot.values * 100, cmap="Oranges", aspect="auto", vmin=20, vmax=55)
ax.set_xticks(range(len(pivot.columns)), pivot.columns)
ax.set_yticks(range(len(pivot.index)), pivot.index); ax.grid(False)
for i in range(pivot.shape[0]):
    for j in range(pivot.shape[1]):
        v = pivot.values[i, j] * 100
        ax.text(j, i, f"{v:.1f}%", ha="center", va="center", color="white" if v > 45 else TEKS)
fig.colorbar(im, ax=ax, label="Churn rate (%)")
ax.set_title("Korporat Sulawesi: churn tertinggi (52,6%)")
simpan(fig, "15_heatmap_region_segmen.png")
```

Hasilnya ditunjukkan pada Gambar 2.15.



Gambar 2.15. *Heatmap churn rate region x segmen*

Insight: Korporat di Sulawesi memiliki churn tertinggi (52,6%), sedangkan Retail di Bali-Nusa Tenggara terendah (27,1%). Heatmap memperlihatkan kombinasi dua dimensi yang tidak terlihat pada grafik satu dimensi.

`ax.imshow()` mewarnai sel matriks; teks di setiap sel ditulis dengan dua loop. Warna teks berganti putih pada sel gelap agar tetap kontras.

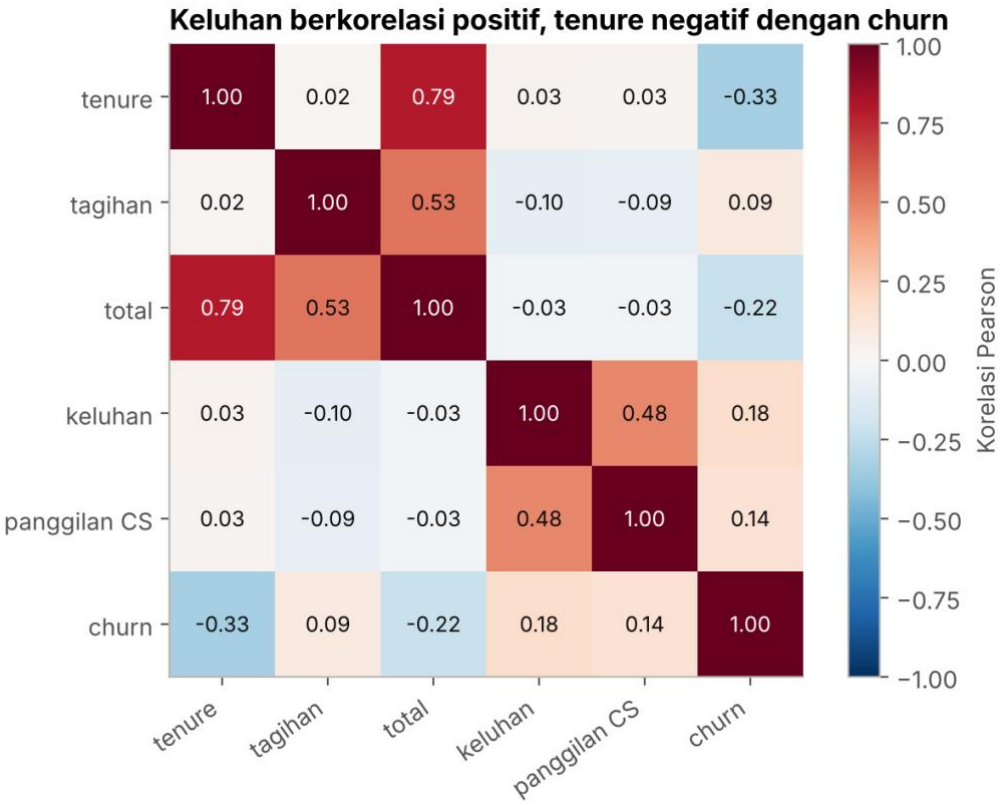
2.4.16 Heatmap korelasi

```python
# 7.16 Heatmap korelasi — hubungan antar-variabel numerik
corr = df[num_cols + ["churn"]].corr()
fig, ax = plt.subplots(figsize=(6.5, 5))
im = ax.imshow(corr, cmap="RdBu_r", vmin=-1, vmax=1)
lbl = ["tenure", "tagihan", "total", "keluhan", "panggilan CS", "churn"]
ax.set_xticks(range(len(lbl)), lbl, rotation=35, ha="right"); ax.set_yticks(range(len(lbl)), lbl)
ax.grid(False)
for i in range(len(lbl)):
    for j in range(len(lbl)):
        ax.text(j, i, f"{corr.values[i, j]:.2f}", ha="center", va="center",
                color="white" if abs(corr.values[i, j]) > 0.6 else TEKS, fontsize=9)
fig.colorbar(im, ax=ax, label="Korelasi Pearson")
```

```
ax.set_title("Keluhan berkorelasi positif, tenure negatif dengan churn")
simpan(fig, "16_heatmap_korelasi.png")

print("\n16 grafik tersimpan di", OUT)
```

Hasilnya ditunjukkan pada Gambar 2.16.



Gambar 2.16. *Heatmap korelasi variabel numerik*

Insight: tenure berkorelasi negatif dengan churn (−0,33), sedangkan jumlah keluhan (+0,18) dan panggilan CS (+0,14) berkorelasi positif. Korelasi tenure–total tagihan (0,79) wajar karena total tagihan = tagihan × tenure.

Skala warna divergen (`RdBu_r`, −1 sampai +1) dipakai karena korelasi memiliki arah: merah positif, biru negatif, putih mendekati nol.

> **PERHATIAN**
> Korelasi bukan sebab-akibat. Korelasi Pearson hanya mengukur hubungan linier antara variabel numerik; untuk variabel kategori gunakan perbandingan grup seperti pada 2.4.4–2.4.7.

2.5 Praktikum B — Dashboard EDA interaktif di Streamlit

Grafik Matplotlib di atas bersifat statis. Di dashboard, grafik sebaiknya interaktif: pengguna dapat mengarahkan kursor, memilih variabel, dan berpindah tab. Aplikasi berikut menyusun EDA menjadi KPI card dan enam tab (Komposisi, Perbandingan, Distribusi, Tren, Hubungan, Data) memakai Plotly Express, yang dibahas lebih dalam di Sesi 6.

Buat `sesi2/app_eda.py`:

PYTHON

```
# sesi2/app_eda.py — EDA interaktif di Streamlit dengan beragam visualisasi Plotly
from pathlib import Path
import pandas as pd
import plotly.express as px
```

```
import streamlit as st

ROOT = Path(__file__).resolve().parents[1]
st.set_page_config(page_title="EDA Churn", page_icon="🔎", layout="wide")

STATUS = {"Bertahan": "#2a78d6", "Churn": "#eb6834"}
KATEGORI = ["#2a78d6", "#1baf7a", "#eda100", "#4a3aa7", "#e87ba4"]
KONTRAK = ["Bulanan", "Tahunan", "Dua Tahun"]

@st.cache_data
def load_data():
    df = pd.read_csv(ROOT / "data" / "customer_churn.csv", parse_dates=["signup_date"])
    df["total_charges_missing"] = df["total_charges"].isna()        # catat sebelum imputasi
    df["total_charges"] = df["total_charges"].fillna(df["total_charges"].median())
    df["status"] = df["churn"].map({0: "Bertahan", 1: "Churn"})
    df["kelompok_tenure"] = pd.cut(df["tenure_months"], [0, 12, 24, 48, 72],
                                   labels=["0-12", "13-24", "25-48", "49-72"]).astype(str)
    return df

def gaya(fig, tinggi=360):
    fig.update_layout(template="plotly_white", height=tinggi, margin=dict(l=10, r=10, t=50, b=10),
                      title_font_size=15, legend_title_text="")
    return fig

df = load_data()
st.title("🔎 Exploratory Data Analysis — Customer Churn")
st.caption("Data latihan sintetis · 3.000 pelanggan · pendaftaran Januari 2023 – Desember 2025")

# ---------- KPI CARDS ----------
k1, k2, k3, k4, k5 = st.columns(5)
k1.metric("Total pelanggan", f"{len(df):,}")
k2.metric("Churn rate", f"{df.churn.mean():.1%}")
k3.metric("Rata-rata tagihan", f"Rp{df.monthly_charges.mean():,.0f}".replace(",", "."))
k4.metric("Rata-rata tenure", f"{df.tenure_months.mean():.1f} bln")
k5.metric("Missing total_charges", f"{df.total_charges_missing.sum()} baris", help="Diisi median")

tab = st.tabs(["🍩 Komposisi", "📊 Perbandingan", "☑ Distribusi", "⏱ Tren", "🔗 Hubungan", "📋 Data"])

# ---------- 1. KOMPOSISI: pie, donut, treemap ----------
with tab[0]:
    c1, c2 = st.columns(2)
    pie = px.pie(df, names="status", color="status", color_discrete_map=STATUS,
                 title="Satu dari tiga pelanggan churn")
    pie.update_traces(textinfo="percent+label", textfont_size=14)
    c1.plotly_chart(gaya(pie), width="stretch")
    donut = px.pie(df, names="contract_type", hole=0.55, color_discrete_sequence=KATEGORI,
                   category_orders={"contract_type": KONTRAK},
                   title="Separuh pelanggan berkontrak bulanan")
    donut.update_traces(textinfo="percent+label")
    donut.add_annotation(text=f"<b>{len(df):,}</b><br>pelanggan", showarrow=False, font_size=16)
    c2.plotly_chart(gaya(donut), width="stretch")
    tm = df.groupby(["region", "segment"], as_index=False).agg(pelanggan=("customer_id", "count"),
                                                               churn_rate=("churn", "mean"))
    tree = px.treemap(tm, path=["region", "segment"], values="pelanggan", color="churn_rate",
                      color_continuous_scale="Oranges",
                      title="Treemap: luas = jumlah pelanggan, warna = churn rate")
    st.plotly_chart(gaya(tree, 420), width="stretch")

# ---------- 2. PERBANDINGAN: bar, grouped bar, stacked 100% ----------
with tab[1]:
    kolom = st.selectbox("Bandingkan churn rate berdasarkan",
                         ["contract_type", "region", "segment", "payment_method", "internet_service"])
    rk = (df.groupby(kolom, as_index=False).agg(pelanggan=("customer_id", "count"),
                                                churn_rate=("churn", "mean"))
            .sort_values("churn_rate"))
    bar = px.bar(rk, x="churn_rate", y=kolom, orientation="h", text_auto=".1%",
                 color_discrete_sequence=["#eb6834"], title=f"Churn rate per {kolom}")
    bar.add_vline(x=df.churn.mean(), line_dash="dot", annotation_text="rata-rata")
    bar.update_xaxes(tickformat=".0%")
    c1, c2 = st.columns([3, 2])
    c1.plotly_chart(gaya(bar), width="stretch")
    c2.dataframe(rk.sort_values("churn_rate", ascending=False)
                   .style.format({"churn_rate": "{:.1%}"})
                   .background_gradient(subset=["churn_rate"], cmap="Oranges"),
```

```
                width="stretch", hide_index=True)
    c3, c4 = st.columns(2)
    gb = df.groupby(["contract_type", "segment"], as_index=False)["churn"].mean()
    grouped = px.bar(gb, x="contract_type", y="churn", color="segment", barmode="group",
                     color_discrete_sequence=KATEGORI, category_orders={"contract_type": KONTRAK},
                     title="Grouped bar: kontrak × segmen")
    grouped.update_yaxes(tickformat=".0%")
    c3.plotly_chart(gaya(grouped), width="stretch")
    stacked = px.histogram(df, y="payment_method", color="status", barnorm="percent",
                           color_discrete_map=STATUS, title="Stacked bar 100%: status per pembayaran")
    stacked.update_layout(xaxis_title="Proporsi (%)", yaxis_title="")
    c4.plotly_chart(gaya(stacked), width="stretch")

# ---------- 3. DISTRIBUSI: histogram, box, violin ----------
with tab[2]:
    num = st.selectbox("Variabel numerik", ["tenure_months", "monthly_charges",
                                            "num_complaints", "support_calls"])
    c1, c2 = st.columns(2)
    hist = px.histogram(df, x=num, color="status", barmode="overlay", opacity=0.7, nbins=30,
                        color_discrete_map=STATUS, marginal="box", title=f"Histogram {num}")
    c1.plotly_chart(gaya(hist, 400), width="stretch")
    vio = px.violin(df, x="contract_type", y=num, color="contract_type", box=True,
                    color_discrete_sequence=KATEGORI, category_orders={"contract_type": KONTRAK},
                    title=f"Violin {num} per kontrak")
    c2.plotly_chart(gaya(vio, 400), width="stretch")

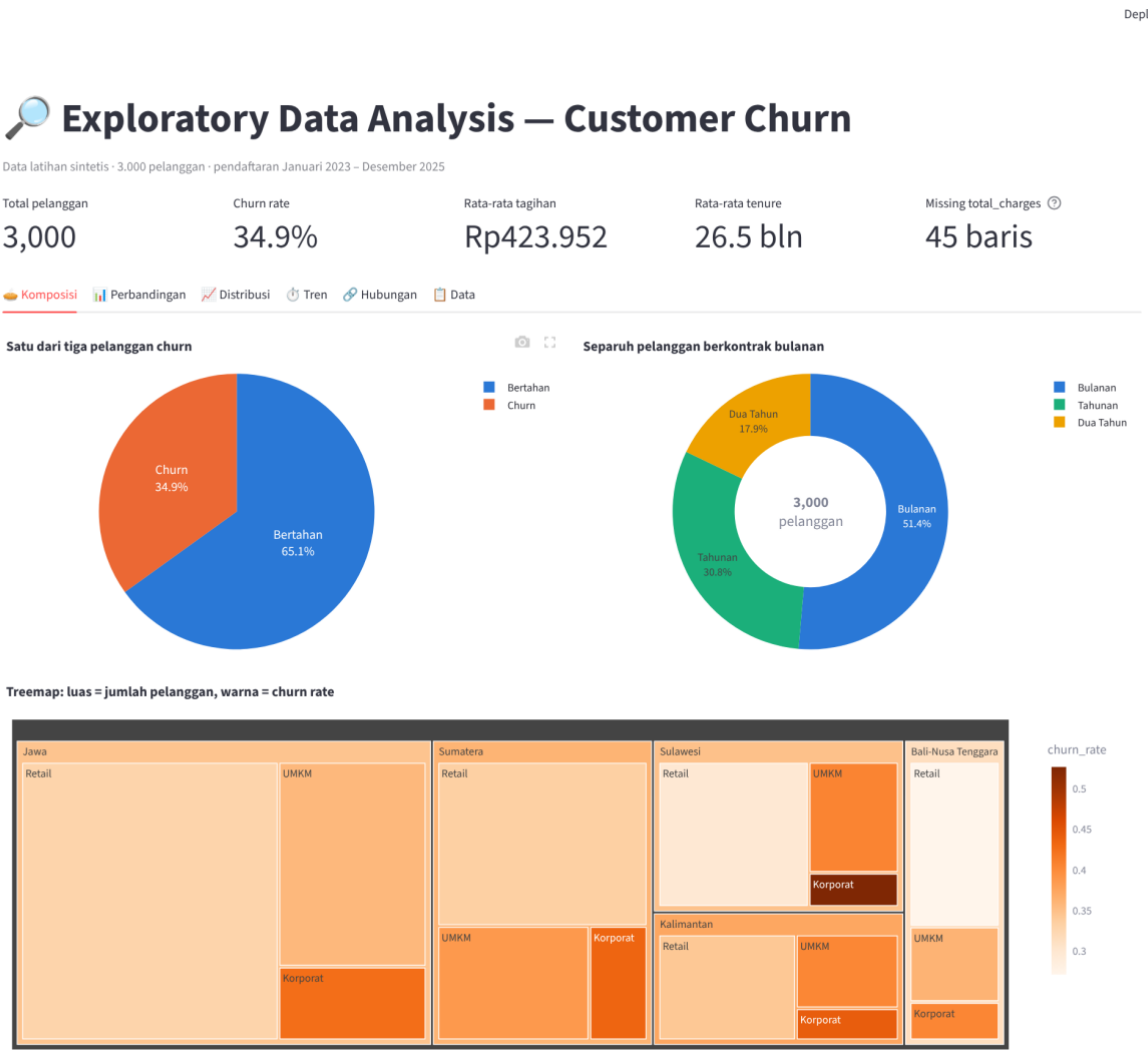
# ---------- 4. TREN: line, area ----------
with tab[3]:
    bulanan = (df.set_index("signup_date").resample("ME")["customer_id"].count()
                 .rename("pelanggan").reset_index())
    bulanan["rata2_3bln"] = bulanan["pelanggan"].rolling(3).mean()
    line = px.line(bulanan, x="signup_date", y=["pelanggan", "rata2_3bln"], markers=True,
                   color_discrete_sequence=["#9dbbe6", "#2a78d6"],
                   title="Pelanggan baru per bulan dan rata-rata bergerak 3 bulan")
    st.plotly_chart(gaya(line), width="stretch")
    c1, c2 = st.columns(2)
    kt = df.groupby("kelompok_tenure", as_index=False)["churn"].mean()
    lt = px.line(kt, x="kelompok_tenure", y="churn", markers=True, text=kt["churn"].map("{:.1%}".format),
                 color_discrete_sequence=["#eb6834"], title="Churn rate per kelompok tenure")
    lt.update_traces(textposition="top center", marker_size=10)
    lt.update_yaxes(tickformat=".0%", range=[0, 0.65])
    c1.plotly_chart(gaya(lt), width="stretch")
    q = (df.assign(kuartal=df.signup_date.dt.to_period("Q").astype(str))
           .groupby(["kuartal", "segment"], as_index=False)["customer_id"].count())
    area = px.area(q, x="kuartal", y="customer_id", color="segment",
                   color_discrete_sequence=KATEGORI, title="Area: pelanggan baru per kuartal")
    c2.plotly_chart(gaya(area), width="stretch")

# ---------- 5. HUBUNGAN: scatter, heatmap ----------
with tab[4]:
    c1, c2 = st.columns(2)
    sc = px.scatter(df, x="tenure_months", y="monthly_charges", color="status", opacity=0.55,
                    color_discrete_map=STATUS, hover_name="customer_id",
                    hover_data=["contract_type", "num_complaints"], title="Tenure vs tagihan")
    c1.plotly_chart(gaya(sc, 420), width="stretch")
    corr = df[["tenure_months", "monthly_charges", "num_complaints", "support_calls", "churn"]].corr()
    hm = px.imshow(corr.round(2), text_auto=True, color_continuous_scale="RdBu_r", zmin=-1, zmax=1,
                   title="Heatmap korelasi")
    c2.plotly_chart(gaya(hm, 420), width="stretch")
    pv = pd.pivot_table(df, values="churn", index="region", columns="segment", aggfunc="mean")
    hm2 = px.imshow(pv * 100, text_auto=".1f", color_continuous_scale="Oranges",
                    labels={"color": "Churn (%)"}, title="Heatmap churn rate region × segmen (%)")
    st.plotly_chart(gaya(hm2, 360), width="stretch")

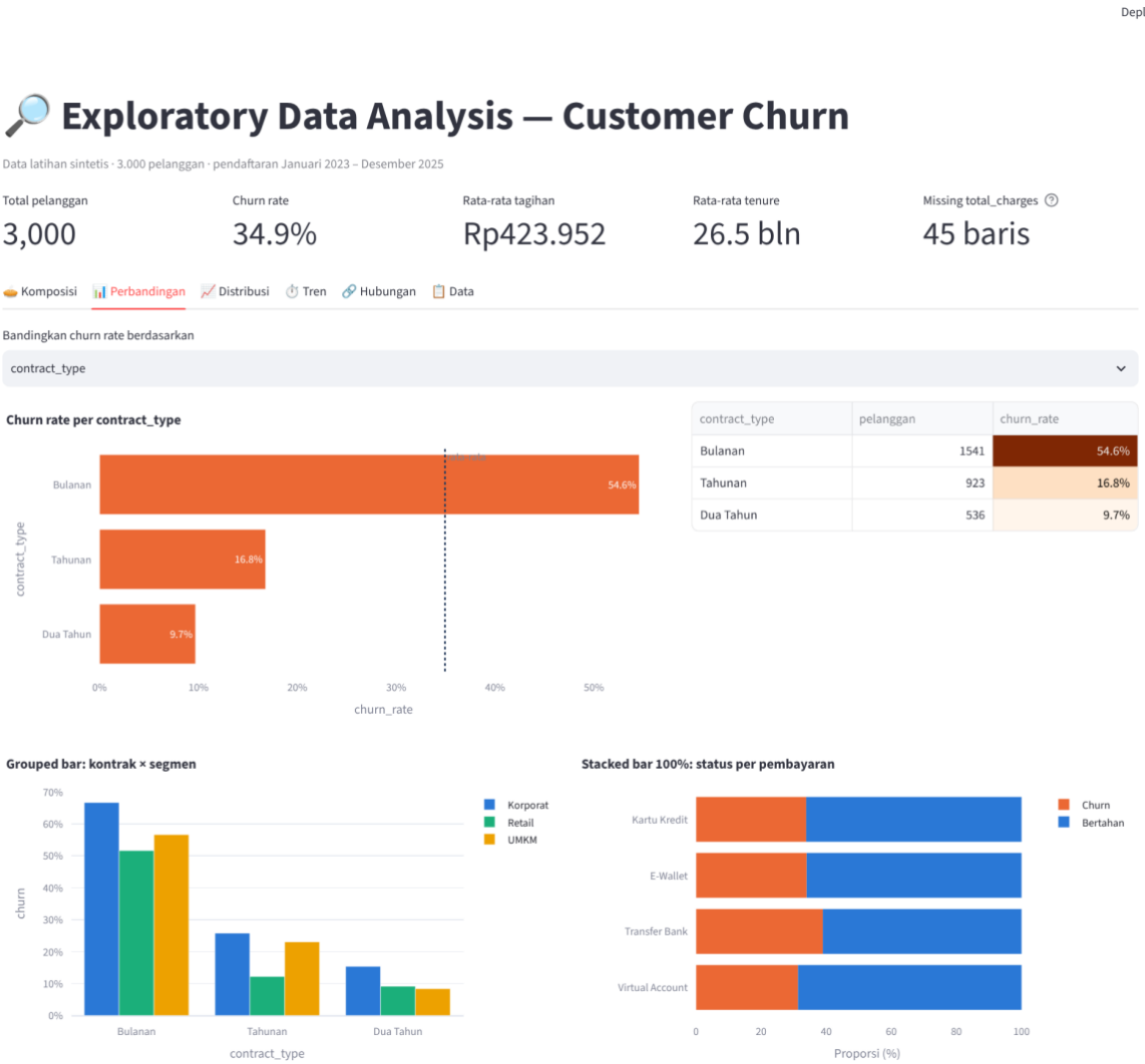
# ---------- 6. DATA: profil + tabel ----------
with tab[5]:
    profil = pd.DataFrame({"tipe": df.dtypes.astype(str),
                           "missing": df.isna().sum().where(df.columns != "total_charges",
                                                            df.total_charges_missing.sum()),
                           "unik": df.nunique()}).reset_index(names="kolom")
    c1, c2 = st.columns([2, 3])
    c1.markdown("**Profil kolom**")
    c1.dataframe(profil, width="stretch", hide_index=True)
    c2.markdown("**Statistik deskriptif**")
    c2.dataframe(df.describe(include="number").T.round(1), width="stretch")
    st.markdown("**10 baris pertama**")
```

```
    st.dataframe(df.head(10), width="stretch", hide_index=True)
```

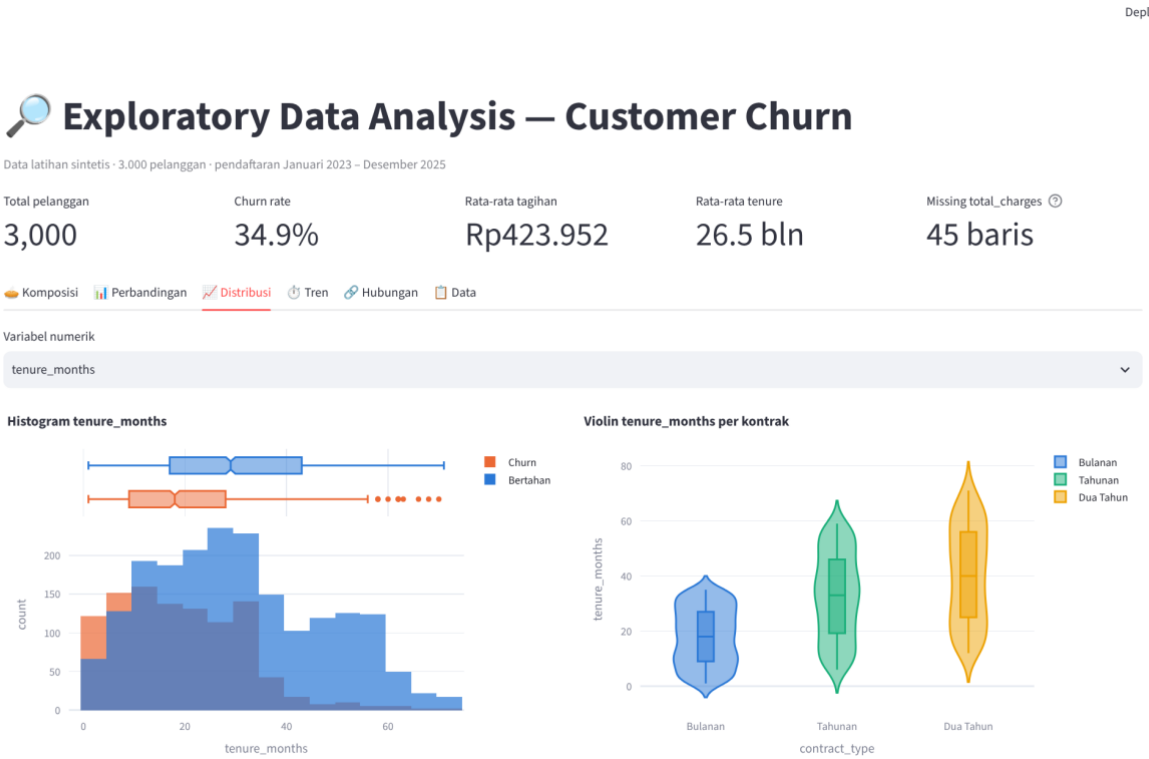
Jalankan dengan `streamlit run sesi2/app_eda.py` (lihat Gambar 2.17, 2.18, 2.19, 2.20, 2.21, dan 2.22).



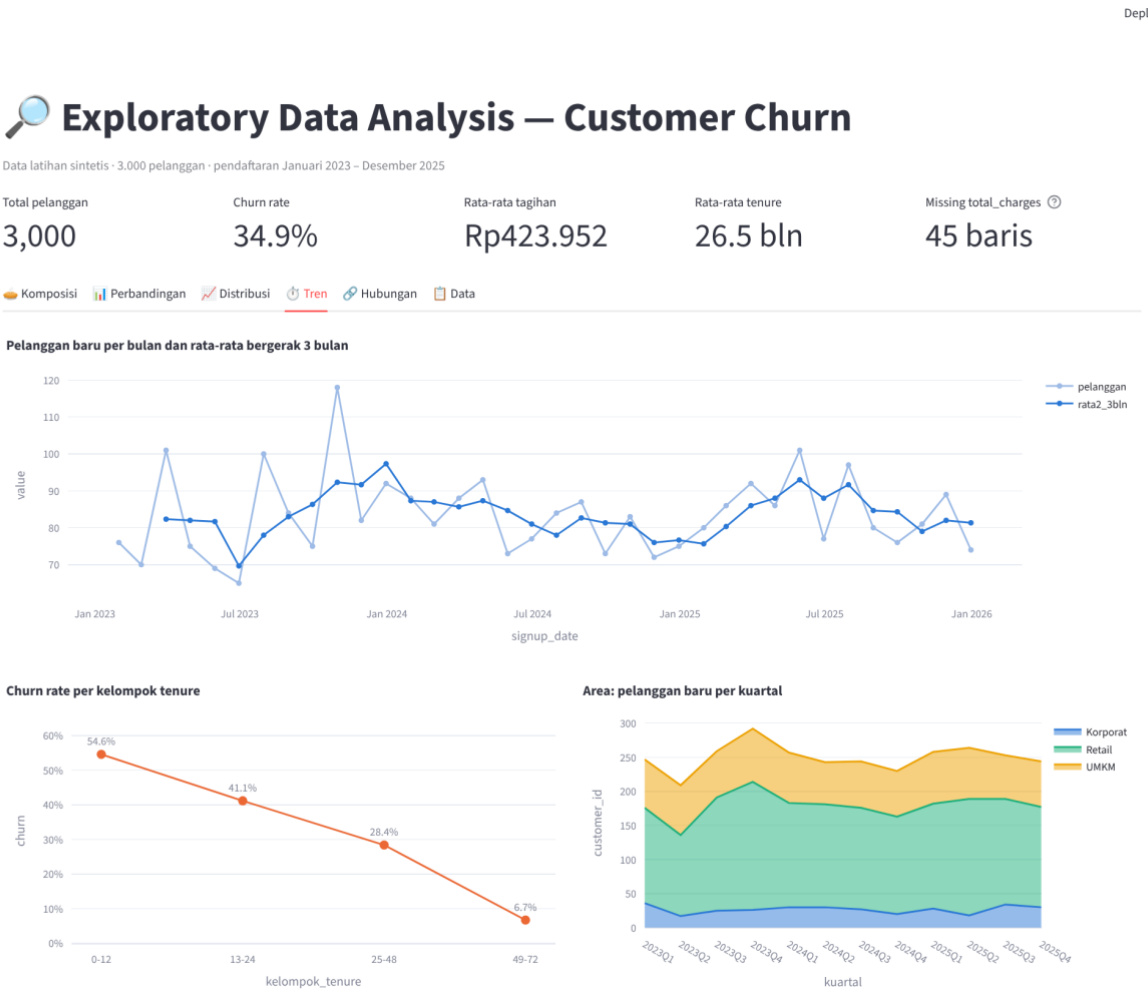
Gambar 2.17. *Tab Komposisi: KPI card, pie, donut, dan treemap*



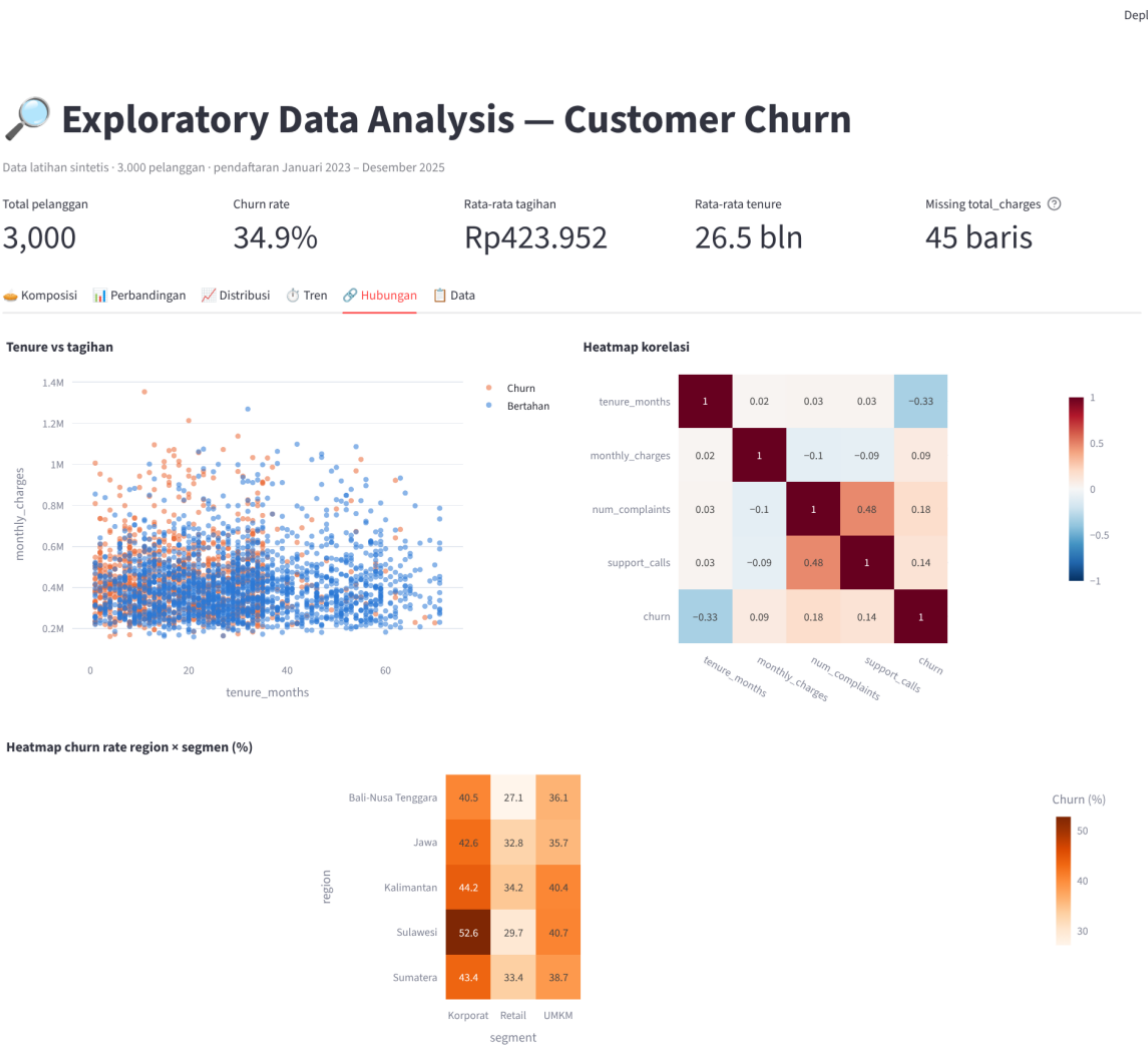
Gambar 2.18. *Tab Perbandingan: bar terurut dengan tabel berwarna, grouped bar, dan stacked bar 100%*



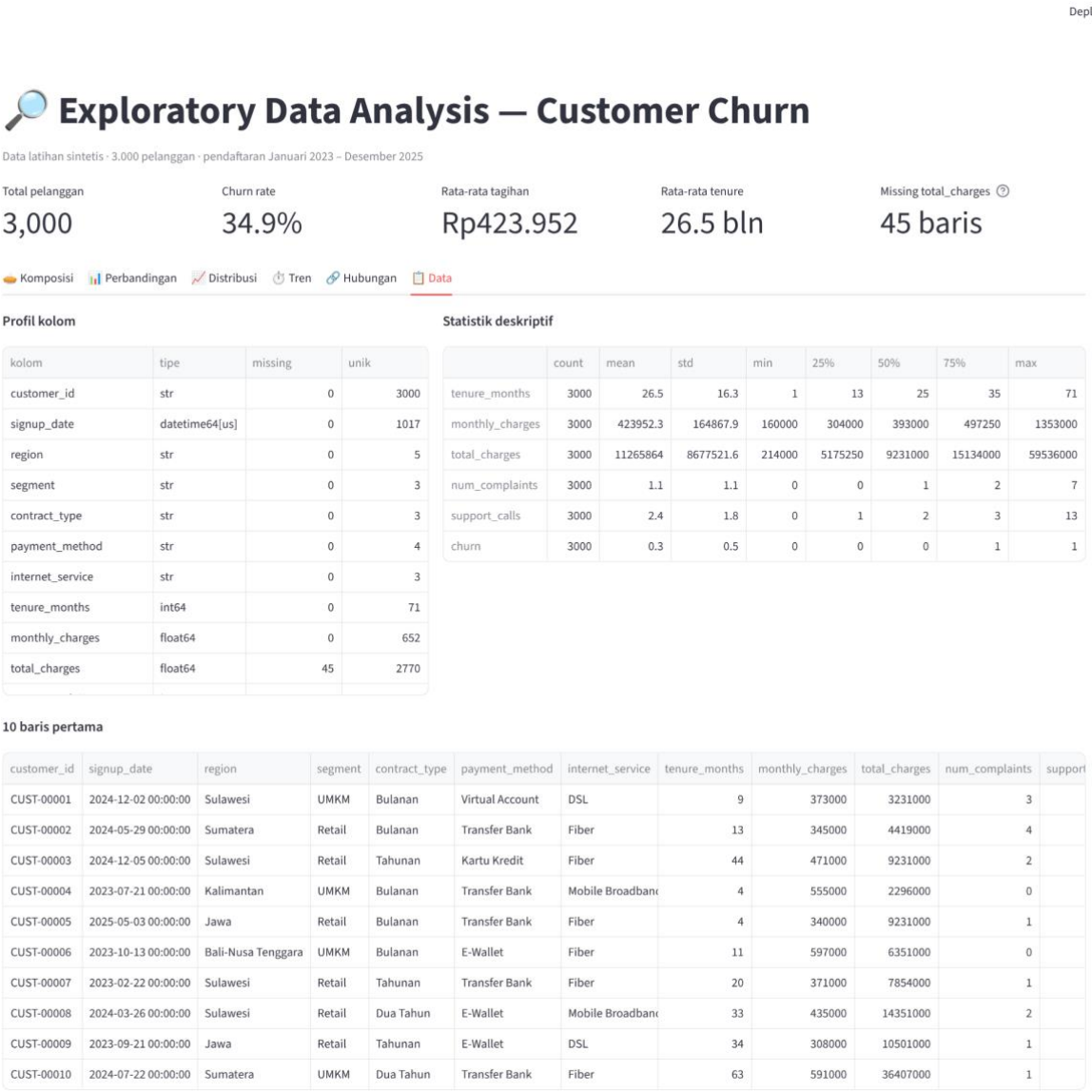
Gambar 2.19. *Tab Distribusi: histogram dengan box marginal dan violin plot*



Gambar 2.20. *Tab Tren: line chart dan area chart*



Gambar 2.21. *Tab Hubungan: scatter plot, heatmap korelasi, dan heatmap pivot*



Gambar 2.22. *Tab Data: profil kolom, statistik deskriptif, dan tabel data*

Penjelasan kode

- `@st.cache_data` menyimpan hasil `load_data()` di memori, sehingga CSV tidak dibaca ulang setiap kali pengguna berpindah tab atau memilih variabel.
- `total_charges_missing` dicatat **sebelum** imputasi agar KPI "Missing total_charges" tetap melaporkan 45 baris yang asli.
- Fungsi `gaya()` menyeragamkan template, tinggi, dan margin semua chart Plotly; cukup ditulis sekali.
- `st.tabs([...])` membagi 14 grafik ke enam kelompok tujuan, sehingga halaman tidak terlalu panjang.
- `px.pie(..., hole=0.55)` menghasilkan donut; `add_annotation()` menaruh angka total di tengah.
- `px.treemap(path=[...], values=..., color=...)` memvisualkan dua dimensi sekaligus: luas kotak = jumlah pelanggan, warna = churn rate.
- `px.histogram(..., barnorm="percent")` membuat stacked bar 100% langsung dari data mentah.
- `px.histogram(..., marginal="box")` menambahkan box plot kecil di atas histogram.
- `.style.background_gradient(cmap="Oranges")` mewarnai sel tabel sesuai nilainya (conditional formatting), membuat tabel ikut berfungsi sebagai visualisasi.

Checkpoint

1. Lima KPI card tampil: 3,000 pelanggan; 34.9%; Rp423.952; 26.5 bln; 45 baris.
2. Tab Komposisi menampilkan pie (Churn 34,9%), donut (Bulanan 51,4%), dan treemap.
3. Di tab Perbandingan, pilih `contract_type`: batang Bulanan (54,6%) paling panjang dan berada di kanan garis rata-rata.
4. Di tab Distribusi, ganti variabel menjadi `num_complaints`; histogram dan violin ikut berubah.

2.6 Menulis insight dari EDA

Tulis temuan dalam format **temuan + angka + implikasi**. Ringkasan insight dari sesi ini (Tabel 2.5):

Tabel 2.5. *Menulis insight dari EDA*

No	Temuan	Angka kunci	Implikasi
1	Jenis kontrak adalah pembeda churn terkuat	Bulanan 54,6% vs Tahunan 16,8%	Program migrasi ke kontrak tahunan
2	Masa awal berlangganan paling rawan	0–12 bulan 54,6%; > 48 bulan 6,7%	Perkuat onboarding 12 bulan pertama
3	Keluhan berulang adalah sinyal dini	Bulanan + keluhan ≥ 3: churn 71,9%	Tindak lanjut keluhan maksimal 48 jam
4	Metode pembayaran manual lebih berisiko	Transfer Bank 39% vs Virtual Account 31%	Dorong autodebit/e-wallet
5	Kombinasi segmen-region perlu ditelusuri	Korporat Sulawesi 52,6%	Investigasi kualitas layanan regional

Latihan mandiri Sesi 2

1. Buat pie chart komposisi `internet_service` dan donut chart komposisi `region`. Mana yang lebih mudah dibaca untuk 5 kategori? Ganti dengan bar horizontal terurut dan bandingkan.
2. Buat stacked bar 100% status churn per `internet_service`.
3. Buat box plot `support_calls` berdasarkan status churn.
4. Buat line chart churn rate per kuartal pendaftaran (`dt.to_period("Q")`).
5. Tambahkan tab ketujuh "Keluhan" di `app_eda.py` berisi bar chart churn rate per jumlah keluhan.
6. Tambahkan `st.download_button` di tab Perbandingan untuk mengunduh tabel ringkasan sebagai CSV.

Kesalahan umum Sesi 2

Rinciannya disajikan pada Tabel 2.6.

Tabel 2.6. *Kesalahan umum Sesi 2*

Gejala	Penyebab	Solusi
`FileNotFoundError: customer_churn.csv`	Script dijalankan dari folder lain dengan path relatif	Pakai `ROOT = Path(__file__).resolve().parents[1]` seperti contoh
`TypeError: Only valid with DatetimeIndex` saat `resample`	Kolom tanggal masih bertipe teks	Tambahkan `parse_dates=["signup_date"]` saat `read_csv`
`ValueError: The truth value of a Series is ambiguous`	Memakai `and`/`or` pada filter pandas	Ganti dengan `&` atau `|` dan beri kurung tiap kondisi

Gejala	Penyebab	Solusi
`TypeError: boxplot() got an unexpected keyword 'tick_labels'`	Matplotlib versi lama (< 3.9)	Ganti `tick_labels=` dengan `labels=`
Grafik tumpang tindih atau memori penuh	Figure tidak ditutup	Panggil `plt.close(fig)` setelah menyimpan (sudah ada di fungsi `simpan`)
`background_gradient` error di Streamlit	Matplotlib belum terinstal	`pip install matplotlib`
Pie chart sulit dibaca	Terlalu banyak kategori atau nilai mirip	Batasi 2–5 kategori; gunakan bar terurut

Sesi 3 — Dynamic Layout with Streamlit (120 menit)

Sesi ini mengubah EDA menjadi dashboard: filter di sidebar, empat KPI card dalam satu baris, tab untuk memisahkan ringkasan, segmentasi, dan data detail, serta tombol unduh hasil filter.

Tujuan sesi

- Menempatkan input di sidebar dengan `st.sidebar`.
- Menyusun tata letak dengan `st.columns`, `st.tabs`, `st.expander`, dan `st.container`.
- Menerapkan beberapa filter sekaligus ke DataFrame dengan boolean mask.
- Menyusun KPI card dengan `st.metric` dan delta.
- Menangani kondisi data kosong dengan `st.warning` dan `st.stop()`.

3.1 Komponen layout Streamlit

Rinciannya disajikan pada Tabel 3.1.

Tabel 3.1. *Komponen layout Streamlit*

Komponen	Kegunaan	Contoh pemakaian
`st.set_page_config(layout="wide")`	Memakai lebar layar penuh	Wajib untuk dashboard
`st.sidebar`	Panel kiri untuk filter global	`st.sidebar.multiselect(...)`
`st.columns(n)` atau `st.columns([3, 2])`	Membagi baris menjadi kolom sama lebar atau berproporsi	Baris KPI, grafik berdampingan
`st.tabs([...])`	Memisahkan konten ke beberapa tab	Ringkasan / Segmentasi / Detail
`st.expander("...")`	Konten yang bisa dilipat	Opsi lanjutan, catatan metodologi
`st.container()`	Mengelompokkan elemen; dapat diberi border	Kartu berisi grafik
`st.form("id")`	Menahan rerun sampai tombol submit diklik	Form input prediksi (Sesi 4)

Dua cara menulis isi kolom atau tab:

```python
# Cara 1: notasi titik
k1, k2 = st.columns(2)
k1.metric("Pelanggan", 3000)

# Cara 2: blok with (cocok untuk banyak elemen)
with k2:
    st.markdown("**Grafik**")
    st.line_chart(data)
```

3.2 Pola desain dashboard

Susun dashboard dari atas ke bawah mengikuti cara orang membaca:

1. **Header** — judul, periode data, sumber.
2. **KPI cards** — 3 sampai 5 angka terpenting dalam satu baris.
3. **Grafik utama** — tren dan perbandingan untuk menjawab pertanyaan bisnis.
4. **Grafik pendukung / segmentasi** — drill-down per kategori.
5. **Tabel detail** — untuk investigasi dan unduhan.

Filter global diletakkan di sidebar agar berlaku untuk semua bagian.

3.3 Praktikum — Dashboard Streamlit dengan layout dinamis

Buat `sesi3/app_layout_dashboard.py`:

PYTHON

```python
# sesi3/app_layout_dashboard.py — Dashboard Streamlit dengan sidebar, columns, tabs
from pathlib import Path
import pandas as pd
import plotly.express as px
import streamlit as st

ROOT = Path(__file__).resolve().parents[1]
st.set_page_config(page_title="Dashboard Churn", page_icon="📊", layout="wide")

@st.cache_data
def load_data():
    df = pd.read_csv(ROOT / "data" / "customer_churn.csv", parse_dates=["signup_date"])
    df["total_charges"] = df["total_charges"].fillna(df["total_charges"].median())
    return df

def rupiah(x):
    return f"Rp{x:,.0f}".replace(",", ".")

df = load_data()

# ---------- SIDEBAR: filter ----------
st.sidebar.header("🎛️ Filter Data")
region = st.sidebar.multiselect("Region", sorted(df["region"].unique()),
                                default=sorted(df["region"].unique()))
segment = st.sidebar.multiselect("Segmen", sorted(df["segment"].unique()),
                                 default=sorted(df["segment"].unique()))
kontrak = st.sidebar.radio("Jenis kontrak", ["Semua"] + sorted(df["contract_type"].unique()))
tmin, tmax = int(df.tenure_months.min()), int(df.tenure_months.max())
tenure = st.sidebar.slider("Tenure (bulan)", tmin, tmax, (tmin, tmax))
tgl = st.sidebar.date_input("Periode pendaftaran",
                            (df.signup_date.min().date(), df.signup_date.max().date()))

mask = (df["region"].isin(region) & df["segment"].isin(segment)
        & df["tenure_months"].between(*tenure))
if kontrak != "Semua":
    mask &= df["contract_type"] == kontrak
if isinstance(tgl, tuple) and len(tgl) == 2:
    mask &= df["signup_date"].between(pd.Timestamp(tgl[0]), pd.Timestamp(tgl[1]))
dff = df[mask]

st.sidebar.caption(f"{len(dff):,} dari {len(df):,} pelanggan terpilih")

# ---------- HEADER ----------
st.title("📊 Dashboard Customer Churn")
st.caption("Data latihan sintetis · periode pendaftaran 2023–2025")

if dff.empty:
    st.warning("Tidak ada data untuk kombinasi filter ini. Longgarkan filter di sidebar.")
    st.stop()

# ---------- KPI CARDS (4 kolom) ----------
k1, k2, k3, k4 = st.columns(4)
k1.metric("Total pelanggan", f"{len(dff):,}")
k2.metric("Churn rate", f"{dff.churn.mean():.1%}",
          delta=f"{(dff.churn.mean() - df.churn.mean()) * 100:+.1f} pp vs total",
          delta_color="inverse")
k3.metric("Rata-rata tagihan", rupiah(dff.monthly_charges.mean()))
k4.metric("Rata-rata tenure", f"{dff.tenure_months.mean():.1f} bln")

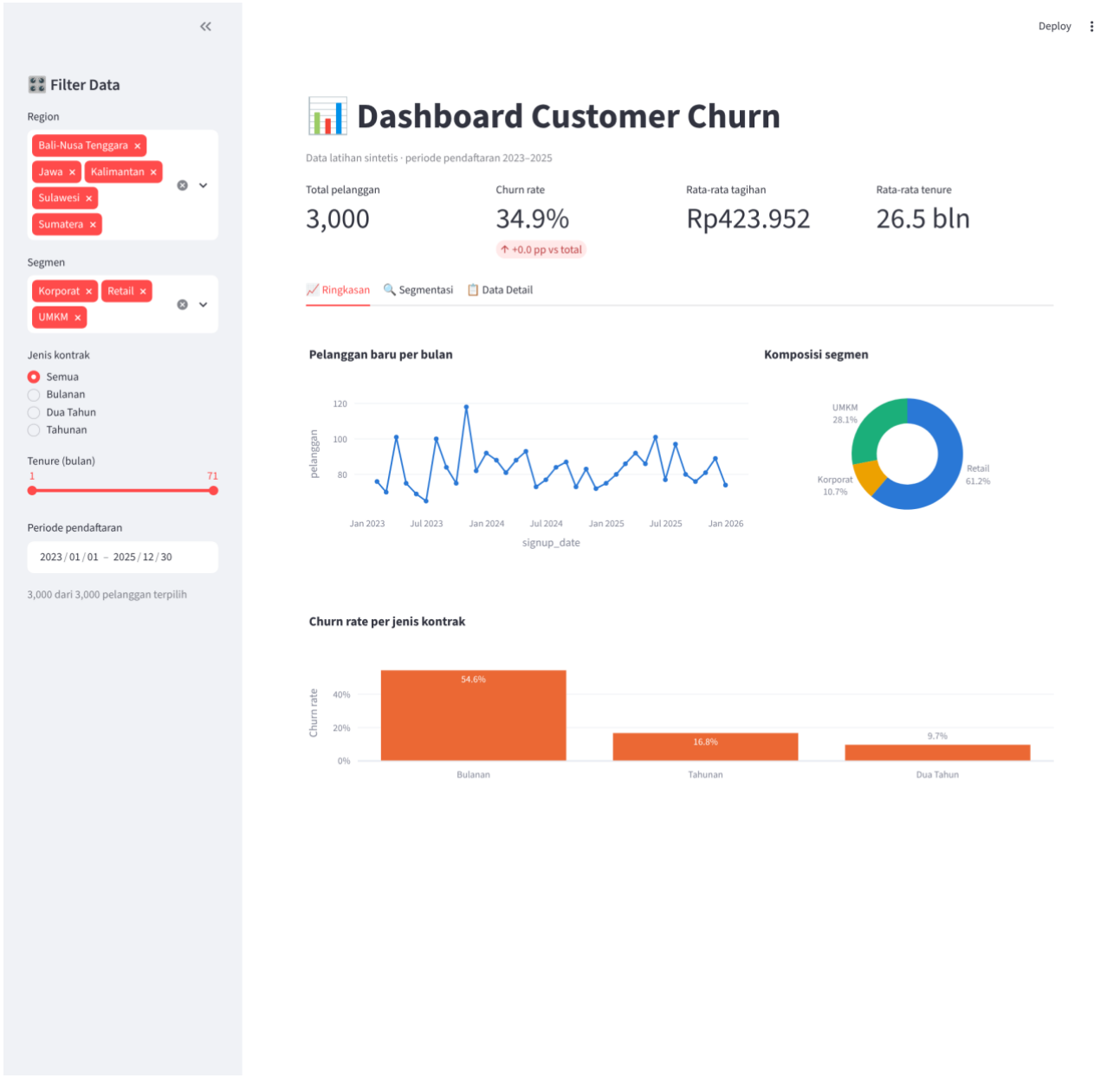
# ---------- TABS ----------
tab1, tab2, tab3 = st.tabs(["☑ Ringkasan", "🔍 Segmentasi", "📋 Data Detail"])
```

```
with tab1:
    c1, c2 = st.columns([3, 2])
    with c1:
        tren = (dff.set_index("signup_date").resample("ME")["customer_id"].count()
                  .rename("pelanggan").reset_index())
        fig_tren = px.line(tren, x="signup_date", y="pelanggan", markers=True,
                           title="Pelanggan baru per bulan", color_discrete_sequence=["#2a78d6"])
        st.plotly_chart(fig_tren.update_layout(height=320, template="plotly_white"), width="stretch")
    with c2:
        fig_donut = px.pie(dff, names="segment", hole=0.55, title="Komposisi segmen",
                           color_discrete_sequence=["#2a78d6", "#1baf7a", "#eda100"])
        fig_donut.update_traces(textinfo="percent+label")
        st.plotly_chart(fig_donut.update_layout(height=320, showlegend=False), width="stretch")
    ck = (dff.groupby("contract_type", as_index=False)["churn"].mean()
             .sort_values("churn", ascending=False))
    fig_bar = px.bar(ck, x="contract_type", y="churn", text_auto=".1%",
                     title="Churn rate per jenis kontrak", color_discrete_sequence=["#eb6834"])
    fig_bar.update_yaxes(tickformat=".0%", title="Churn rate")
    st.plotly_chart(fig_bar.update_layout(height=300, template="plotly_white", xaxis_title=""),
                    width="stretch")

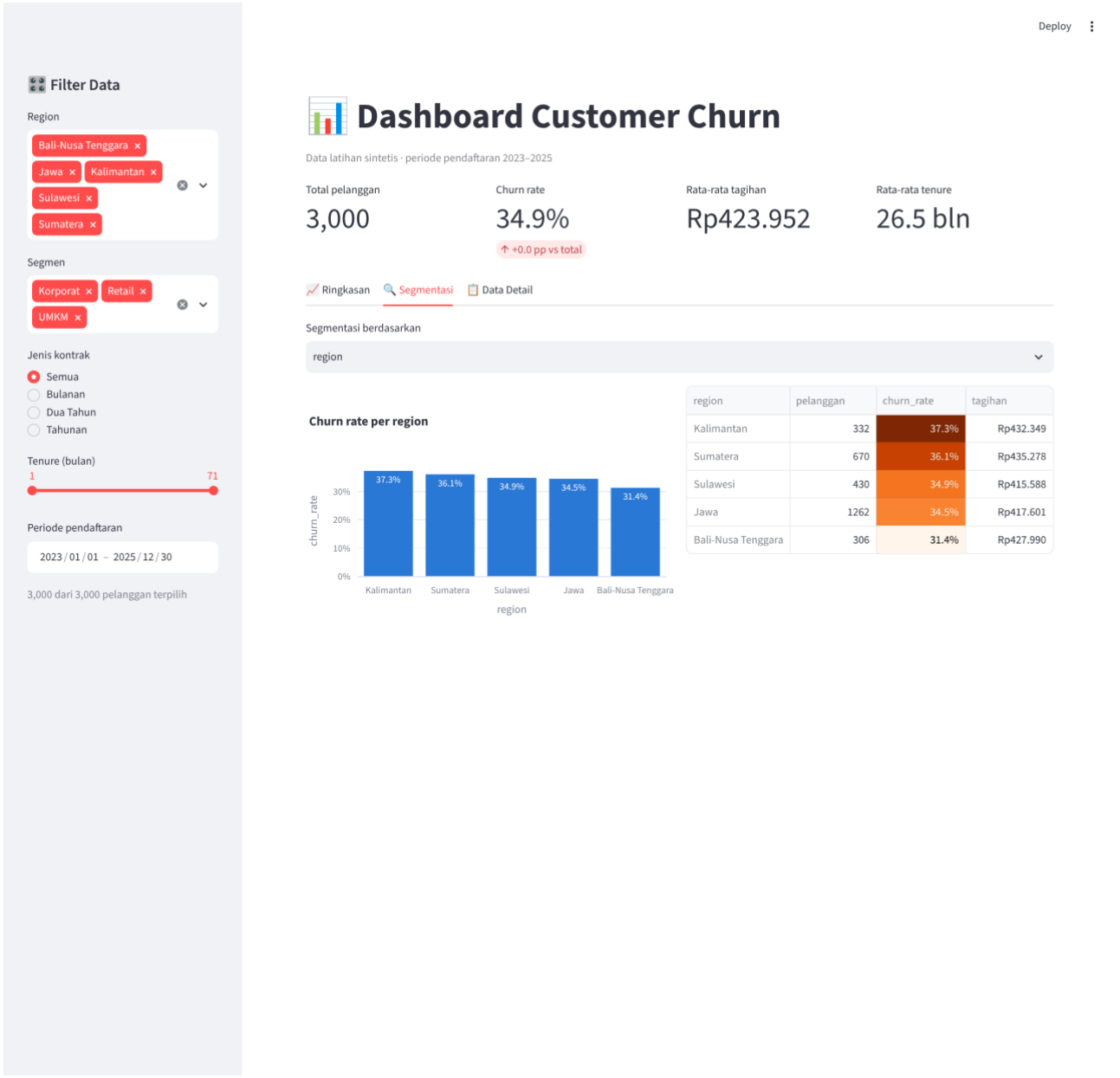
with tab2:
    pilih = st.selectbox("Segmentasi berdasarkan", ["region", "segment", "payment_method",
                                                    "internet_service"])
    seg = (dff.groupby(pilih).agg(pelanggan=("customer_id", "count"),
                                  churn_rate=("churn", "mean"),
                                  tagihan=("monthly_charges", "mean"))
              .sort_values("churn_rate", ascending=False))
    a, b = st.columns(2)
    fig_seg = px.bar(seg.reset_index(), x=pilih, y="churn_rate", text_auto=".1%",
                     color_discrete_sequence=["#2a78d6"], title=f"Churn rate per {pilih}")
    fig_seg.update_yaxes(tickformat=".0%")
    a.plotly_chart(fig_seg.update_layout(height=320, template="plotly_white"), width="stretch")
    b.dataframe(seg.style.format({"churn_rate": "{:.1%}", "tagihan": rupiah})
                   .background_gradient(subset=["churn_rate"], cmap="Oranges"), width="stretch")

with tab3:
    with st.expander("Pilih kolom yang ditampilkan", expanded=False):
        cols = st.multiselect("Kolom", list(dff.columns), default=list(dff.columns)[:8])
    st.dataframe(dff[cols], width="stretch", height=360)
    st.download_button("⬇ Unduh CSV", dff.to_csv(index=False).encode("utf-8"),
                       file_name="churn_filtered.csv", mime="text/csv")
```

Jalankan dengan `streamlit run sesi3/app_layout_dashboard.py` (lihat Gambar 3.1, dan 3.2).



Gambar 3.1. *Dashboard Streamlit Sesi 3 — tab Ringkasan*



Gambar 3.2. *Dashboard Streamlit Sesi 3 — tab Segmentasi dengan tabel berwarna*

Penjelasan kode

- **Satu mask untuk semua filter.** Setiap filter menghasilkan Series `True/False`; digabung dengan `&` lalu diterapkan sekali (`df[mask]`). Pola ini mudah ditambah filter baru.
- `mask &= ...` hanya dijalankan jika pengguna memilih kontrak tertentu, sehingga opsi "Semua" tidak memfilter apa pun.
- `st.date_input` dengan nilai awal tuple menghasilkan rentang tanggal. Saat pengguna baru memilih tanggal awal, nilainya sementara berisi satu tanggal; pengecekan `len(tgl) == 2` mencegah error.
- `st.stop()` menghentikan eksekusi script setelah peringatan data kosong, sehingga grafik tidak mencoba memproses DataFrame kosong.
- `delta_color="inverse"` membuat kenaikan churn tampil merah, karena untuk churn kenaikan adalah kabar buruk.
- `st.columns([3, 2])` membagi lebar 60:40 agar line chart tren mendapat ruang lebih besar daripada donut komposisi segmen.

- `st.plotly_chart(fig, width="stretch")` menampilkan figure Plotly (line, donut, bar) yang dapat di-hover dan di-zoom. Donut dibuat dengan `px.pie(..., hole=0.55)`.
- `.background_gradient(subset=["churn_rate"], cmap="Oranges")` memberi warna gradasi pada kolom churn rate sehingga tabel segmentasi juga berfungsi sebagai heatmap.
- Isi tab tetap dijalankan walaupun tab tidak sedang dibuka; untuk perhitungan sangat berat, pertimbangkan `st.cache_data`.

Checkpoint

1. Tanpa filter, KPI menampilkan 3,000 pelanggan, churn rate 34.9%, rata-rata tagihan Rp423.952, dan rata-rata tenure 26.5 bln.
2. Pilih jenis kontrak **Bulanan**: churn rate naik menjadi 54.6% dengan delta merah sekitar +19.7 pp.
3. Kosongkan semua region: muncul peringatan kuning dan dashboard berhenti dengan rapi.
4. Di tab **Data Detail**, klik **Unduh CSV** dan pastikan file berisi baris sesuai filter.

3.4 Teknik tambahan

Container dengan border untuk efek kartu:

```python
with st.container(border=True):
    st.markdown("**Catatan analis**")
    st.write("Churn tertinggi pada kontrak bulanan.")
```

Session state untuk menyimpan nilai antar-rerun, misalnya tombol reset filter:

```python
if "reset" not in st.session_state:
    st.session_state.reset = 0
if st.sidebar.button("Reset filter"):
    st.session_state.reset += 1          # mengganti key widget = mengembalikan ke default
region = st.sidebar.multiselect("Region", opsi, default=opsi,
                                key=f"region_{st.session_state.reset}")
```

Multipage app: untuk dashboard besar, buat folder `pages/` berisi file `.py` terpisah. Streamlit otomatis membuat menu navigasi di sidebar.

Latihan mandiri Sesi 3

1. Tambahkan filter `internet_service` (multiselect) di sidebar dan pastikan semua KPI ikut berubah.
2. Tambahkan KPI ke-5: persentase pelanggan dengan keluhan ≥ 3.
3. Pada tab Ringkasan, tambahkan grafik churn rate per kuartal pendaftaran (`dt.to_period("Q")`).
4. Bungkus setiap grafik di tab Ringkasan dengan `st.container(border=True)`.
5. Tantangan: buat tombol **Reset filter** memakai pola session state di atas.

Kesalahan umum Sesi 3

Rinciannya disajikan pada Tabel 3.2.

Tabel 3.2. *Kesalahan umum Sesi 3*

Gejala	Penyebab	Solusi
`KeyError` setelah semua opsi multiselect dihapus	DataFrame hasil filter kosong	Cek `dff.empty`, tampilkan `st.warning`, lalu `st.stop()`
`ValueError: not enough values to unpack` pada date_input	Pengguna baru memilih satu tanggal	Periksa `isinstance(tgl, tuple) and len(tgl) == 2`
Layout sempit di tengah layar	Belum memakai layout wide	`st.set_page_config(layout="wide")`
Aplikasi lambat setiap kali filter digeser	Data dibaca ulang dari disk	Gunakan `@st.cache_data` pada fungsi load

Sesi 4 — ML Model Inference & Dashboard Integration (105 menit)

Sesi ini menghubungkan final trained model ke dashboard Streamlit: model dimuat sekali, input pengguna divalidasi, inference dijalankan, dan hasilnya tampil sebagai probabilitas, kelas prediksi, kategori risiko, dan rekomendasi. Tidak ada training ulang di sesi ini.

Tujuan sesi

- Memahami batas tanggung jawab: model sudah final; peserta menangani loading, input, inference, dan penyajian output.
- Membaca kontrak input model dari `model_metadata.json`.
- Memuat model `joblib` dan menyimpannya di cache dengan `@st.cache_resource`.
- Menyiapkan input inference yang konsisten dengan saat model difinalisasi.
- Menjalankan prediksi untuk satu pelanggan (form) dan banyak pelanggan (upload CSV).
- Menyajikan prediction output sebagai KPI, grafik, tabel, dan rekomendasi tindakan.

4.1 Posisi sesi ini dalam siklus ML

Rinciannya disajikan pada Tabel 4.1.

Tabel 4.1. *Posisi sesi ini dalam siklus ML*

Tahap	Dilakukan di pelatihan ini?
Pengumpulan data, feature engineering, training	Tidak — sudah dilakukan sebelumnya
Hyperparameter tuning, cross-validation, perbandingan model	Tidak
Pemilihan best-performing model dan penyimpanan (serialisasi)	Tidak — file `churn_model.joblib` disediakan fasilitator
Memuat model	Ya
Menyiapkan input inference	Ya
Menjalankan prediksi	Ya
Menyajikan prediction output di dashboard	Ya

Script `models/prepare_final_model.py` disertakan dalam paket hanya sebagai dokumentasi asal-usul model. Peserta tidak perlu menjalankannya.

4.2 Kontrak input model

Model hanya dapat memberi hasil yang benar jika input sama persis dengan yang ia kenal saat difinalisasi: nama kolom, tipe data, dan nilai kategori. File `models/model_metadata.json` mendokumentasikan kontrak ini. Cuplikannya:

```JSON
{
  "model_name": "Customer Churn Classifier",
  "model_version": "1.0.0",
  "algorithm": "GradientBoostingClassifier (scikit-learn Pipeline)",
  "sklearn_version": "1.9.1",
  "numeric_features": ["tenure_months", "monthly_charges", "total_charges",
                       "num_complaints", "support_calls"],
  "categorical_features": ["region", "segment", "contract_type",
                           "payment_method", "internet_service"],
```

```
  "categories": {
    "contract_type": ["Bulanan", "Dua Tahun", "Tahunan"],
    "...": "..."
  },
  "decision_threshold": 0.5,
  "risk_bins": {"Rendah": [0.0, 0.3], "Sedang": [0.3, 0.6], "Tinggi": [0.6, 1.0]},
  "holdout_metrics": {"roc_auc": 0.7887, "accuracy": 0.7183, "f1": 0.5786}
}
```

Prinsip konsistensi preprocessing: karena imputasi, standarisasi, dan one-hot encoding berada **di dalam** `Pipeline`, peserta cukup mengirim data mentah dengan nama kolom yang benar. Jangan melakukan scaling atau encoding sendiri sebelum memanggil model; itu akan membuat preprocessing terjadi dua kali dan hasil prediksi menjadi salah.

Jika model yang Anda terima di proyek lain **tidak** dibungkus `Pipeline`, Anda harus memuat juga objek preprocessing (scaler, encoder) yang disimpan terpisah dan menerapkannya dengan urutan yang sama persis.

4.3 Praktikum A — Modul inference yang dapat dipakai ulang

Logika inference dipisahkan ke `utils/model_utils.py` agar dapat dipakai oleh Streamlit (Sesi 4) maupun Dash (Sesi 8). Buat juga file kosong `utils/__init__.py`.

PYTHON

```python
"""utils/model_utils.py — fungsi bantu inference untuk Streamlit & Dash."""
import json
from pathlib import Path
import joblib
import pandas as pd

ROOT = Path(__file__).resolve().parents[1]
MODEL_PATH = ROOT / "models" / "churn_model.joblib"
META_PATH = ROOT / "models" / "model_metadata.json"



def load_metadata(path=META_PATH) -> dict:
    """Membaca kontrak input model (fitur, kategori, threshold, risk bins)."""
    return json.loads(Path(path).read_text(encoding="utf-8"))



def load_model(path=MODEL_PATH):
    """Memuat final trained model (Pipeline preprocessing + classifier). Tanpa training ulang."""
    return joblib.load(path)



def validate_input(df: pd.DataFrame, meta: dict) -> list[str]:
    """Mengembalikan daftar pesan error; list kosong berarti input valid."""
    errors = []
    missing = [c for c in meta["feature_order"] if c not in df.columns]
    if missing:
        errors.append(f"Kolom wajib tidak ada: {missing}")
        return errors
    for c in meta["categorical_features"]:
        unknown = set(df[c].dropna().unique()) - set(meta["categories"][c])
        if unknown:
            errors.append(f"Nilai tidak dikenal pada '{c}': {sorted(unknown)}")
    for c in meta["numeric_features"]:
        if not pd.api.types.is_numeric_dtype(df[c]):
            errors.append(f"Kolom '{c}' harus numerik")
    return errors



def prepare_input(df: pd.DataFrame, meta: dict) -> pd.DataFrame:
    """Memilih & mengurutkan kolom sesuai urutan fitur saat model difinalisasi."""
    X = df[meta["feature_order"]].copy()
    for c in meta["numeric_features"]:
        X[c] = pd.to_numeric(X[c], errors="coerce")
    return X
```

```
def risk_category(p: float, meta: dict) -> str:
    for label, (lo, hi) in meta["risk_bins"].items():
        if lo <= p < hi or (hi == 1.0 and p == 1.0):
            return label
    return "Tidak diketahui"


def predict(model, df: pd.DataFrame, meta: dict) -> pd.DataFrame:
    """Menjalankan inference dan menambahkan kolom output prediksi."""
    X = prepare_input(df, meta)
    proba = model.predict_proba(X)[:, 1]
    out = df.copy()
    out["churn_probability"] = proba.round(4)
    out["predicted_churn"] = (proba >= meta["decision_threshold"]).astype(int)
    out["risk_category"] = [risk_category(p, meta) for p in proba]
    return out
```

Penjelasan alur inference

1. `load_model()` — `joblib.load()` mengembalikan objek `Pipeline` yang siap dipakai. Tidak ada `.fit()` di mana pun.
2. `validate_input()` — menolak data yang kolomnya hilang atau berisi kategori yang tidak dikenal model (misalnya salah ketik "bulanan" huruf kecil).
3. `prepare_input()` — memilih kolom dengan urutan yang sama dengan saat model difinalisasi dan memastikan kolom numerik bertipe angka.
4. `predict()` — `predict_proba(X)[:, 1]` mengambil probabilitas kelas 1 (churn). Kelas prediksi ditentukan dengan threshold 0,5, lalu probabilitas dipetakan ke kategori risiko.

4.4 Praktikum B — Uji inference di terminal

Sebelum menyentuh dashboard, pastikan model dapat dimuat dan menghasilkan prediksi. Buat `sesi4/cek_inference.py`:

PYTHON

```
# sesi4/cek_inference.py — uji inference di terminal sebelum masuk dashboard
import sys
from pathlib import Path
import pandas as pd
pd.set_option("display.width", 140)
pd.set_option("display.max_columns", 20)

ROOT = Path(__file__).resolve().parents[1]
sys.path.append(str(ROOT))
from utils.model_utils import load_model, load_metadata, validate_input, predict

meta = load_metadata()
model = load_model()
print("Model :", type(model).__name__, "->", [name for name, _ in model.steps])
print("Fitur :", meta["feature_order"])

baru = pd.read_csv(ROOT / "data" / "new_customers.csv")
print("Error validasi:", validate_input(baru, meta) or "tidak ada")

hasil = predict(model, baru, meta)
print(hasil[["customer_id", "contract_type", "num_complaints",
             "churn_probability", "predicted_churn", "risk_category"]].head())
print("\nJumlah per kategori risiko:\n", hasil["risk_category"].value_counts())
```

Jalankan `python sesi4/cek_inference.py`. Output yang diharapkan:

```
Model : Pipeline -> ['preprocess', 'clf']
Error validasi: tidak ada
  customer_id contract_type  num_complaints  churn_probability  predicted_churn risk_category
0    NEW-0001       Bulanan               1             0.3775                0        Sedang
1    NEW-0002       Bulanan               0             0.3127                0        Sedang
2    NEW-0003       Tahunan               3             0.3641                0        Sedang
3    NEW-0004       Bulanan               0             0.5566                1        Sedang
4    NEW-0005       Tahunan               1             0.1098                0        Rendah

Jumlah per kategori risiko:
Rendah    102
Tinggi     51
Sedang     47
```

Perhatikan NEW-0004: probabilitas 0,5566 melewati threshold 0,5 sehingga diprediksi churn, tetapi kategorinya masih Sedang karena di bawah 0,60. Kelas prediksi dan kategori risiko adalah dua cara membaca probabilitas yang sama, dan keduanya perlu dijelaskan kepada pengguna dashboard.

4.5 Praktikum C — Dashboard prediksi di Streamlit

Buat `sesi4/app_prediksi.py`:

```python
# sesi4/app_prediksi.py — Integrasi final trained model ke dashboard Streamlit
import sys
from pathlib import Path
import pandas as pd
import plotly.express as px
import streamlit as st

ROOT = Path(__file__).resolve().parents[1]
sys.path.append(str(ROOT))                      # agar folder utils bisa di-import
from utils.model_utils import load_model, load_metadata, validate_input, predict

st.set_page_config(page_title="Prediksi Churn", page_icon="🤖", layout="wide")

# 1) LOAD MODEL — sekali saja, disimpan di cache
@st.cache_resource
def get_model():
    return load_model()

@st.cache_data
def get_meta():
    return load_metadata()

model, meta = get_model(), get_meta()

st.title("🤖 Prediksi Risiko Churn Pelanggan")
with st.expander("ℹ️ Informasi model final", expanded=False):
    st.write(f"**{meta['model_name']}** · versi {meta['model_version']} · {meta['algorithm']}")
    st.write("Fitur input:", ", ".join(meta["feature_order"]))
    st.write(f"Threshold keputusan: {meta['decision_threshold']} · "
             f"Risk bins: {meta['risk_bins']}")
    st.caption("Model tidak dilatih ulang di aplikasi ini; aplikasi hanya melakukan inference.")

COLOR = {"Rendah": "🟢", "Sedang": "🟡", "Tinggi": "🔴"}
tab_single, tab_batch = st.tabs(["👤 Prediksi satu pelanggan", "📂 Prediksi batch (CSV)"])

# 2) PREDIKSI SATU PELANGGAN — input lewat form
with tab_single:
    cats, rng = meta["categories"], meta["numeric_ranges"]
    with st.form("form_prediksi"):
        c1, c2, c3 = st.columns(3)
        region = c1.selectbox("Region", cats["region"])
        segment = c1.selectbox("Segmen", cats["segment"])
        contract = c2.selectbox("Jenis kontrak", cats["contract_type"])
        payment = c2.selectbox("Metode pembayaran", cats["payment_method"])
        service = c3.selectbox("Layanan internet", cats["internet_service"])
        tenure = c3.number_input("Tenure (bulan)", 1, 72, 6)
```

```
        d1, d2, d3 = st.columns(3)
        monthly = d1.number_input("Tagihan bulanan (Rp)", 100_000, 2_000_000, 450_000, step=10_000)
        complaints = d2.number_input("Jumlah keluhan", 0, 10, 2)
        calls = d3.number_input("Panggilan ke CS", 0, 15, 3)
        submitted = st.form_submit_button("🔮 Prediksi", type="primary")

    if submitted:
        row = pd.DataFrame([{
            "tenure_months": tenure, "monthly_charges": monthly,
            "total_charges": monthly * tenure, "num_complaints": complaints,
            "support_calls": calls, "region": region, "segment": segment,
            "contract_type": contract, "payment_method": payment, "internet_service": service,
        }])
        errs = validate_input(row, meta)
        if errs:
            st.error("\n".join(errs))
        else:
            res = predict(model, row, meta).iloc[0]
            m1, m2, m3 = st.columns(3)
            m1.metric("Probabilitas churn", f"{res.churn_probability:.1%}")
            m2.metric("Prediksi", "Churn" if res.predicted_churn else "Bertahan")
            m3.metric("Kategori risiko", f"{COLOR[res.risk_category]} {res.risk_category}")
            st.progress(float(res.churn_probability))
            if res.risk_category == "Tinggi":
                st.error("Rekomendasi: prioritaskan retensi — tawarkan migrasi ke kontrak tahunan "
                         "dan tindak lanjuti keluhan dalam 48 jam.")
            elif res.risk_category == "Sedang":
                st.warning("Rekomendasi: pantau, kirim survei kepuasan dan penawaran loyalitas.")
            else:
                st.success("Rekomendasi: pertahankan layanan standar.")

# 3) PREDIKSI BATCH — upload CSV pelanggan baru
with tab_batch:
    up = st.file_uploader("Upload CSV pelanggan baru (format sama dengan data/new_customers.csv)",
                          type="csv")
    pakai_contoh = st.checkbox("Gunakan contoh data/new_customers.csv", value=up is None)
    if up is not None:
        data = pd.read_csv(up)
    elif pakai_contoh:
        data = pd.read_csv(ROOT / "data" / "new_customers.csv")
    else:
        st.info("Upload file atau centang contoh data.")
        st.stop()

    errs = validate_input(data, meta)
    if errs:
        for e in errs:
            st.error(e)
        st.stop()

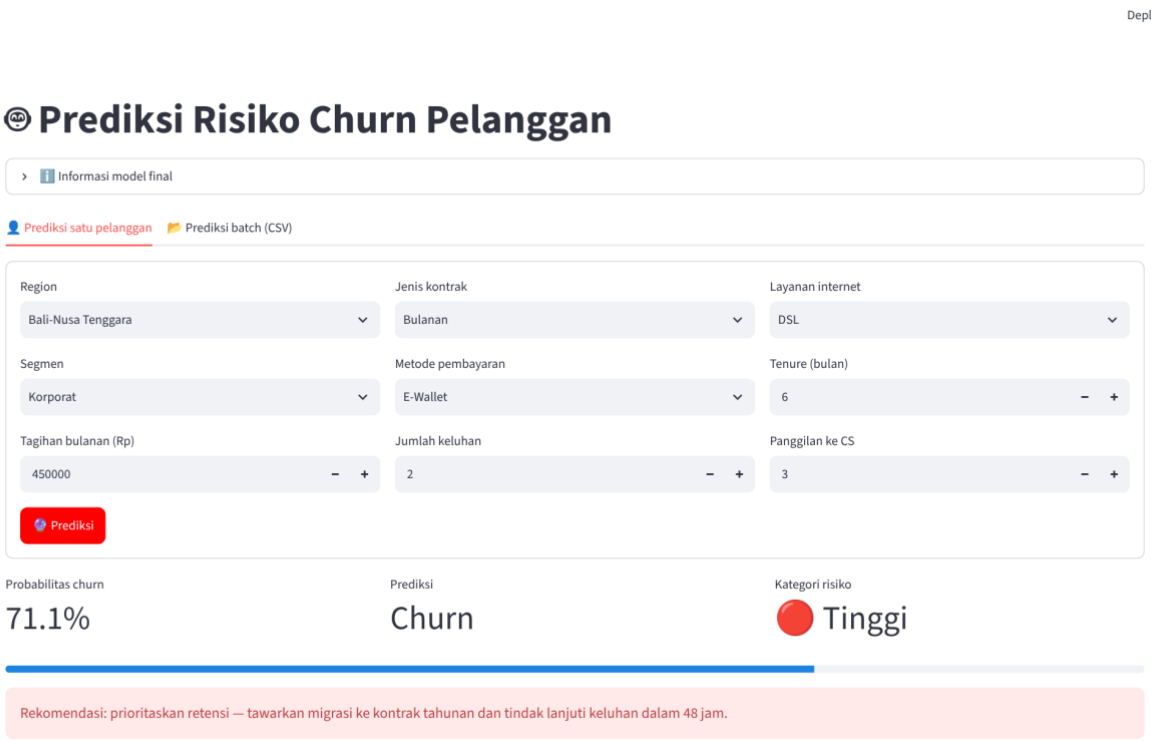
    hasil = predict(model, data, meta)
    k1, k2, k3, k4 = st.columns(4)
    k1.metric("Pelanggan diproses", f"{len(hasil):,}")
    k2.metric("Prediksi churn", f"{hasil.predicted_churn.sum():,}")
    k3.metric("Rata-rata probabilitas", f"{hasil.churn_probability.mean():.1%}")
    k4.metric("Risiko tinggi", f"{(hasil.risk_category == 'Tinggi').sum():,}")

    RISK_COLOR = {"Rendah": "#1baf7a", "Sedang": "#eda100", "Tinggi": "#e34948"}
    a, b = st.columns(2)
    donut = px.pie(hasil, names="risk_category", hole=0.55, color="risk_category",
                   color_discrete_map=RISK_COLOR, title="Komposisi kategori risiko",
                   category_orders={"risk_category": ["Rendah", "Sedang", "Tinggi"]})
    donut.update_traces(textinfo="percent+label")
    a.plotly_chart(donut.update_layout(height=330, showlegend=False), width="stretch")
    pk = hasil.groupby("contract_type", as_index=False)["churn_probability"].mean()
    bar = px.bar(pk, x="contract_type", y="churn_probability", text_auto=".1%",
                 color_discrete_sequence=["#eb6834"], title="Rata-rata probabilitas churn per kontrak")
    bar.update_yaxes(tickformat=".0%")
    b.plotly_chart(bar.update_layout(height=330, template="plotly_white", xaxis_title=""),
                   width="stretch")

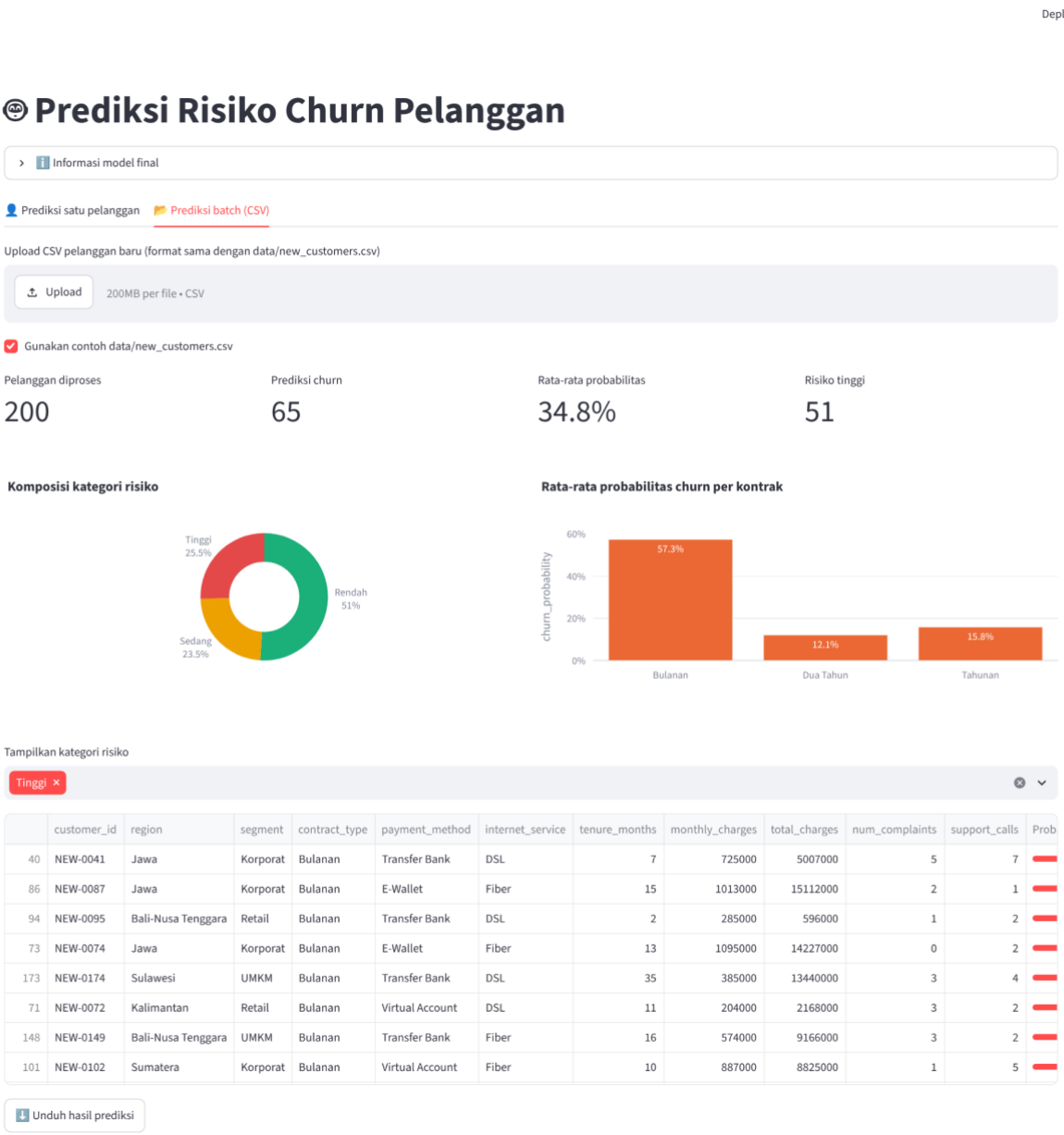
    risiko = st.multiselect("Tampilkan kategori risiko", ["Tinggi", "Sedang", "Rendah"],
                            default=["Tinggi"])
    tampil = hasil[hasil.risk_category.isin(risiko)].sort_values("churn_probability",
                                                                 ascending=False)
```

```
    st.dataframe(tampil, width="stretch", height=320,
                 column_config={"churn_probability": st.column_config.ProgressColumn(
                     "Probabilitas churn", min_value=0.0, max_value=1.0, format="%.2f")})
    st.download_button("⬇ Unduh hasil prediksi", hasil.to_csv(index=False).encode("utf-8"),
                       "hasil_prediksi_churn.csv", "text/csv")
```

Jalankan `streamlit run sesi4/app_prediksi.py` (lihat Gambar 4.1, dan 4.2).



Gambar 4.1. *Prediksi satu pelanggan di Streamlit*



Gambar 4.2. *Prediksi batch: KPI, donut kategori risiko, bar probabilitas, dan tabel progres*

Penjelasan kode

- `@st.cache_resource` dipakai untuk objek yang tidak boleh disalin, seperti model ML dan koneksi database. Model dimuat sekali saat aplikasi pertama dibuka, lalu dipakai bersama oleh semua pengguna. `@st.cache_data` dipakai untuk data (metadata dict).
- Pilihan dropdown diambil dari `meta["categories"]`, bukan diketik manual. Dengan begitu pengguna tidak mungkin memasukkan kategori yang tidak dikenal model.
- `st.form` menahan rerun sampai tombol **Prediksi** diklik, sehingga model tidak dipanggil setiap kali satu input diubah.
- `total_charges` diisi `monthly * tenure` karena pengguna tidak diminta memasukkannya; ini aturan penyiapan input yang harus didokumentasikan.
- Di tab batch, `validate_input()` dijalankan sebelum prediksi. Jika ada kolom hilang, pesan error spesifik ditampilkan dan aplikasi berhenti dengan `st.stop()`.
- Donut `px.pie(..., hole=0.55, color_discrete_map=RISK_COLOR)` memperlihatkan komposisi kategori risiko dengan warna tetap: hijau Rendah, kuning Sedang, merah Tinggi.

- `st.column_config.ProgressColumn` menampilkan probabilitas sebagai bar di dalam tabel, sehingga pelanggan paling berisiko langsung terlihat.

Checkpoint

1. Form dengan nilai default (Bali-Nusa Tenggara, Korporat, Bulanan, E-Wallet, DSL, tenure 6, tagihan 450.000, keluhan 2, CS 3) menghasilkan **probabilitas 71,1%, Prediksi Churn, kategori Tinggi**.
2. Ubah kontrak menjadi Dua Tahun dan tenure 48, lalu klik Prediksi: probabilitas harus turun drastis.
3. Tab batch dengan contoh data: 200 pelanggan diproses, 65 diprediksi churn, rata-rata probabilitas 34,8%, dan 51 pelanggan risiko tinggi.

4.6 Menyajikan prediction output secara bertanggung jawab

- Tampilkan probabilitas, bukan hanya label. Pelanggan dengan 0,51 dan 0,95 sama-sama "Churn", tetapi prioritasnya sangat berbeda.
- Jelaskan arti kategori risiko dan threshold di dashboard (expander informasi model).
- Cantumkan versi model agar hasil dapat ditelusuri.
- Hubungkan setiap kategori dengan tindakan yang jelas (rekomendasi).
- Hindari menyebut prediksi sebagai kepastian; gunakan kata "berisiko" atau "diperkirakan".

Latihan mandiri Sesi 4

1. Ubah threshold keputusan menjadi `st.slider` (0,3–0,7) di sidebar dan lihat dampaknya pada jumlah "Prediksi churn" di tab batch.
2. Buat file CSV berisi 3 pelanggan dengan satu nilai `contract_type` salah (misalnya "Mingguan"). Upload dan pastikan pesan validasi muncul.
3. Tambahkan tabel ringkasan jumlah pelanggan per `region` × `risk_category` (`pd.crosstab`).
4. Tantangan: tampilkan potensi pendapatan berisiko = jumlah `monthly_charges` pelanggan berkategori Tinggi.

Kesalahan umum Sesi 4

Rinciannya disajikan pada Tabel 4.2.

Tabel 4.2. *Kesalahan umum Sesi 4*

Gejala	Penyebab	Solusi
`ModuleNotFoundError: No module named 'utils'`	Folder akar proyek tidak ada di `sys.path`	Pertahankan baris `sys.path.append(str(ROOT))` sebelum import
`InconsistentVersionWarning` atau error saat `joblib.load`	Versi scikit-learn berbeda dengan saat model disimpan	Instal versi yang tercatat di `model_metadata.json`
`ValueError: columns are missing`	Nama kolom input berbeda dari saat training	Jalankan `validate_input()`; samakan nama kolom
Prediksi aneh walau tidak error	Preprocessing dilakukan dua kali (scaling manual + Pipeline)	Kirim data mentah ke Pipeline; jangan scale sendiri
Aplikasi lambat setiap klik	Model dimuat ulang tiap rerun	Gunakan `@st.cache_resource`

Sesi 5 — Dash Fundamentals (90 menit)

Sesi ini memperkenalkan Dash dan menghasilkan dashboard statis pertama: header, empat KPI card, grafik bar, dan tabel ringkasan. Interaktivitas melalui callback baru ditambahkan di Sesi 7.

Tujuan sesi

- Menjelaskan arsitektur Dash: Flask (server), React (tampilan), Plotly (grafik).
- Memahami struktur aplikasi Dash: objek `app`, `app.layout`, callback, dan `app.run()`.
- Menggunakan Dash HTML Components (`html.*`), Dash Core Components (`dcc.*`), dan `dash_table.DataTable`.
- Mengatur tata letak dengan properti `style` (CSS flexbox dan grid).
- Menjalankan aplikasi Dash dengan mode debug dan hot reload.

5.1 Perbedaan cara berpikir Streamlit dan Dash

Di Streamlit, script dijalankan ulang dari atas setiap ada interaksi. Di Dash, aplikasi dibagi menjadi dua bagian yang terpisah tegas:

1. **Layout** — pohon komponen yang mendeskripsikan tampilan. Didefinisikan sekali.
2. **Callback** — fungsi Python yang dipanggil hanya ketika properti komponen tertentu berubah, lalu mengembalikan nilai baru untuk properti komponen lain.

Pemisahan ini membuat Dash lebih efisien untuk dashboard besar dengan banyak pengguna, karena hanya bagian yang terkait yang dihitung ulang.

5.2 Struktur minimal aplikasi Dash

```python
from dash import Dash, html

app = Dash(__name__)                    # 1. buat aplikasi
app.layout = html.Div([                 # 2. definisikan layout
    html.H1("Halo, Dash!"),
    html.P("Aplikasi Dash pertama saya."),
])

if __name__ == "__main__":              # 3. jalankan server
    app.run(debug=True)
```

Simpan sebagai `sesi5/app_halo.py`, lalu jalankan dengan **python** (bukan `streamlit run`):

```bash
python sesi5/app_halo.py
```

Buka `http://127.0.0.1:8050` di browser. Hentikan dengan `Ctrl + C`.

Catatan versi: Dash 3 ke atas memakai `app.run()`. Perintah lama `app.run_server()` sudah dihapus.

5.3 Tiga keluarga komponen

Rinciannya disajikan pada Tabel 5.1.

Tabel 5.1. *Tiga keluarga komponen*

Keluarga	Import	Isi	Contoh
HTML Components	`from dash import html`	Padanan tag HTML	`html.Div`, `html.H1`, `html.P`, `html.Button`, `html.Label`, `html.Hr`
Core Components	`from dash import dcc`	Komponen interaktif	`dcc.Graph`, `dcc.Dropdown`, `dcc.Slider`, `dcc.RangeSlider`, `dcc.RadioItems`, `dcc.Checklist`, `dcc.Input`, `dcc.DatePickerRange`, `dcc.Tabs`, `dcc.Store`
DataTable	`from dash import dash_table`	Tabel interaktif	`dash_table.DataTable` dengan sort, filter, paging

Setiap komponen menerima argumen yang disebut **properti**. Tiga properti penting:

- `children` — isi komponen: teks, komponen lain, atau list komponen. Selalu argumen pertama.
- `id` — nama unik komponen; wajib bagi komponen yang akan dihubungkan dengan callback.
- `style` — dictionary CSS dengan nama properti camelCase, contoh `{"fontSize": "14px", "marginTop": "8px"}`.

5.4 Praktikum — Dashboard Dash statis

Buat `sesi5/app_dash_dasar.py`:

PYTHON

```python
# sesi5/app_dash_dasar.py — struktur dasar aplikasi Dash (tanpa callback)
from pathlib import Path
import pandas as pd
from dash import Dash, html, dcc, dash_table

ROOT = Path(__file__).resolve().parents[1]
df = pd.read_csv(ROOT / "data" / "customer_churn.csv", parse_dates=["signup_date"])

ringkas = (df.groupby("contract_type")
             .agg(pelanggan=("customer_id", "count"), churn_rate=("churn", "mean"))
             .reset_index())
ringkas["churn_rate"] = (ringkas["churn_rate"] * 100).round(1)

# 1. Membuat objek aplikasi
app = Dash(__name__, title="Dash Dasar")

# 2. Gaya sederhana (dict Python = CSS inline)
KARTU = {"flex": 1, "padding": "16px", "borderRadius": "10px",
         "background": "#F4F6FA", "textAlign": "center"}

def kartu(judul, nilai):
    return html.Div([html.P(judul, style={"margin": 0, "color": "#555"}),
                     html.H2(nilai, style={"margin": "4px 0 0 0"})], style=KARTU)

# 3. Layout = pohon komponen
app.layout = html.Div(style={"fontFamily": "Arial, sans-serif", "maxWidth": "1100px",
                             "margin": "auto", "padding": "20px"}, children=[
    html.H1("Dashboard Churn — Versi Dash"),
    html.P("Contoh layout statis: header, KPI cards, grafik, dan tabel."),

    html.Div(style={"display": "flex", "gap": "12px"}, children=[
        kartu("Total pelanggan", f"{len(df):,}"),
        kartu("Churn rate", f"{df.churn.mean():.1%}"),
        kartu("Rata-rata tenure", f"{df.tenure_months.mean():.1f} bln"),
        kartu("Region", df.region.nunique()),
    ]),

    html.H3("Churn rate per jenis kontrak"),
    dcc.Graph(figure={
        "data": [{"type": "bar", "x": ringkas["contract_type"], "y": ringkas["churn_rate"],
                  "marker": {"color": "#C44E52"}}],
```

```
        "layout": {"yaxis": {"title": "Churn rate (%)"}, "height": 350},
    }),

    html.H3("Ringkasan per kontrak"),
    dash_table.DataTable(data=ringkas.to_dict("records"),
                         columns=[{"name": c, "id": c} for c in ringkas.columns],
                         style_cell={"padding": "6px", "textAlign": "left"},
                         style_header={"fontWeight": "bold", "background": "#E9EDF5"}),

    html.Hr(),
    html.Small("Sumber: data latihan sintetis customer_churn.csv"),
])

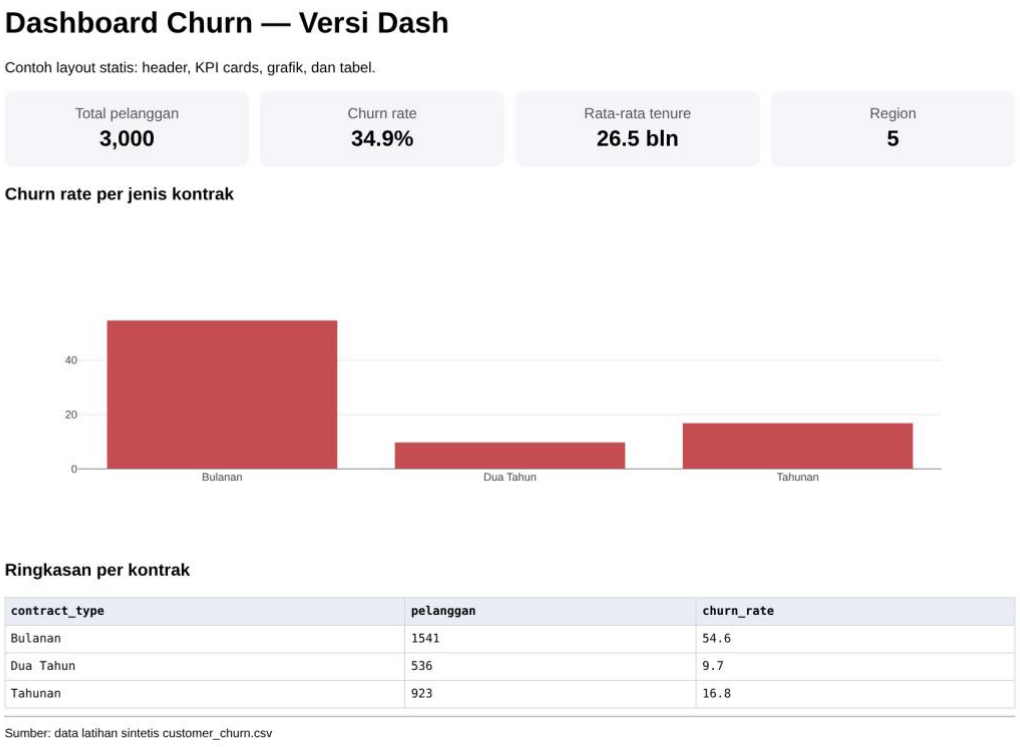
# 4. Menjalankan server
if __name__ == "__main__":
    app.run(debug=True, port=8050)
```

Jalankan `python sesi5/app_dash_dasar.py` dan buka `http://127.0.0.1:8050`.

Penjelasan kode

- Data dibaca dan diringkas **sekali** saat aplikasi dimulai, di luar layout. Ini pola standar Dash untuk data yang tidak berubah.
- Fungsi `kartu()` adalah komponen buatan sendiri: fungsi Python yang mengembalikan `html.Div`. Pola ini membuat KPI card konsisten dan mudah diulang.
- `"display": "flex"` pada Div pembungkus membuat empat kartu berjajar; `"flex": 1` pada setiap kartu membuat lebarnya sama.
- `dcc.Graph(figure=...)` menerima figure dalam bentuk dictionary. Di Sesi 6 dictionary ini diganti dengan objek Plotly Express yang jauh lebih ringkas.
- `DataTable` membutuhkan `data` dalam format list of dict (`df.to_dict("records")`) dan definisi `columns`.
- `debug=True` mengaktifkan hot reload (aplikasi memuat ulang otomatis saat file disimpan) dan panel debug di pojok kanan bawah yang menampilkan error callback. Matikan `debug` saat aplikasi dipresentasikan atau di-deploy.

Checkpoint: halaman menampilkan judul, empat kartu (3,000 pelanggan; 34.9%; 26.5 bln; 5 region), grafik bar dengan Bulanan tertinggi (54,6%), dan tabel tiga baris (lihat Gambar 5.1).



Gambar 5.1. *Dashboard Dash statis Sesi 5*

5.5 Pola dasar tata letak dashboard di Dash

Dua teknik CSS yang cukup untuk hampir semua dashboard:

```python
# Baris berjajar (KPI cards)
html.Div(style={"display": "flex", "gap": "12px"}, children=[kartu1, kartu2, kartu3])

# Grid dua kolom (grafik)
html.Div(style={"display": "grid", "gridTemplateColumns": "1fr 1fr", "gap": "12px"},
         children=[dcc.Graph(id="g1"), dcc.Graph(id="g2")])

# Sidebar + konten
html.Div(style={"display": "flex"}, children=[
    html.Div(sidebar, style={"width": "250px"}),
    html.Div(konten, style={"flex": 1}),
])
```

Untuk gaya yang dipakai berulang, simpan CSS di file `assets/style.css` pada folder yang sama dengan aplikasi. Dash otomatis memuat semua file `.css` di folder `assets/`. Alternatif lain adalah library `dash-bootstrap-components` yang menyediakan sistem grid Bootstrap (`dbc.Row`, `dbc.Col`, `dbc.Card`).

Latihan mandiri Sesi 5

1. Tambahkan KPI card kelima: rata-rata jumlah keluhan.
2. Tambahkan grafik kedua (churn rate per region) dan susun kedua grafik berdampingan dengan CSS grid.
3. Aktifkan sorting dan paging pada DataTable dengan `sort_action="native"` dan `page_size=5`.
4. Pindahkan dictionary gaya ke file `sesi5/assets/style.css` dan gunakan `className` pada komponen.

Kesalahan umum Sesi 5

Rinciannya disajikan pada Tabel 5.2.

Tabel 5.2. *Kesalahan umum Sesi 5*

Gejala	Penyebab	Solusi
`AttributeError: 'Dash' object has no attribute 'run_server'`	Dash versi 3 ke atas	Ganti dengan `app.run()`
`Address already in use`	Port 8050 masih dipakai	Hentikan aplikasi lama atau gunakan `app.run(port=8051)`
Gaya tidak berlaku	Nama CSS memakai tanda hubung (`font-size`)	Gunakan camelCase (`fontSize`) di dict `style`
Error "Invalid argument children"	`children` berisi objek yang bukan teks/komponen, misalnya DataFrame	Ubah DataFrame menjadi DataTable atau teks
Halaman kosong dan error di panel debug	Komponen ber-`id` sama	Pastikan setiap `id` unik

Sesi 6 — Interactive Visualization with Plotly (90 menit)

Sesi ini menghasilkan 13 chart interaktif Plotly (bar, pie, donut, histogram, scatter facet, line dengan range slider, box, bar Graph Objects, heatmap, treemap, sunburst, funnel, dan gauge) dan menampilkannya di aplikasi Dash. Setiap chart diberi judul berupa insight, bukan sekadar nama variabel.

Tujuan sesi

- Membedakan Plotly Express (cepat, ringkas) dan Graph Objects (kontrol penuh).
- Membuat chart interaktif dengan hover, zoom, legend toggle, dan range slider.
- Mengatur hover dengan `hover_name`, `hover_data`, dan `hovertemplate`.
- Menggunakan visual encoding: `color`, `size`, `facet_col`, `barmode`.
- Membuat filter visual tanpa callback (legend dan `updatemenus`).
- Menulis judul chart yang menyampaikan insight.

6.1 Plotly Express vs Graph Objects

Rinciannya disajikan pada Tabel 6.1.

Tabel 6.1. *Plotly Express vs Graph Objects*

Aspek	Plotly Express (`px`)	Graph Objects (`go`)
Gaya	Satu fungsi satu chart, input DataFrame	Bangun figure dari trace satu per satu
Panjang kode	Pendek	Lebih panjang
Cocok untuk	90% kebutuhan dashboard	Gauge/indicator, anotasi khusus, kustomisasi detail
Contoh	`px.bar(df, x="region", y="churn")`	`go.Figure(go.Bar(x=..., y=...))`

Keduanya menghasilkan objek `go.Figure` yang sama, sehingga figure dari `px` tetap dapat diubah dengan `fig.update_layout()`, `fig.update_traces()`, `fig.update_xaxes()`, dan `fig.add_hline()`.

6.2 Interaksi bawaan Plotly

Setiap chart Plotly sudah interaktif tanpa kode tambahan:

- **Hover** — arahkan kursor ke titik/batang untuk melihat nilai.
- **Zoom** — drag area pada chart; double-click untuk kembali.
- **Legend toggle** — klik item legenda untuk menyembunyikan/menampilkan kelompok; double-click untuk menampilkan hanya kelompok itu. Ini adalah filter visual paling sederhana.
- **Modebar** — ikon di pojok kanan atas: unduh PNG, pan, box select, lasso select, reset.

6.3 Praktikum A — Galeri chart Plotly

Buat `sesi6/plotly_galeri.py`. Semua figure dibuat di fungsi `buat_figur()` yang mengembalikan dictionary, sehingga figure yang sama dapat dipakai ulang di aplikasi Dash. Script ini menyimpan 13 chart sebagai file HTML interaktif di `sesi6/output/` yang dapat dibuka langsung di browser.

```python
# sesi6/plotly_galeri.py — galeri 12 chart Plotly (jalankan: python sesi6/plotly_galeri.py)
```

```
from pathlib import Path
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go

ROOT = Path(__file__).resolve().parents[1]
STATUS = {"Bertahan": "#2a78d6", "Churn": "#eb6834"}
KATEGORI = ["#2a78d6", "#1baf7a", "#eda100", "#4a3aa7", "#e87ba4"]
KONTRAK = ["Bulanan", "Tahunan", "Dua Tahun"]



def load():
    df = pd.read_csv(ROOT / "data" / "customer_churn.csv", parse_dates=["signup_date"])
    df["status"] = df["churn"].map({0: "Bertahan", 1: "Churn"})
    df["tagihan_ribu"] = df["monthly_charges"] / 1000
    return df



def buat_figur(df):
    fig = {}
    # 1. Bar chart (Plotly Express) + hover kustom
    bar = df.groupby(["contract_type", "segment"], as_index=False)["churn"].mean()
    bar["churn_pct"] = bar["churn"] * 100
    f = px.bar(bar, x="contract_type", y="churn_pct", color="segment", barmode="group",
               color_discrete_sequence=KATEGORI, category_orders={"contract_type": KONTRAK},
               labels={"contract_type": "Jenis kontrak", "churn_pct": "Churn rate (%)", "segment": "Segmen"},
               title="Kontrak bulanan memiliki churn rate tertinggi di semua segmen")
    f.update_traces(hovertemplate="Kontrak: %{x}<br>Churn rate: %{y:.1f}%<extra></extra>")
    fig["bar"] = f

    # 2. Pie chart
    f = px.pie(df, names="status", color="status", color_discrete_map=STATUS,
               title="Proporsi status pelanggan")
    f.update_traces(textinfo="percent+label", pull=[0, 0.06])
    fig["pie"] = f

    # 3. Donut chart dengan angka di tengah
    f = px.pie(df, names="internet_service", hole=0.55, color_discrete_sequence=KATEGORI,
               title="Komposisi layanan internet")
    f.add_annotation(text=f"<b>{len(df):,}</b><br>pelanggan", showarrow=False, font_size=15)
    fig["donut"] = f

    # 4. Histogram dengan warna kategori
    fig["histogram"] = px.histogram(
        df, x="tenure_months", color="status", nbins=36, barmode="overlay", opacity=0.7,
        color_discrete_map=STATUS, labels={"tenure_months": "Tenure (bulan)", "status": "Status"},
        title="Churn terkonsentrasi pada pelanggan dengan tenure pendek")

    # 5. Scatter interaktif: hover_data, facet
    fig["scatter"] = px.scatter(
        df, x="tenure_months", y="tagihan_ribu", color="status", color_discrete_map=STATUS,
        hover_name="customer_id", hover_data={"region": True, "num_complaints": True, "tagihan_ribu":
":.0f"},
        facet_col="contract_type", category_orders={"contract_type": KONTRAK}, opacity=0.6,
        labels={"tenure_months": "Tenure (bulan)", "tagihan_ribu": "Tagihan (ribu Rp)"},
        title="Tenure vs tagihan per jenis kontrak")

    # 6. Line chart tren bulanan + range slider
    tren = (df.set_index("signup_date").groupby([pd.Grouper(freq="ME"), "status"])
              .size().reset_index(name="jumlah"))
    f = px.line(tren, x="signup_date", y="jumlah", color="status", markers=True,
                color_discrete_map=STATUS, labels={"signup_date": "Bulan pendaftaran", "jumlah":
"Pelanggan"},
                title="Pelanggan baru per bulan menurut status akhir")
    f.update_xaxes(rangeslider_visible=True)
    fig["line"] = f

    # 7. Box plot
    fig["box"] = px.box(
        df, x="num_complaints", y="tagihan_ribu", color="status", color_discrete_map=STATUS,
        labels={"num_complaints": "Jumlah keluhan", "tagihan_ribu": "Tagihan (ribu Rp)"},
        title="Distribusi tagihan menurut jumlah keluhan")
```

```
    # 8. Graph Objects: bar dengan anotasi teks + garis referensi rata-rata
    reg = df.groupby("region", as_index=False)["churn"].mean().sort_values("churn", ascending=False)
    f = go.Figure(go.Bar(x=reg["region"], y=reg["churn"] * 100, marker_color="#eb6834",
                         text=(reg["churn"] * 100).round(1).astype(str) + "%", textposition="outside"))
    f.add_hline(y=df.churn.mean() * 100, line_dash="dot",
                annotation_text="rata-rata nasional", annotation_position="top right")
    f.update_layout(title="Churn rate per region (Graph Objects)", yaxis_title="Churn rate (%)",
                    yaxis_range=[0, 45])
    fig["go_bar"] = f

    # 9. Heatmap churn rate region x segmen
    pv = pd.pivot_table(df, values="churn", index="region", columns="segment", aggfunc="mean") * 100
    fig["heatmap"] = px.imshow(pv.round(1), text_auto=True, color_continuous_scale="Oranges",
                               aspect="auto", labels={"color": "Churn (%)"},
                               title="Heatmap churn rate region × segmen")

    # 10. Treemap & 11. Sunburst — data hierarkis
    cnt = df.groupby(["region", "segment", "contract_type"], as_index=False).agg(
        pelanggan=("customer_id", "count"), churn_rate=("churn", "mean"))
    fig["treemap"] = px.treemap(cnt, path=[px.Constant("Semua"), "region", "segment"],
                                values="pelanggan", color="churn_rate",
                                color_continuous_scale="Oranges",
                                title="Treemap: luas = pelanggan, warna = churn rate")
    fig["sunburst"] = px.sunburst(cnt, path=["contract_type", "segment"], values="pelanggan",
                                  color="contract_type", color_discrete_sequence=KATEGORI,
                                  title="Sunburst: kontrak → segmen")

    # 12. Funnel — dari seluruh pelanggan ke kelompok paling berisiko
    m1 = df.contract_type == "Bulanan"
    m2 = m1 & (df.tenure_months <= 12)
    m3 = m2 & (df.num_complaints >= 2)
    tahap = pd.DataFrame({
        "tahap": ["Semua pelanggan", "Kontrak bulanan", "+ tenure ≤ 12 bulan",
                  "+ keluhan ≥ 2", "Akhirnya churn"],
        "jumlah": [len(df), m1.sum(), m2.sum(), m3.sum(), (m3 & (df.churn == 1)).sum()]})
    f = px.funnel(tahap, x="jumlah", y="tahap", title="Funnel: penyempitan kelompok berisiko")
    f.update_traces(marker_color="#2a78d6", textinfo="value+percent initial")
    fig["funnel"] = f

    # 13. Indicator / gauge — KPI tunggal
    fig["gauge"] = go.Figure(go.Indicator(
        mode="gauge+number+delta", value=df.churn.mean() * 100, number={"suffix": "%"},
        delta={"reference": 30, "increasing": {"color": "#e34948"}, "suffix": " pp"},
        title={"text": "Churn rate vs target 30%"},
        gauge={"axis": {"range": [0, 100]}, "bar": {"color": "#eb6834"},
               "threshold": {"line": {"color": "black", "width": 3}, "value": 30}}))

    for f in fig.values():
        f.update_layout(template="plotly_white")
    return fig


if __name__ == "__main__":
    OUT = ROOT / "sesi6" / "output"; OUT.mkdir(exist_ok=True)
    figs = buat_figur(load())
    for nama, f in figs.items():
        f.write_html(OUT / f"{nama}.html", include_plotlyjs="cdn")
    print(f"{len(figs)} chart HTML tersimpan di", OUT)
```

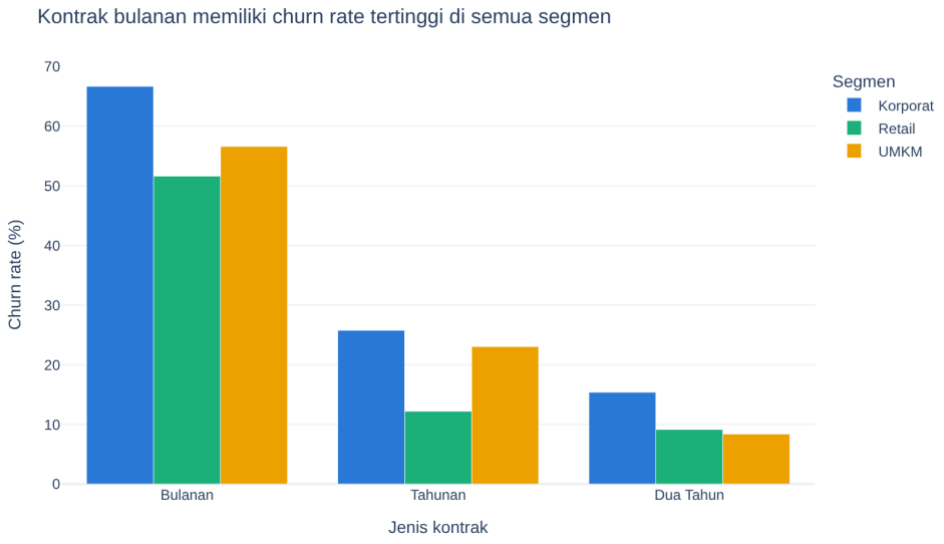
Jalankan `python sesi6/plotly_galeri.py`, lalu buka file `bar.html`, `pie.html`, sampai `gauge.html` di folder `sesi6/output/` dengan browser. Di Jupyter Notebook, cukup tulis `fig1.show()` atau `fig1` di akhir sel.

Penjelasan kode

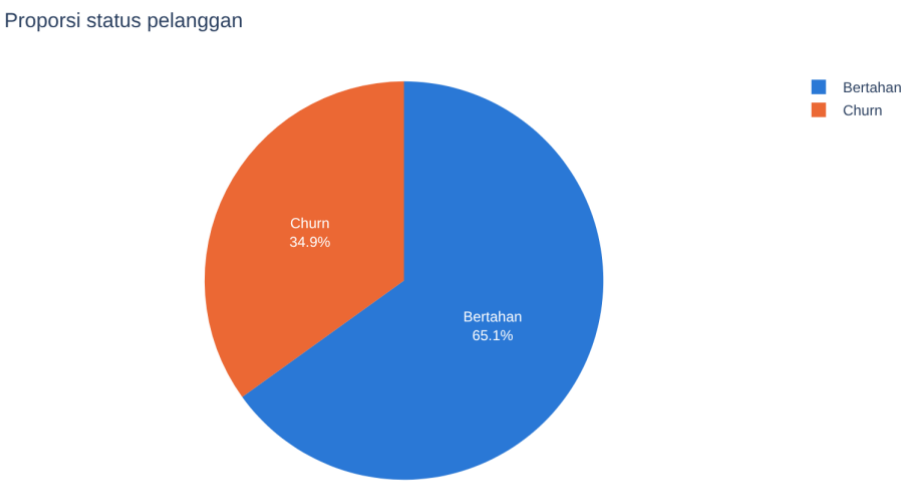
- **Data rapi (long format)** — Plotly Express bekerja paling baik jika setiap baris adalah satu observasi dan kategori ada di kolom, bukan di nama kolom. Karena itu data di-`groupby(..., as_index=False)` sebelum dikirim ke `px.bar`.
- `labels={...}` mengganti nama kolom teknis menjadi label yang dipahami pembaca, di sumbu, legenda, dan hover sekaligus.

- `category_orders` mengatur urutan kategori; tanpa ini Plotly memakai urutan kemunculan data.
- `color_discrete_map` mengunci warna per kategori (Churn selalu merah, Bertahan selalu biru) agar konsisten di semua chart.
- `hover_name` menjadi judul tebal di tooltip; `hover_data` menambah kolom, dan nilai seperti `":.0f"` mengatur format angka.
- `hovertemplate` memberi kontrol penuh atas isi tooltip. `%{x}` dan `%{y:.1f}` adalah placeholder; `<extra></extra>` menghilangkan kotak nama trace.
- `facet_col` memecah satu chart menjadi panel kecil per kategori (small multiples) untuk perbandingan yang adil.
- `rangeslider_visible=True` menambah slider di bawah line chart untuk memilih rentang waktu.
- `px.pie(..., hole=0.55)` mengubah pie menjadi donut; `pull=[0, 0.06]` menarik satu irisan keluar untuk penekanan.
- `px.treemap` dan `px.sunburst` memvisualkan data hierarkis lewat `path=[...]`; `px.Constant("Semua")` menambah simpul akar.
- `px.funnel` cocok untuk tahapan yang menyempit; `textinfo="value+percent initial"` menampilkan jumlah dan persentase terhadap tahap pertama.
- `go.Indicator(mode="gauge+number+delta")` membuat gauge KPI tunggal lengkap dengan selisih terhadap target.
- Contoh Graph Objects sengaja **tidak** memakai dua sumbu Y. Dua ukuran berbeda skala pada satu grafik mudah menyesatkan; pisahkan menjadi dua grafik.
- `add_hline` menambah garis referensi (rata-rata) yang membantu pembaca menilai nilai di atas atau di bawah normal.

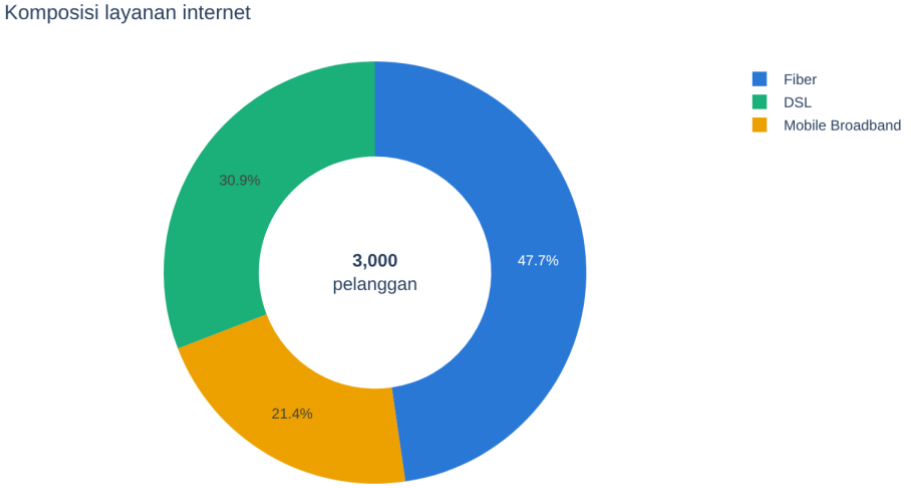
Hasil ke-13 chart (versi statis untuk buku; versi HTML dapat di-hover, di-zoom, dan difilter lewat legenda) (lihat Gambar 6.1, 6.2, 6.3, 6.4, 6.5, 6.6, 6.7, 6.8, 6.9, 6.10, 6.11, 6.12, dan 6.13):



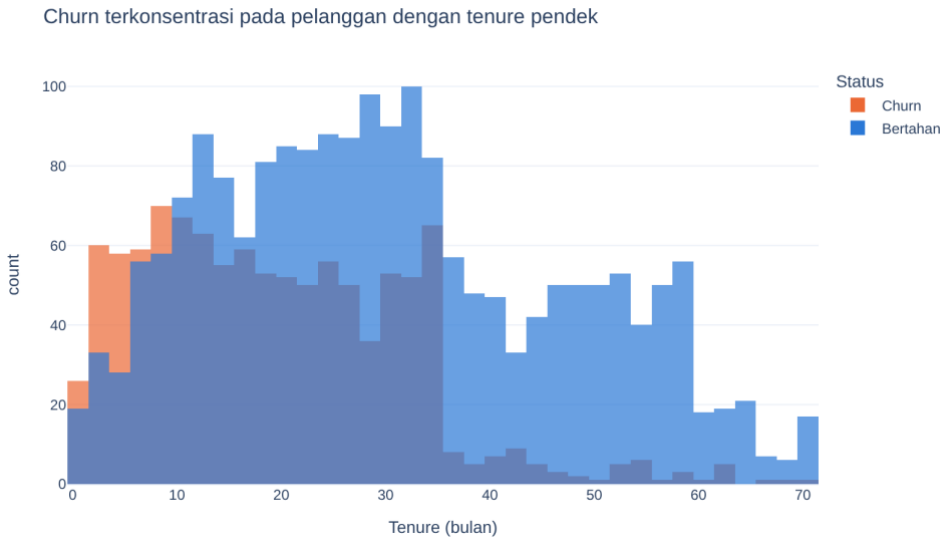
Gambar 6.1. *Bar chart berkelompok dengan hover kustom*



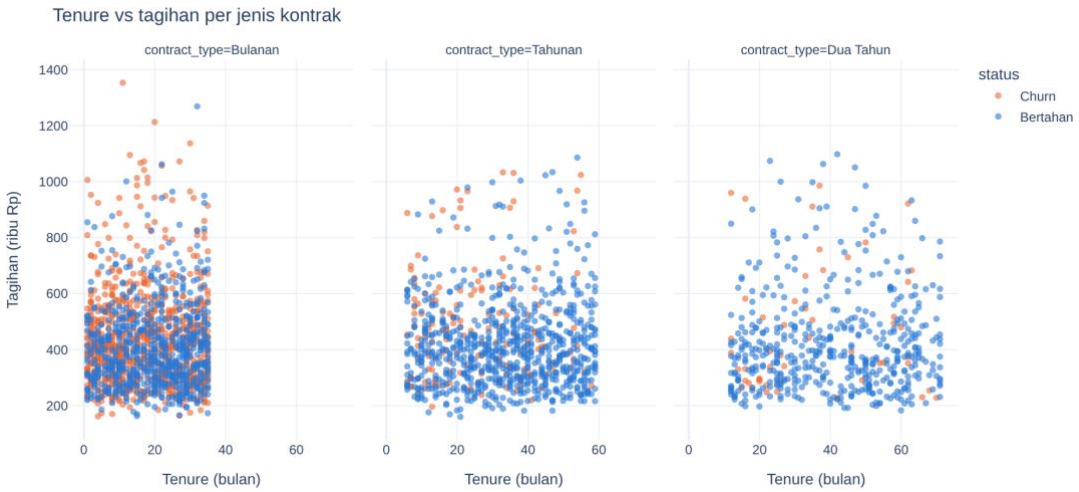
Gambar 6.2. *Pie chart proporsi status*



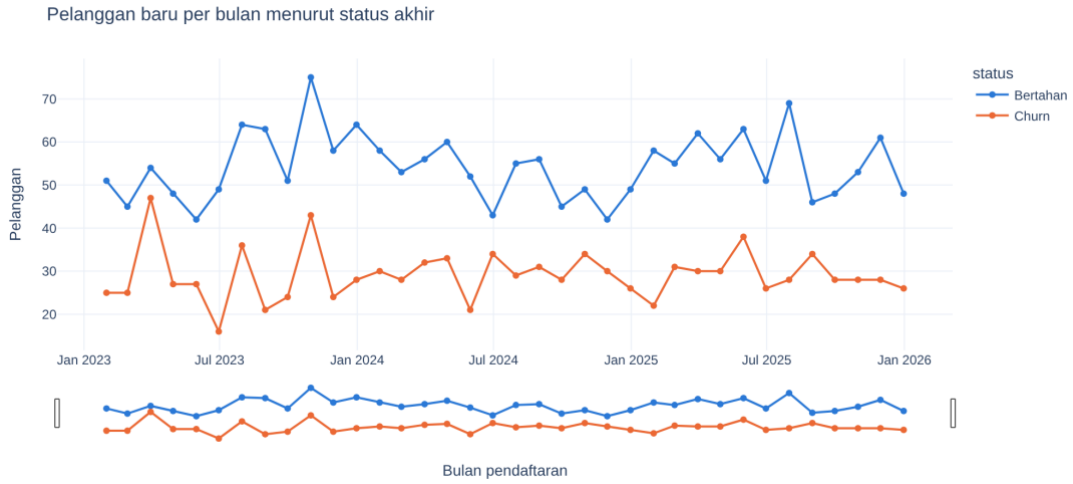
Gambar 6.3. *Donut chart dengan angka total di tengah*



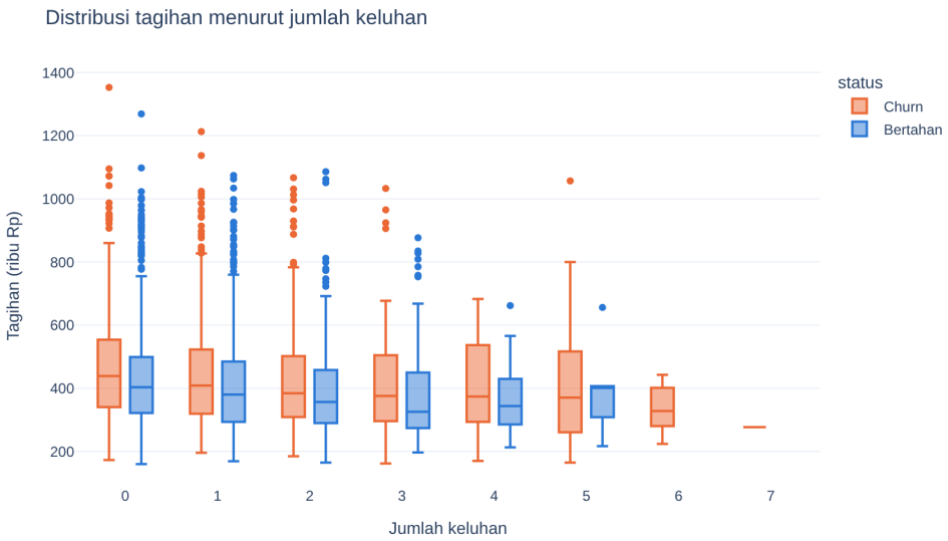
Gambar 6.4. *Histogram overlay per status*



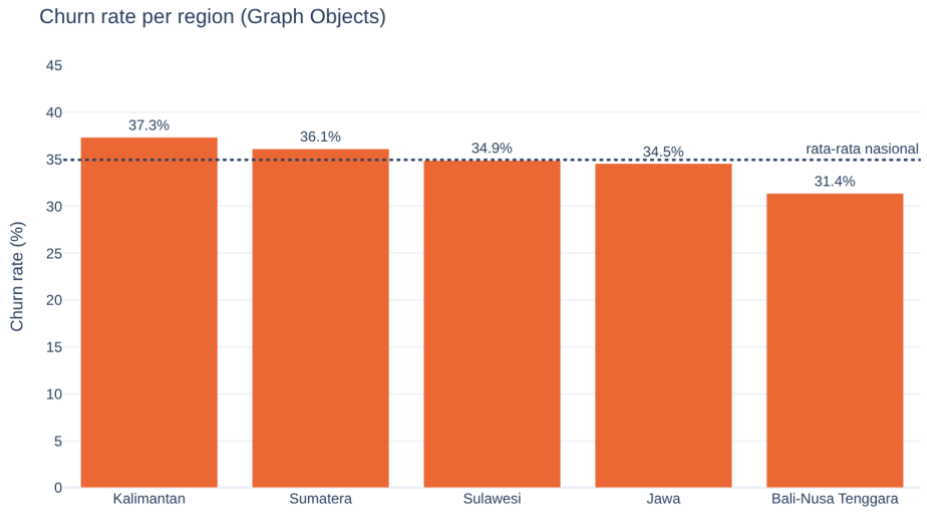
Gambar 6.5. *Scatter dengan facet per jenis kontrak (small multiples)*



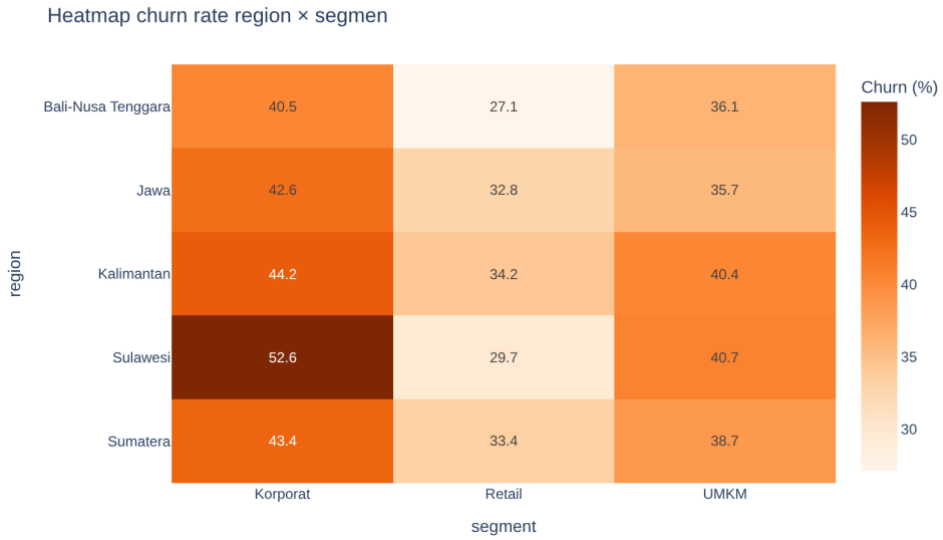
Gambar 6.6. *Line chart dengan range slider*



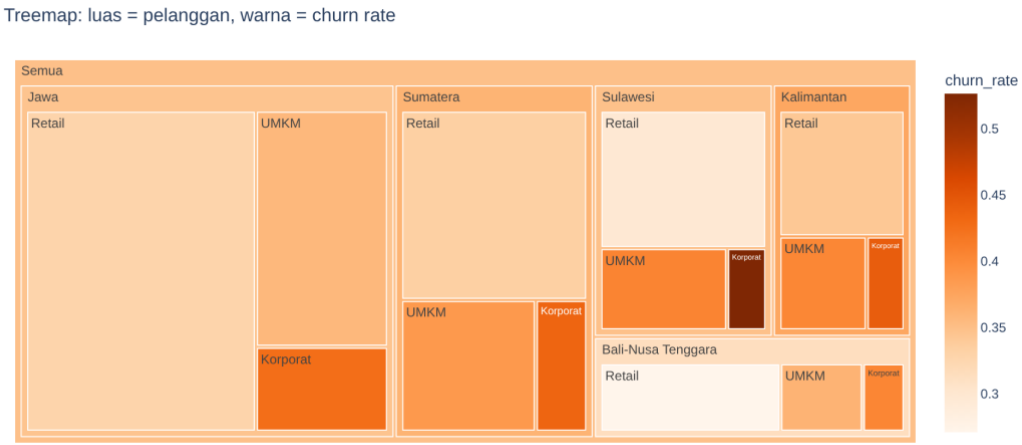
Gambar 6.7. *Box plot tagihan per jumlah keluhan*



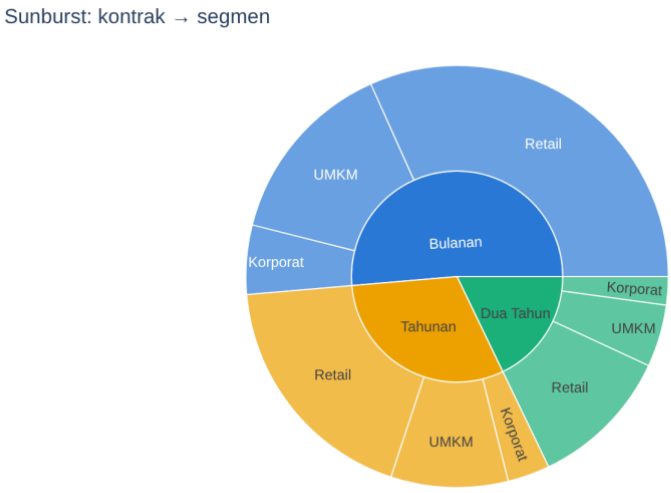
Gambar 6.8. *Bar Graph Objects dengan label dan garis rata-rata*



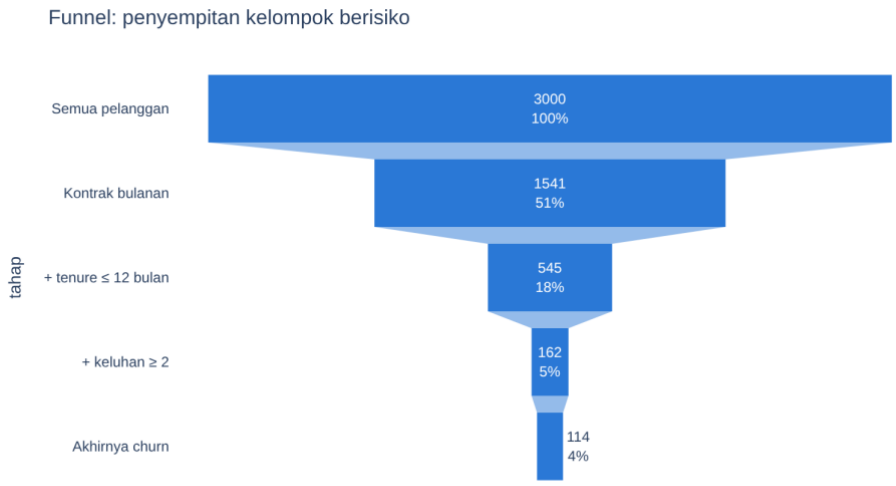
Gambar 6.9. *Heatmap churn rate region × segmen*



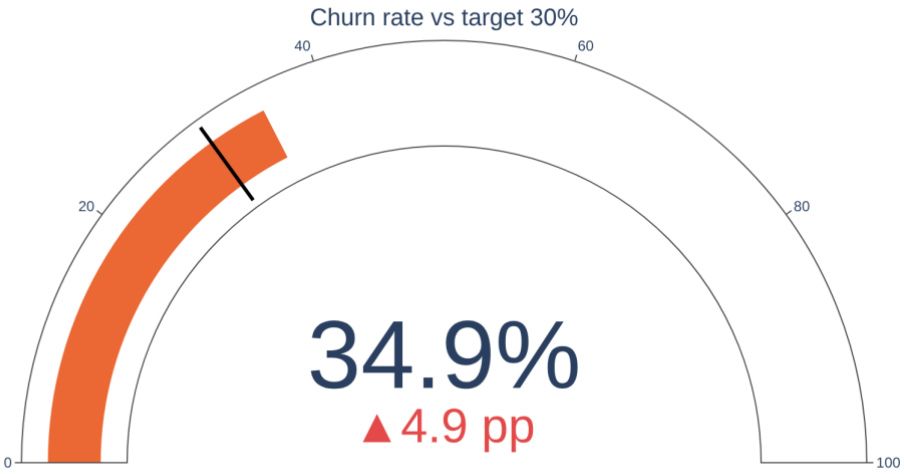
Gambar 6.10. *Treemap region → segmen*



Gambar 6.11. *Sunburst kontrak → segmen*



Gambar 6.12. *Funnel penyempitan kelompok berisiko*



Gambar 6.13. *Gauge/indicator churn rate terhadap target*

6.4 Praktikum B — Chart Plotly di aplikasi Dash

Buat `sesi6/app_dash_plotly.py`. Figure Plotly dimasukkan ke `dcc.Graph(figure=fig)`.

PYTHON

```python
# sesi6/app_dash_plotly.py — menampilkan chart Plotly di aplikasi Dash
from pathlib import Path
import pandas as pd
import plotly.express as px
from dash import Dash, html, dcc

ROOT = Path(__file__).resolve().parents[1]
df = pd.read_csv(ROOT / "data" / "customer_churn.csv", parse_dates=["signup_date"])
df["status"] = df["churn"].map({0: "Bertahan", 1: "Churn"})
WARNA = {"Bertahan": "#4C72B0", "Churn": "#C44E52"}

hist = px.histogram(df, x="tenure_months", color="status", nbins=36, barmode="overlay",
                    opacity=0.7, color_discrete_map=WARNA, template="plotly_white",
                    labels={"tenure_months": "Tenure (bulan)", "status": "Status"},
                    title="Churn terkonsentrasi pada tenure pendek")

scatter = px.scatter(df, x="tenure_months", y="monthly_charges", color="status",
                     color_discrete_map=WARNA, opacity=0.55, template="plotly_white",
                     hover_name="customer_id", hover_data=["region", "contract_type", "num_complaints"],
                     labels={"tenure_months": "Tenure (bulan)", "monthly_charges": "Tagihan (Rp)"},
                     title="Klik legenda untuk menyembunyikan/menampilkan kelompok")

# Filter visual tanpa callback: tombol dropdown bawaan Plotly (updatemenus)
bar_df = df.groupby(["region", "contract_type"], as_index=False)["churn"].mean()
bar = px.bar(bar_df, x="region", y="churn", color="contract_type", barmode="group",
             template="plotly_white", labels={"churn": "Churn rate", "region": "Region",
                                              "contract_type": "Kontrak"},
             title="Churn rate per region dan kontrak")
bar.update_yaxes(tickformat=".0%")
bar.update_layout(updatemenus=[dict(type="buttons", direction="right", x=1, y=1.15, buttons=[
    dict(label="Grouped", method="relayout", args=[{"barmode": "group"}]),
    dict(label="Stacked", method="relayout", args=[{"barmode": "stack"}]),
])])

app = Dash(__name__, title="Plotly di Dash")
app.layout = html.Div(style={"maxWidth": "1150px", "margin": "auto", "fontFamily": "Arial"}, children=[
    html.H1("Interactive Visualization with Plotly"),
    html.P("Arahkan kursor (hover), zoom dengan drag, double-click untuk reset, klik legenda untuk filter."),
    html.Div(style={"display": "grid", "gridTemplateColumns": "1fr 1fr", "gap": "12px"}, children=[
        dcc.Graph(figure=hist), dcc.Graph(figure=scatter)]),
    dcc.Graph(figure=bar, config={"displaylogo": False,
                                  "toImageButtonOptions": {"format": "png", "scale": 2}}),
])

if __name__ == "__main__":
    app.run(debug=True, port=8051)
```

Jalankan `python sesi6/app_dash_plotly.py` dan buka `http://127.0.0.1:8051`.

Penjelasan kode

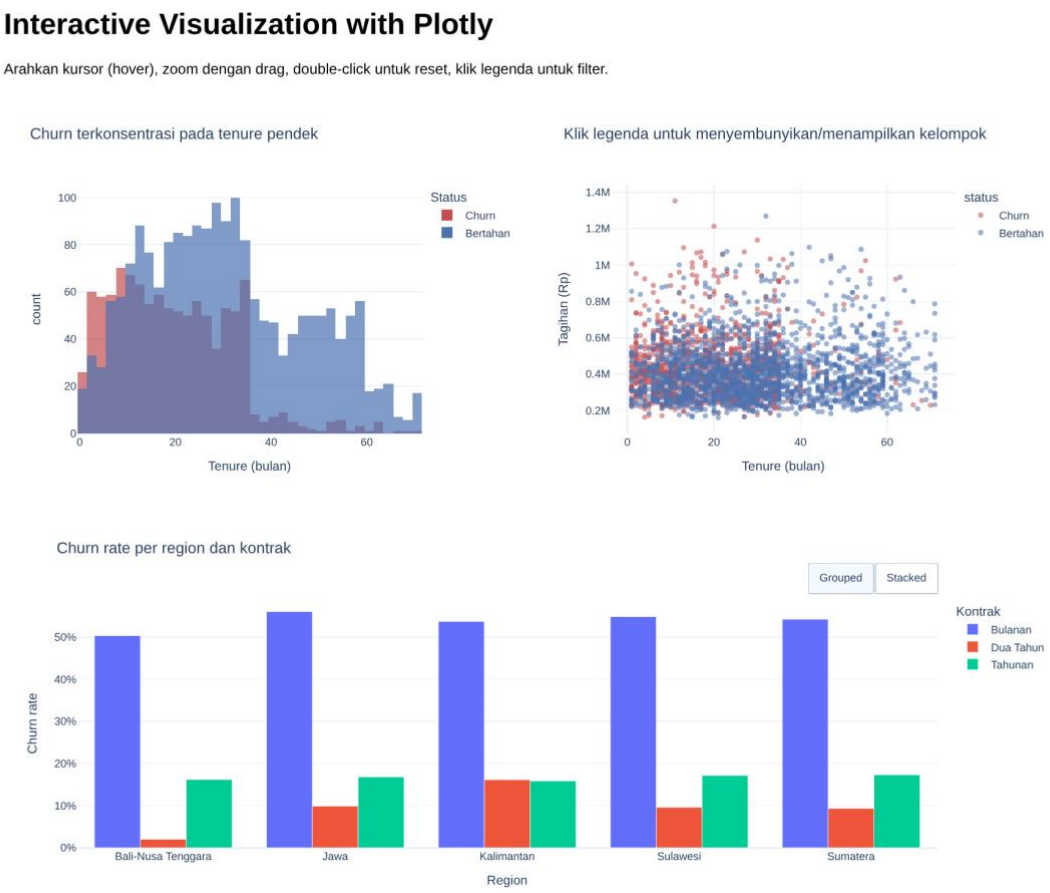
- `updatemenus` menambah tombol di dalam chart. `method="relayout"` mengubah properti layout (di sini `barmode`) langsung di browser tanpa memanggil Python.
- Properti `config` pada `dcc.Graph` mengatur modebar: menyembunyikan logo Plotly dan mengatur resolusi gambar unduhan (`scale: 2`).
- `tickformat=".0%"` menampilkan 0.546 sebagai 55% di sumbu.

Checkpoint

1. Arahkan kursor ke titik scatter: tooltip menampilkan ID pelanggan, region, kontrak, dan jumlah keluhan.
2. Klik "Bertahan" di legenda scatter: hanya titik Churn yang tersisa.

3. Klik tombol **Stacked** pada bar chart: batang berubah menjadi bertumpuk.

Hasilnya ditunjukkan pada Gambar 6.14.



Gambar 6.14. *Aplikasi Dash Sesi 6 dengan chart Plotly*

6.5 Dari chart ke insight

Chart yang baik menjawab satu pertanyaan. Gunakan daftar periksa berikut sebelum chart masuk dashboard:

1. **Judul = temuan.** "Kontrak bulanan memiliki churn rate tertinggi di semua segmen" lebih berguna daripada "Churn per kontrak".
2. **Jenis chart sesuai pertanyaan.** Perbandingan → bar; komposisi → pie/donut (2–5 kategori), treemap/sunburst (hierarki); tahapan → funnel; KPI tunggal → card/gauge; tren → line/area; distribusi → histogram/box/violin; hubungan → scatter; dua dimensi kategori → heatmap.
3. **Label lengkap dengan satuan.** "Tagihan (ribu Rp)", "Churn rate (%)".
4. **Warna bermakna dan konsisten.** Merah untuk churn/risiko, biru/hijau untuk bertahan/aman.
5. **Referensi pembanding.** Garis rata-rata, target, atau threshold.
6. **Tidak ada elemen dekoratif yang mengganggu.** Template `plotly_white` sudah cukup bersih.

Latihan mandiri Sesi 6

1. Buat bar chart horizontal churn rate per `payment_method`, diurutkan dari tertinggi, dengan garis vertikal rata-rata nasional (`add_vline`).
2. Ubah scatter menjadi bubble chart: `size="support_calls"`.

3. Buat `px.sunburst(df, path=["region", "segment", "contract_type"], values=...)` jumlah pelanggan dan jelaskan cara membacanya.
4. Buat gauge dengan `go.Indicator(mode="gauge+number")` yang menampilkan churn rate keseluruhan.

Kesalahan umum Sesi 6

Rinciannya disajikan pada Tabel 6.2.

Tabel 6.2. *Kesalahan umum Sesi 6*

Gejala	Penyebab	Solusi
`ValueError: Value of 'x' is not the name of a column`	Nama kolom salah atau sudah menjadi index setelah groupby	Pakai `as_index=False` atau `.reset_index()`
Chart tidak muncul di Jupyter	Renderer belum sesuai	`import plotly.io as pio; pio.renderers.default = "notebook"`
Warna kategori berubah-ubah antar-chart	Plotly memberi warna berdasarkan urutan data	Gunakan `color_discrete_map`
Sumbu tanggal tidak urut	Kolom tanggal bertipe teks	`pd.to_datetime()` atau `parse_dates`

Sesi 7 — Dash Callback System (120 menit)

Sesi ini membuat dashboard Dash benar-benar interaktif: filter mengubah KPI, chart, dan tabel; opsi dropdown bergantung pada pilihan lain; tombol menerapkan dan mereset filter; dan klik pada batang chart memfilter tabel.

Tujuan sesi

- Memahami anatomi callback: `Input`, `Output`, `State`.
- Menulis callback dengan banyak input dan banyak output.
- Membuat chained callback (output satu callback menjadi input callback lain).
- Memakai `State` dan tombol agar callback tidak berjalan setiap ketikan.
- Menggunakan `ctx.triggered_id`, `PreventUpdate`, `no_update`, dan `prevent_initial_call`.
- Menghubungkan interaksi pada chart (`clickData`) dengan komponen lain.
- Menyimpan nilai antar-callback dengan `dcc.Store`.

7.1 Anatomi callback

```python
from dash import Input, Output, State, callback

@callback(
    Output("id-komponen-tujuan", "properti"),   # apa yang diperbarui
    Input("id-komponen-pemicu", "properti"),    # apa yang memicu
    State("id-komponen-lain", "properti"),      # dibaca, tetapi tidak memicu
)
def nama_fungsi(nilai_input, nilai_state):
    ...
    return nilai_baru_untuk_output
```

Aturan yang harus diingat:

1. Urutan argumen fungsi **sama dengan** urutan `Input` lalu `State` di dekorator.
2. Jumlah nilai yang di-`return` **sama dengan** jumlah `Output` (gunakan tuple untuk lebih dari satu).
3. Setiap pasangan `Output(id, properti)` hanya boleh dimiliki satu callback (kecuali memakai `allow_duplicate=True`).
4. Semua callback dijalankan sekali saat halaman dibuka, kecuali diberi `prevent_initial_call=True`.
5. Fungsi callback harus **stateless**: jangan mengubah variabel global (misalnya DataFrame global), karena satu server melayani banyak pengguna.

Rinciannya disajikan pada Tabel 7.1.

Tabel 7.1. *Anatomi callback*

Elemen	Fungsi
`Input`	Perubahan properti ini memicu callback
`State`	Nilai dibaca saat callback berjalan, tetapi perubahannya tidak memicu callback
`Output`	Properti yang diisi dengan nilai return
`ctx.triggered_id`	ID komponen yang memicu callback, berguna jika ada beberapa Input
`raise PreventUpdate`	Membatalkan seluruh update
`no_update`	Membiarkan satu Output tertentu tidak berubah

Elemen	Fungsi
`dcc.Store`	Menyimpan data di browser untuk dibagi antar-callback

7.2 Contoh paling sederhana

PYTHON
```
from dash import Dash, html, dcc, Input, Output, callback

app = Dash(__name__)
app.layout = html.Div([
    dcc.Input(id="nama", value="", placeholder="Ketik nama"),
    html.H3(id="sapaan"),
])

@callback(Output("sapaan", "children"), Input("nama", "value"))
def sapa(nama):
    return f"Halo, {nama}!" if nama else "Silakan isi nama."

if __name__ == "__main__":
    app.run(debug=True)
```

Setiap kali isi `dcc.Input` berubah, Dash mengirim nilai `value` ke fungsi `sapa`, lalu hasilnya ditulis ke properti `children` milik `html.H3`.

7.3 Praktikum — Dashboard dengan sistem callback lengkap

Buat `sesi7/app_dash_callback.py`:

PYTHON
```
# sesi7/app_dash_callback.py — sistem callback Dash: filter -> KPI, chart, tabel
from pathlib import Path
import pandas as pd
import plotly.express as px
from dash import Dash, html, dcc, dash_table, Input, Output, State, callback, ctx, no_update
from dash.exceptions import PreventUpdate

ROOT = Path(__file__).resolve().parents[1]
df = pd.read_csv(ROOT / "data" / "customer_churn.csv", parse_dates=["signup_date"])
df["status"] = df["churn"].map({0: "Bertahan", 1: "Churn"})
WARNA = {"Bertahan": "#4C72B0", "Churn": "#C44E52"}
KARTU = {"flex": 1, "padding": "14px", "borderRadius": "10px", "background": "#F4F6FA",
         "textAlign": "center"}
TABEL_COLS = ["customer_id", "region", "segment", "contract_type", "tenure_months",
              "monthly_charges", "num_complaints", "status"]

app = Dash(__name__, title="Dash Callback")
app.layout = html.Div(style={"maxWidth": "1200px", "margin": "auto", "fontFamily": "Arial"}, children=[
    html.H1("Dash Callback System — Dashboard Churn"),

    # ---- Panel filter ----
    html.Div(style={"display": "grid", "gridTemplateColumns": "1fr 1fr 1fr", "gap": "16px"}, children=[
        html.Div([html.Label("Region"),
                  dcc.Dropdown(id="dd-region", options=sorted(df.region.unique()), multi=True,
                               placeholder="Semua region")]),
        html.Div([html.Label("Segmen (opsi bergantung region)"),
                  dcc.Dropdown(id="dd-segment", multi=True, placeholder="Semua segmen")]),
        html.Div([html.Label("Jenis kontrak"),
                  dcc.RadioItems(id="rd-kontrak", inline=True, value="Semua",
                                 options=["Semua", "Bulanan", "Tahunan", "Dua Tahun"])]),
    ]),
    html.Div(style={"marginTop": "12px"}, children=[
        html.Label("Tenure (bulan)"),
        dcc.RangeSlider(id="rs-tenure", min=1, max=72, step=1, value=[1, 72],
                        marks={i: str(i) for i in (1, 12, 24, 36, 48, 60, 72)}),
    ]),
    html.Div(style={"display": "flex", "gap": "8px", "alignItems": "center"}, children=[
        html.Label("Minimal jumlah keluhan:"),
        dcc.Input(id="in-keluhan", type="number", min=0, max=10, value=0, style={"width": "70px"}),
```

```
        html.Button("Terapkan", id="btn-terapkan", n_clicks=0),
        html.Button("Reset filter", id="btn-reset", n_clicks=0),
        html.Span(id="info-keluhan", style={"color": "#666"}),
    ]),
    dcc.Store(id="store-keluhan", data=0),

    # ---- KPI ----
    html.Div(id="kpi", style={"display": "flex", "gap": "12px", "margin": "16px 0"}),

    # ---- Charts ----
    html.Div(style={"display": "grid", "gridTemplateColumns": "1fr 1fr", "gap": "12px"}, children=[
        dcc.Graph(id="g-bar"), dcc.Graph(id="g-scatter")]),
    html.P("Tip: klik salah satu batang region untuk memfilter tabel di bawah.",
           style={"color": "#666"}),
    html.H3(id="judul-tabel"),
    dash_table.DataTable(id="tbl", page_size=10, sort_action="native",
                         columns=[{"name": c, "id": c} for c in TABEL_COLS],
                         style_cell={"padding": "6px", "textAlign": "left"},
                         style_header={"fontWeight": "bold", "background": "#E9EDF5"},
                         style_data_conditional=[{"if": {"filter_query": "{status} = 'Churn'"},
                                                  "backgroundColor": "#FBE9EA"}]),
])


def filter_df(region, segment, kontrak, tenure, min_keluhan):
    d = df
    if region:
        d = d[d.region.isin(region)]
    if segment:
        d = d[d.segment.isin(segment)]
    if kontrak and kontrak != "Semua":
        d = d[d.contract_type == kontrak]
    d = d[d.tenure_months.between(tenure[0], tenure[1])]
    return d[d.num_complaints >= (min_keluhan or 0)]


# 1) Chained callback: opsi segmen bergantung pada region terpilih
@callback(Output("dd-segment", "options"), Input("dd-region", "value"))
def update_segment_options(region):
    d = df[df.region.isin(region)] if region else df
    return sorted(d.segment.unique())


# 2) State + tombol: nilai keluhan baru dipakai setelah tombol diklik; ctx untuk tombol mana
@callback(Output("store-keluhan", "data"), Output("in-keluhan", "value"),
          Output("dd-region", "value"), Output("dd-segment", "value"),
          Output("rd-kontrak", "value"), Output("rs-tenure", "value"),
          Input("btn-terapkan", "n_clicks"), Input("btn-reset", "n_clicks"),
          State("in-keluhan", "value"), prevent_initial_call=True)
def terapkan_atau_reset(n_apply, n_reset, nilai):
    if ctx.triggered_id == "btn-reset":
        return 0, 0, None, None, "Semua", [1, 72]
    if nilai is None:
        raise PreventUpdate                       # input kosong: jangan ubah apa pun
    return nilai, no_update, no_update, no_update, no_update, no_update


# 3) Callback utama: banyak input -> banyak output
@callback(Output("kpi", "children"), Output("g-bar", "figure"), Output("g-scatter", "figure"),
          Output("info-keluhan", "children"),
          Input("dd-region", "value"), Input("dd-segment", "value"), Input("rd-kontrak", "value"),
          Input("rs-tenure", "value"), Input("store-keluhan", "data"))
def update_dashboard(region, segment, kontrak, tenure, min_keluhan):
    d = filter_df(region, segment, kontrak, tenure, min_keluhan)
    info = f"Filter keluhan aktif: ≥ {min_keluhan}"
    if d.empty:
        kosong = px.scatter(title="Tidak ada data untuk filter ini")
        return [html.Div("Data kosong", style=KARTU)], kosong, kosong, info

    def kartu(j, v):
        return html.Div([html.Small(j), html.H2(v, style={"margin": "4px 0"})], style=KARTU)
    kpi = [kartu("Pelanggan", f"{len(d):,}"), kartu("Churn rate", f"{d.churn.mean():.1%}"),
           kartu("Rata-rata tagihan", f"Rp{d.monthly_charges.mean():,.0f}".replace(",", ".")),
```

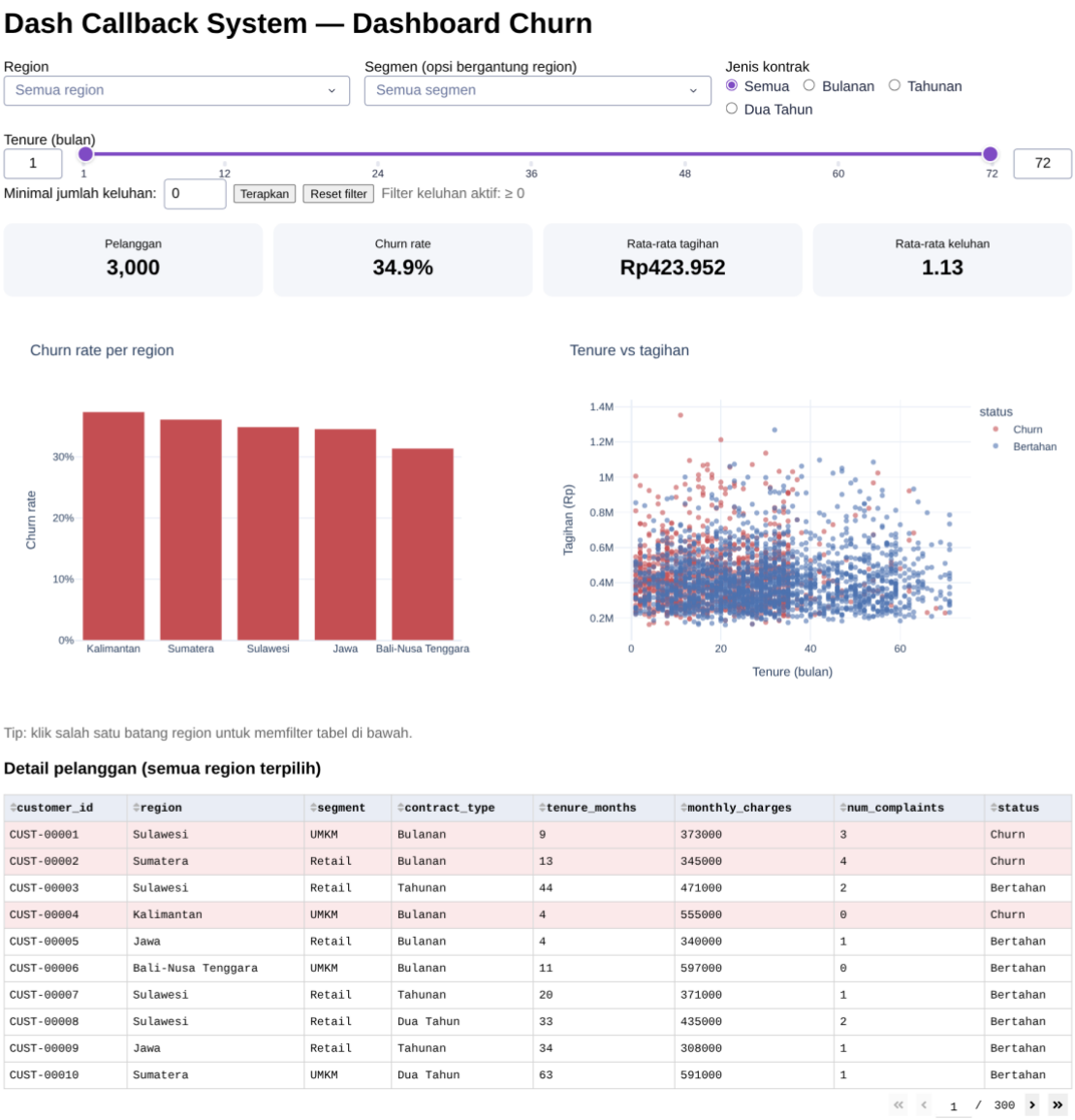
```
            kartu("Rata-rata keluhan", f"{d.num_complaints.mean():.2f}")]

    reg = d.groupby("region", as_index=False)["churn"].mean().sort_values("churn", ascending=False)
    bar = px.bar(reg, x="region", y="churn", template="plotly_white", title="Churn rate per region",
                 labels={"churn": "Churn rate", "region": ""}, color_discrete_sequence=["#C44E52"])
    bar.update_yaxes(tickformat=".0%")
    sc = px.scatter(d, x="tenure_months", y="monthly_charges", color="status", opacity=0.6,
                    color_discrete_map=WARNA, template="plotly_white", hover_name="customer_id",
                    title="Tenure vs tagihan", labels={"tenure_months": "Tenure (bulan)",
                                                       "monthly_charges": "Tagihan (Rp)"})
    return kpi, bar, sc, info


# 4) Interaksi antarkomponen: klik batang chart -> filter tabel
@callback(Output("tbl", "data"), Output("judul-tabel", "children"),
          Input("g-bar", "clickData"), Input("dd-region", "value"), Input("dd-segment", "value"),
          Input("rd-kontrak", "value"), Input("rs-tenure", "value"), Input("store-keluhan", "data"))
def update_tabel(click, region, segment, kontrak, tenure, min_keluhan):
    d = filter_df(region, segment, kontrak, tenure, min_keluhan)
    judul = "Detail pelanggan (semua region terpilih)"
    if ctx.triggered_id == "g-bar" and click:
        reg = click["points"][0]["x"]
        d = d[d.region == reg]
        judul = f"Detail pelanggan — region {reg}"
    return d[TABEL_COLS].to_dict("records"), judul


if __name__ == "__main__":
    app.run(debug=True, port=8052)
```

Jalankan `python sesi7/app_dash_callback.py` dan buka `http://127.0.0.1:8052` (lihat Gambar 7.1).



Tip: klik salah satu batang region untuk memfilter tabel di bawah.

Detail pelanggan (semua region terpilih)

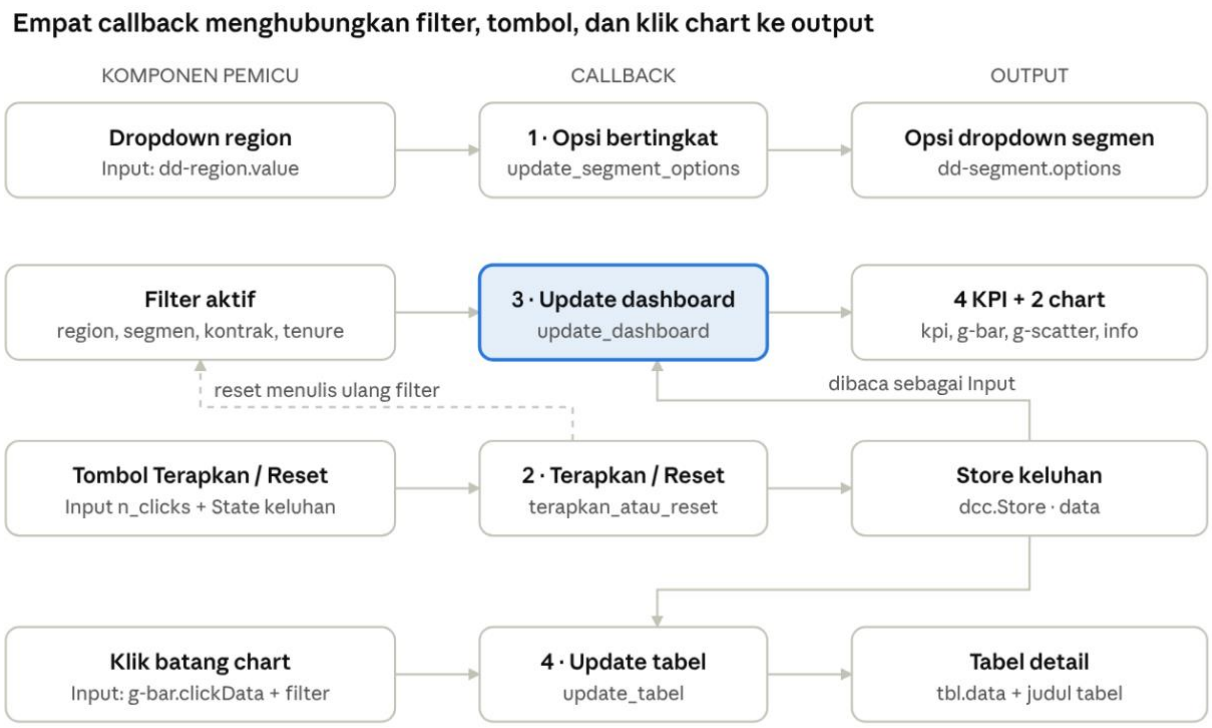
customer_id	region	segment	contract_type	tenure_months	monthly_charges	num_complaints	status
CUST-00001	Sulawesi	UMKM	Bulanan	9	373000	3	Churn
CUST-00002	Sumatera	Retail	Bulanan	13	345000	4	Churn
CUST-00003	Sulawesi	Retail	Tahunan	44	471000	2	Bertahan
CUST-00004	Kalimantan	UMKM	Bulanan	4	555000	0	Churn
CUST-00005	Jawa	Retail	Bulanan	4	340000	1	Bertahan
CUST-00006	Bali-Nusa Tenggara	UMKM	Bulanan	11	597000	0	Bertahan
CUST-00007	Sulawesi	Retail	Tahunan	20	371000	1	Bertahan
CUST-00008	Sulawesi	Retail	Dua Tahun	33	435000	2	Bertahan
CUST-00009	Jawa	Retail	Tahunan	34	308000	1	Bertahan
CUST-00010	Sumatera	UMKM	Dua Tahun	63	591000	1	Bertahan

« ‹ 1 / 300 › »

Gambar 7.1. *Dashboard Dash dengan sistem callback*

7.4 Penjelasan empat callback

Hasilnya ditunjukkan pada Gambar 7.2.



Gambar 7.2. *Graf callback sesi7/app_dash_callback.py*

Baca dari kiri ke kanan: komponen memicu callback, callback menulis output. Callback 3 yang disorot adalah callback utama dengan input dan output terbanyak. Opsi segmen dari callback 1 mengisi dropdown segmen yang menjadi bagian filter aktif; store keluhan dari callback 2 dibaca oleh callback 3 dan 4.

Callback 1 — chained callback. Saat region dipilih, `options` dropdown segmen diperbarui sesuai region tersebut. Nilai segmen kemudian menjadi Input bagi callback 3 dan 4. Rantai "region → opsi segmen → pilihan segmen → dashboard" adalah pola umum untuk filter bertingkat (misalnya provinsi → kabupaten).

Callback 2 — State, tombol, dan konteks.

- `in-keluhan` dipasang sebagai `State`, sehingga mengetik angka tidak langsung memicu perhitungan. Nilai baru dipakai hanya saat tombol **Terapkan** diklik.
- Dua tombol menjadi `Input` pada callback yang sama. `ctx.triggered_id` memberi tahu tombol mana yang diklik.
- Tombol **Reset** mengembalikan seluruh filter ke nilai awal dengan menulis ke properti `value` masing-masing komponen.
- Saat **Terapkan** diklik, hanya `store-keluhan` yang diubah; output lain diisi `no_update` agar tidak tersentuh.
- Jika input kosong, `raise PreventUpdate` membatalkan update sepenuhnya.
- `prevent_initial_call=True` mencegah callback berjalan saat halaman pertama kali dibuka.
- `dcc.Store` menyimpan nilai keluhan yang sudah "disahkan" agar dapat dibaca callback 3 dan 4.

Callback 3 — banyak Input ke banyak Output. Lima Input (region, segmen, kontrak, tenure, store keluhan) menghasilkan empat Output (KPI, dua chart, teks info). Fungsi `filter_df()` dipisah supaya logika filter yang sama dipakai di callback 3 dan 4 tanpa duplikasi.

Callback 4 — interaksi chart ke tabel. Properti `clickData` pada `dcc.Graph` berisi informasi titik yang diklik, misalnya `{"points": [{"x": "Kalimantan", "y": 0.39, ...}]}`. Callback mengambil `x` (nama region) dan memfilter tabel. Jika yang memicu adalah filter lain (bukan klik chart), tabel menampilkan semua region terpilih.

7.5 Checkpoint

Hasil berikut sudah diuji dengan browser otomatis:

1. Saat halaman dibuka, KPI menampilkan Pelanggan 3,000; Churn rate 34.9%; Rata-rata tagihan Rp423.952; Rata-rata keluhan 1.13.
2. Isi minimal keluhan **3** lalu klik **Terapkan**: Pelanggan menjadi 334 dengan churn rate 51.5%, dan teks "Filter keluhan aktif: ≥ 3" muncul.
3. Klik batang region pertama di chart: judul tabel berubah menjadi "Detail pelanggan — region …" sesuai batang yang diklik.
4. Klik **Reset filter**: semua filter dan KPI kembali ke kondisi awal (3,000 pelanggan).
5. Pilih region Jawa: opsi dropdown segmen menyesuaikan.

7.6 Debugging callback

- Panel debug Dash (ikon di pojok kanan bawah saat `debug=True`) menampilkan error callback lengkap dengan traceback, dan tab **Callbacks** menampilkan graf hubungan Input → callback → Output.
- Tambahkan `print()` di dalam callback untuk melihat nilai input di terminal.
- Error "Callback error updating ..." hampir selalu berarti jumlah atau tipe nilai return tidak sesuai dengan Output.
- Komponen yang dibuat oleh callback lain (tidak ada di layout awal) membutuhkan `Dash(__name__, suppress_callback_exceptions=True)`.

Latihan mandiri Sesi 7

1. Tambahkan `dcc.Checklist` untuk `internet_service` dan hubungkan ke `filter_df()`.
2. Tambahkan Output kelima pada callback 3: histogram tenure yang ikut terfilter.
3. Buat callback baru: saat titik pada scatter diklik, tampilkan detail pelanggan tersebut (ID, region, kontrak, keluhan) di sebuah `html.Div`.
4. Gunakan `dcc.Store` untuk menyimpan DataFrame hasil filter (`to_dict("records")`) sekali, lalu baca dari store di callback tabel.
5. Tantangan: gunakan `selectedData` (box/lasso select) pada scatter untuk memfilter tabel.

Kesalahan umum Sesi 7

Rinciannya disajikan pada Tabel 7.2.

Tabel 7.2. *Kesalahan umum Sesi 7*

Gejala	Penyebab	Solusi
`Duplicate callback outputs`	Dua callback menulis ke Output yang sama	Gabungkan dalam satu callback dengan `ctx.triggered_id`, atau `Output(..., allow_duplicate=True)` dengan `prevent_initial_call=True`

Gejala	Penyebab	Solusi
`A nonexistent object was used in an Input`	ID salah ketik atau komponen belum ada di layout	Samakan ID; gunakan `suppress_callback_exceptions=True` untuk komponen dinamis
`Expected 4 output(s), got 3`	Jumlah nilai return tidak sama dengan jumlah Output	Kembalikan tuple dengan panjang yang sama, termasuk di cabang `if`
Callback berjalan setiap ketikan	Komponen input dipasang sebagai `Input`	Ubah menjadi `State` dan tambahkan tombol sebagai `Input`
Data antar-pengguna saling memengaruhi	Callback mengubah variabel global	Jangan memodifikasi global; buat salinan atau gunakan `dcc.Store`

Sesi 8 — Final Dashboard Project (105 menit)

Sesi terakhir menggabungkan semua materi menjadi satu Customer Churn Intelligence Dashboard di Dash: data, EDA, Plotly, filter, callback, inference model final, prediction output, layout, insight, dan presentasi. Dashboard contoh di bab ini sudah memenuhi seluruh Dashboard Minimum Requirements silabus dan dapat dipakai sebagai acuan proyek peserta.

Tujuan sesi

- Merancang dashboard berawal dari pertanyaan bisnis.
- Mengintegrasikan batch inference dan simulasi prediksi interaktif dalam satu aplikasi.
- Memenuhi requirement minimum: 4 KPI, 3 visualisasi, 2 filter, 1 tampilan prediksi, 1 tabel detail.
- Merumuskan 5–10 insight dan rekomendasi tindakan.
- Mempresentasikan dashboard dalam 5–7 menit.

8.1 Alur kerja proyek akhir (105 menit)

Rinciannya disajikan pada Tabel 8.1.

Tabel 8.1. *Alur kerja proyek akhir*

Waktu	Kegiatan	Output
15 menit	Tetapkan pertanyaan bisnis, pilih KPI, sketsa layout di kertas	Sketsa layout + daftar KPI
50 menit	Bangun aplikasi: data + model, layout, callback filter, tab prediksi	Aplikasi berjalan tanpa error
15 menit	Uji semua filter dan prediksi, rapikan label dan judul	Checklist requirement terpenuhi
15 menit	Tulis 5–10 insight dan rekomendasi	Catatan insight
10 menit	Presentasi singkat	Demo dashboard

8.2 Arsitektur dashboard

Aplikasi dibagi menjadi empat blok kode yang berurutan:

1. **Data + model** — dimuat sekali saat aplikasi start. Batch inference dijalankan untuk seluruh 3.000 pelanggan sehingga setiap baris memiliki `churn_probability`, `predicted_churn`, dan `risk_category`.
2. **Style** — palet warna, gaya kartu, dan fungsi bantu (`kpi()`, `label()`, `rupiah()`).
3. **Layout** — header berisi pertanyaan bisnis, sidebar filter, dan dua tab: "Overview & Risiko" serta "Simulasi Prediksi".
4. **Callback** — `update_overview` (5 filter → 4 KPI, 6 chart, tabel) dan `run_prediction` (tombol + 9 State → hasil prediksi + gauge).

8.3 Kode lengkap final dashboard

Buat `sesi8/app_final_dashboard.py`. Modul `utils/model_utils.py` dari Sesi 4 dipakai ulang tanpa perubahan.

PYTHON

```python
# sesi8/app_final_dashboard.py — FINAL DASHBOARD PROJECT (Dash + Plotly + callback + ML inference)
import sys
from pathlib import Path
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from dash import Dash, html, dcc, dash_table, Input, Output, State, callback
from dash.exceptions import PreventUpdate

ROOT = Path(__file__).resolve().parents[1]
sys.path.append(str(ROOT))
from utils.model_utils import load_model, load_metadata, validate_input, predict

# ============ 1. DATA + MODEL (dimuat sekali saat aplikasi start) ============
META = load_metadata()
MODEL = load_model()
raw = pd.read_csv(ROOT / "data" / "customer_churn.csv", parse_dates=["signup_date"])
raw["total_charges"] = raw["total_charges"].fillna(raw["monthly_charges"] * raw["tenure_months"])
DF = predict(MODEL, raw, META)                      # batch inference seluruh pelanggan
DF["status"] = DF["churn"].map({0: "Bertahan", 1: "Churn"})
DF["periode"] = DF["signup_date"].dt.to_period("Q").astype(str)
RISK_ORDER = ["Rendah", "Sedang", "Tinggi"]
RISK_COLOR = {"Rendah": "#55A868", "Sedang": "#DD8452", "Tinggi": "#C44E52"}
CATS = META["categories"]

# ============ 2. STYLE ============
C = {"bg": "#F5F7FB", "card": "white", "ink": "#1F2A44", "muted": "#6B7280", "accent": "#2F5DA8"}
CARD = {"background": C["card"], "borderRadius": "12px", "padding": "16px",
        "boxShadow": "0 1px 3px rgba(0,0,0,.08)"}
def label(t): return html.Label(t, style={"fontWeight": 600, "fontSize": "13px"})
def kpi(judul, nilai, sub=""):
    return html.Div(style={**CARD, "flex": 1}, children=[
        html.Div(judul, style={"color": C["muted"], "fontSize": "13px"}),
        html.Div(nilai, style={"fontSize": "28px", "fontWeight": 700, "color": C["ink"]}),
        html.Div(sub, style={"color": C["muted"], "fontSize": "12px"})])
def rupiah(x): return f"Rp{x:,.0f}".replace(",", ".")

# ============ 3. LAYOUT ============
sidebar = html.Div(style={**CARD, "width": "250px", "flexShrink": 0}, children=[
    html.H4("Filter", style={"marginTop": 0}),
    label("Region"), dcc.Dropdown(id="f-region", options=CATS["region"], multi=True,
                                  placeholder="Semua"),
    html.Br(), label("Segmen"), dcc.Dropdown(id="f-segment", options=CATS["segment"], multi=True,
                                             placeholder="Semua"),
    html.Br(), label("Jenis kontrak"),
    dcc.Checklist(id="f-kontrak", options=CATS["contract_type"], value=CATS["contract_type"]),
    html.Br(), label("Kategori risiko (prediksi)"),
    dcc.Checklist(id="f-risk", options=RISK_ORDER, value=RISK_ORDER, inline=True),
    html.Br(), label("Tenure (bulan)"),
    dcc.RangeSlider(id="f-tenure", min=1, max=72, value=[1, 72], step=1,
                    marks={1: "1", 24: "24", 48: "48", 72: "72"}),
    html.Hr(), html.Small(f"Model: {META['model_name']} v{META['model_version']}",
                          style={"color": C["muted"]}),
])

tab_overview = html.Div([
    html.Div(id="kpi-row", style={"display": "flex", "gap": "12px", "marginBottom": "12px"}),
    html.Div(style={"display": "grid", "gridTemplateColumns": "1fr 1fr", "gap": "12px"}, children=[
        html.Div(dcc.Graph(id="g-donut"), style=CARD),
        html.Div(dcc.Graph(id="g-region-risk"), style=CARD),
        html.Div(dcc.Graph(id="g-trend"), style=CARD),
        html.Div(dcc.Graph(id="g-contract"), style=CARD),
        html.Div(dcc.Graph(id="g-risk"), style=CARD),
        html.Div(dcc.Graph(id="g-actual-pred"), style=CARD)]),
    html.Div(style={**CARD, "marginTop": "12px"}, children=[
        html.H4(id="tbl-title", style={"marginTop": 0}),
        dash_table.DataTable(
            id="tbl-risk", page_size=10, sort_action="native", filter_action="native",
```

```
            columns=[{"name": n, "id": i, **({"type": "numeric", "format": {"specifier": ".1%"}}
                                             if i == "churn_probability" else {})}
                     for n, i in [("ID", "customer_id"), ("Region", "region"), ("Segmen", "segment"),
                                  ("Kontrak", "contract_type"), ("Tenure", "tenure_months"),
                                  ("Keluhan", "num_complaints"), ("Prob. churn", "churn_probability"),
                                  ("Risiko", "risk_category")]],
            style_cell={"padding": "6px", "fontSize": "13px", "textAlign": "left"},
            style_header={"fontWeight": "bold", "background": "#E9EDF5"},
            style_data_conditional=[
                {"if": {"filter_query": "{risk_category} = 'Tinggi'", "column_id": "risk_category"},
                 "color": "white", "backgroundColor": RISK_COLOR["Tinggi"]},
                {"if": {"filter_query": "{risk_category} = 'Sedang'", "column_id": "risk_category"},
                 "backgroundColor": "#FBE3D3"}])]),
])

def num_input(id_, lab, val, mn, mx, step=1):
    return html.Div([label(lab), dcc.Input(id=id_, type="number", value=val, min=mn, max=mx,
                                           step=step, style={"width": "100%"})])
def dd_input(id_, lab, opts):
    return html.Div([label(lab), dcc.Dropdown(id=id_, options=opts, value=opts[0], clearable=False)])

tab_predict = html.Div(style={"display": "grid", "gridTemplateColumns": "1fr 1fr", "gap": "12px"},
                       children=[
    html.Div(style=CARD, children=[
        html.H4("Simulasi prediksi pelanggan", style={"marginTop": 0}),
        html.Div(style={"display": "grid", "gridTemplateColumns": "1fr 1fr", "gap": "10px"}, children=[
            dd_input("p-region", "Region", CATS["region"]),
            dd_input("p-segment", "Segmen", CATS["segment"]),
            dd_input("p-contract", "Kontrak", CATS["contract_type"]),
            dd_input("p-payment", "Pembayaran", CATS["payment_method"]),
            dd_input("p-service", "Layanan", CATS["internet_service"]),
            num_input("p-tenure", "Tenure (bulan)", 6, 1, 72),
            num_input("p-monthly", "Tagihan bulanan (Rp)", 450000, 100000, 2000000, 10000),
            num_input("p-complaints", "Jumlah keluhan", 2, 0, 10),
            num_input("p-calls", "Panggilan CS", 3, 0, 15)]),
        html.Br(),
        html.Button("Jalankan prediksi", id="btn-predict", n_clicks=0,
                    style={"background": C["accent"], "color": "white", "border": 0,
                           "padding": "10px 18px", "borderRadius": "8px", "cursor": "pointer"})]),
    html.Div(style=CARD, children=[html.H4("Hasil prediksi", style={"marginTop": 0}),
                                   html.Div(id="pred-output",
                                            children=html.P("Isi form lalu klik tombol.")),
                                   dcc.Graph(id="g-gauge", style={"height": "260px"})]),
])

app = Dash(__name__, title="Final Dashboard — Customer Churn")
server = app.server                                    # untuk deployment (gunicorn)
app.layout = html.Div(style={"background": C["bg"], "minHeight": "100vh", "padding": "16px",
                             "fontFamily": "Inter, Arial, sans-serif", "color": C["ink"]}, children=[
    html.Div(style={"marginBottom": "12px"}, children=[
        html.H2("Customer Churn Intelligence Dashboard", style={"margin": 0}),
        html.Div("Pertanyaan bisnis: pelanggan mana yang berisiko churn dan apa yang harus dilakukan? "
                 "· Data latihan 2023–2025 · Prediksi dari final trained model",
                 style={"color": C["muted"]})]),
    html.Div(style={"display": "flex", "gap": "12px", "alignItems": "flex-start"}, children=[
        sidebar,
        html.Div(style={"flex": 1, "minWidth": 0}, children=[
            dcc.Tabs(id="tabs", value="overview", children=[
                dcc.Tab(label="Overview & Risiko", value="overview", children=[tab_overview]),
                dcc.Tab(label="Simulasi Prediksi", value="predict", children=[tab_predict])])])]),
])

# ============ 4. CALLBACKS ============
def apply_filters(region, segment, kontrak, risk, tenure):
    d = DF
    if region: d = d[d.region.isin(region)]
    if segment: d = d[d.segment.isin(segment)]
    d = d[d.contract_type.isin(kontrak or []) & d.risk_category.isin(risk or [])]
    return d[d.tenure_months.between(*tenure)]

FILTERS = [Input("f-region", "value"), Input("f-segment", "value"), Input("f-kontrak", "value"),
           Input("f-risk", "value"), Input("f-tenure", "value")]

@callback(Output("kpi-row", "children"), Output("g-donut", "figure"), Output("g-region-risk", "figure"),
          Output("g-trend", "figure"), Output("g-contract", "figure"),
```

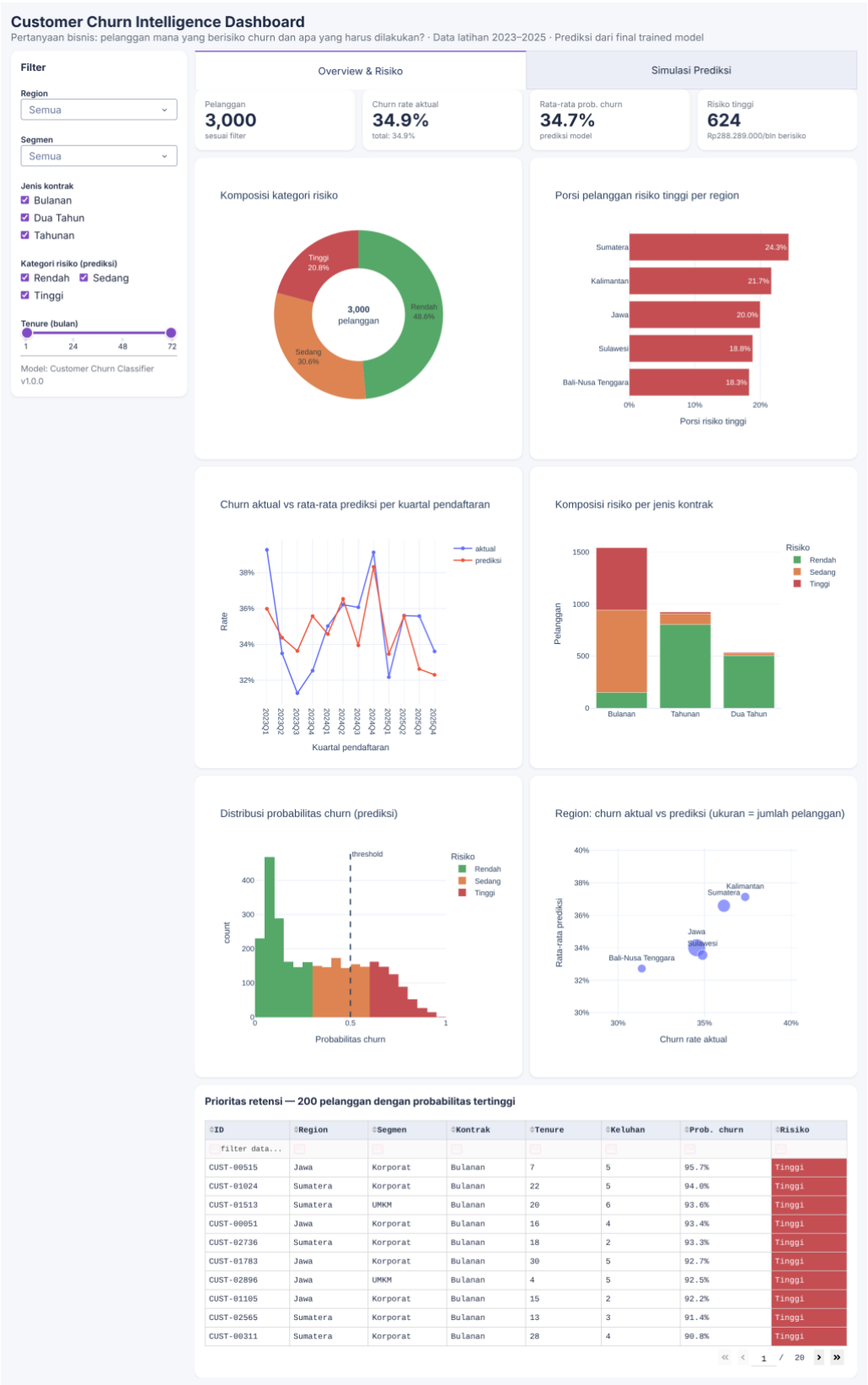
```
          Output("g-risk", "figure"), Output("g-actual-pred", "figure"),
          Output("tbl-risk", "data"), Output("tbl-title", "children"), *FILTERS)
def update_overview(region, segment, kontrak, risk, tenure):
    d = apply_filters(region, segment, kontrak, risk, tenure)
    if d.empty:
        f = px.scatter(title="Tidak ada data — longgarkan filter", template="plotly_white")
        return [kpi("Pelanggan", "0")], f, f, f, f, f, f, [], "Tidak ada data"
    tinggi = (d.risk_category == "Tinggi").sum()
    kpis = [kpi("Pelanggan", f"{len(d):,}", "sesuai filter"),
            kpi("Churn rate aktual", f"{d.churn.mean():.1%}", f"total: {DF.churn.mean():.1%}"),
            kpi("Rata-rata prob. churn", f"{d.churn_probability.mean():.1%}", "prediksi model"),
            kpi("Risiko tinggi", f"{tinggi:,}", rupiah(d.loc[d.risk_category == 'Tinggi',
                                                             'monthly_charges'].sum()) + "/bln berisiko")]
    g0 = px.pie(d, names="risk_category", hole=0.55, color="risk_category",
                color_discrete_map=RISK_COLOR, category_orders={"risk_category": RISK_ORDER},
                title="Komposisi kategori risiko")
    g0.update_traces(textinfo="percent+label", sort=False)
    g0.add_annotation(text=f"<b>{len(d):,}</b><br>pelanggan", showarrow=False, font_size=14)
    g0.update_layout(showlegend=False)
    rr = (d.assign(tinggi=d.risk_category.eq("Tinggi")).groupby("region", as_index=False)["tinggi"]
            .mean().sort_values("tinggi"))
    g5 = px.bar(rr, x="tinggi", y="region", orientation="h", text_auto=".1%",
                template="plotly_white", color_discrete_sequence=[RISK_COLOR["Tinggi"]],
                title="Porsi pelanggan risiko tinggi per region",
                labels={"tinggi": "Porsi risiko tinggi", "region": ""})
    g5.update_xaxes(tickformat=".0%")
    tr = d.groupby("periode").agg(aktual=("churn", "mean"), prediksi=("churn_probability",
"mean")).reset_index()
    g1 = px.line(tr.melt("periode", var_name="seri", value_name="rate"), x="periode", y="rate",
                 color="seri", markers=True, template="plotly_white",
                 title="Churn aktual vs rata-rata prediksi per kuartal pendaftaran",
                 labels={"periode": "Kuartal pendaftaran", "rate": "Rate", "seri": ""})
    g1.update_yaxes(tickformat=".0%")
    ck = d.groupby(["contract_type", "risk_category"]).size().reset_index(name="n")
    g2 = px.bar(ck, x="contract_type", y="n", color="risk_category", color_discrete_map=RISK_COLOR,
                category_orders={"risk_category": RISK_ORDER,
                                 "contract_type": ["Bulanan", "Tahunan", "Dua Tahun"]},
                template="plotly_white", title="Komposisi risiko per jenis kontrak",
                labels={"contract_type": "", "n": "Pelanggan", "risk_category": "Risiko"})
    g3 = px.histogram(d, x="churn_probability", nbins=30, color="risk_category",
                      color_discrete_map=RISK_COLOR, category_orders={"risk_category": RISK_ORDER},
                      template="plotly_white", title="Distribusi probabilitas churn (prediksi)",
                      labels={"churn_probability": "Probabilitas churn", "risk_category": "Risiko"})
    g3.add_vline(x=META["decision_threshold"], line_dash="dash", annotation_text="threshold")
    rg = d.groupby("region").agg(aktual=("churn", "mean"), prediksi=("churn_probability", "mean"),
                                 n=("customer_id", "count")).reset_index()
    g4 = px.scatter(rg, x="aktual", y="prediksi", size="n", text="region", template="plotly_white",
                    title="Region: churn aktual vs prediksi (ukuran = jumlah pelanggan)",
                    labels={"aktual": "Churn rate aktual", "prediksi": "Rata-rata prediksi"})
    g4.update_traces(textposition="top center", cliponaxis=False)
    pad = 0.03
    g4.update_xaxes(tickformat=".0%", range=[rg.aktual.min() - pad, rg.aktual.max() + pad])
    g4.update_yaxes(tickformat=".0%", range=[rg.prediksi.min() - pad, rg.prediksi.max() + pad])
    top = d.sort_values("churn_probability", ascending=False).head(200)
    return (kpis, g0, g5, g1, g2, g3, g4, top.to_dict("records"),
            f"Prioritas retensi — {min(len(d), 200)} pelanggan dengan probabilitas tertinggi")

@callback(Output("pred-output", "children"), Output("g-gauge", "figure"),
          Input("btn-predict", "n_clicks"),
          State("p-region", "value"), State("p-segment", "value"), State("p-contract", "value"),
          State("p-payment", "value"), State("p-service", "value"), State("p-tenure", "value"),
          State("p-monthly", "value"), State("p-complaints", "value"), State("p-calls", "value"),
          prevent_initial_call=True)
def run_prediction(n, region, segment, contract, payment, service, tenure, monthly, complaints, calls):
    if None in (tenure, monthly, complaints, calls):
        raise PreventUpdate
    row = pd.DataFrame([{"tenure_months": tenure, "monthly_charges": monthly,
                         "total_charges": monthly * tenure, "num_complaints": complaints,
                         "support_calls": calls, "region": region, "segment": segment,
                         "contract_type": contract, "payment_method": payment,
                         "internet_service": service}])
    errs = validate_input(row, META)
    if errs:
        return html.Div([html.P(e, style={"color": "crimson"}) for e in errs]), px.scatter()
    r = predict(MODEL, row, META).iloc[0]
    saran = {"Tinggi": "Prioritaskan retensi: hubungi dalam 48 jam, tawarkan kontrak tahunan.",
             "Sedang": "Pantau: kirim survei kepuasan dan penawaran loyalitas.",
```

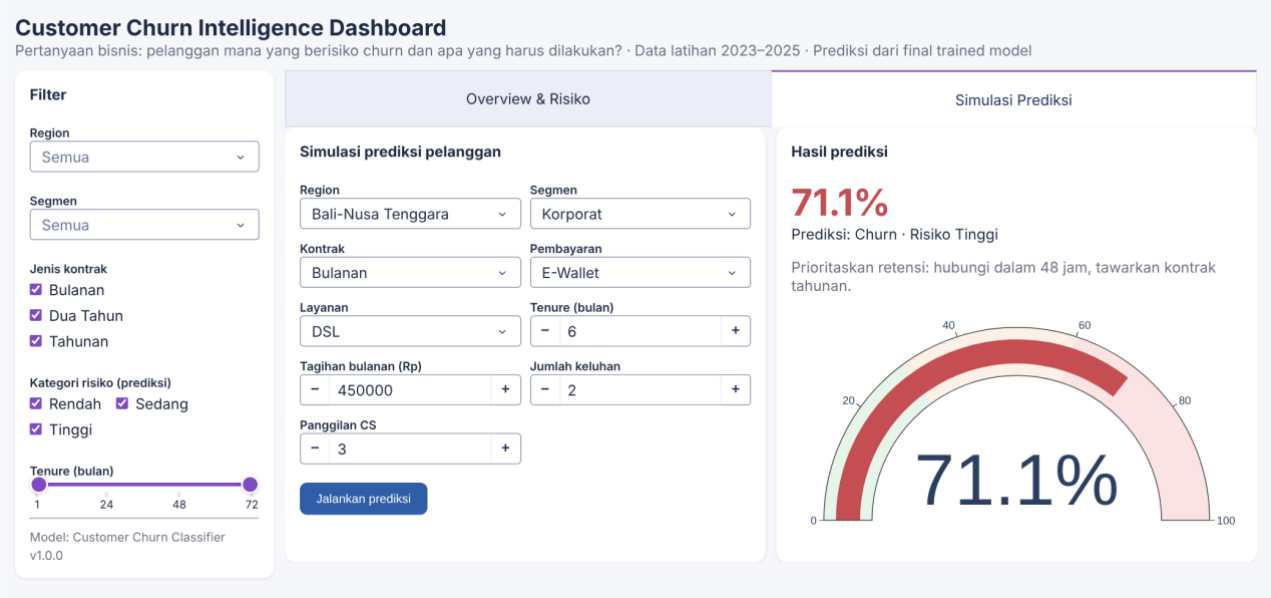
```
            "Rendah": "Pertahankan layanan standar."}[r.risk_category]
    out = html.Div([
        html.Div(f"{r.churn_probability:.1%}", style={"fontSize": "40px", "fontWeight": 700,
                                                      "color": RISK_COLOR[r.risk_category]}),
        html.Div(f"Prediksi: {'Churn' if r.predicted_churn else 'Bertahan'} · Risiko {r.risk_category}"),
        html.P(saran, style={"color": C["muted"]})])
    gauge = go.Figure(go.Indicator(mode="gauge+number", value=r.churn_probability * 100,
                                   number={"suffix": "%"}, gauge={
                                       "axis": {"range": [0, 100]},
                                       "bar": {"color": RISK_COLOR[r.risk_category]},
                                       "steps": [{"range": [0, 30], "color": "#E6F4EA"},
                                                 {"range": [30, 60], "color": "#FDF0E6"},
                                                 {"range": [60, 100], "color": "#FBE3E3"}]}))
    gauge.update_layout(margin=dict(t=20, b=10, l=20, r=20), height=240)
    return out, gauge

if __name__ == "__main__":
    app.run(debug=True, port=8053)
```

Jalankan `python sesi8/app_final_dashboard.py` dan buka `http://127.0.0.1:8053` (lihat Gambar 8.1, dan 8.2).



Gambar 8.1. *Final dashboard — tab Overview & Risiko*



Gambar 8.2. *Final dashboard — tab Simulasi Prediksi*

8.4 Penjelasan keputusan desain

- **Batch inference saat start.** Karena data tidak berubah selama aplikasi berjalan, prediksi untuk 3.000 pelanggan dihitung sekali. Callback filter hanya memotong DataFrame yang sudah berisi prediksi, sehingga respons tetap cepat. Untuk data yang diperbarui berkala, jalankan inference terjadwal dan simpan hasilnya ke file/database.
- **Donut komposisi risiko di posisi pertama** menjawab pertanyaan paling awal manajemen: berapa porsi pelanggan berisiko? Bar horizontal di sebelahnya langsung menunjukkan region mana yang perlu diprioritaskan.
- **Filter kategori risiko memakai output model.** Ini contoh prediction output yang menjadi komponen interaktif, bukan sekadar angka yang ditampilkan.
- **KPI "Risiko tinggi" disertai nilai tagihan bulanan berisiko** agar manajemen langsung melihat dampak finansial.
- **Chart aktual vs prediksi** (per kuartal dan per region) membantu pengguna menilai apakah prediksi model sejalan dengan kenyataan historis.
- **Garis threshold pada histogram** menjelaskan mengapa pelanggan dengan probabilitas 0,55 sudah diprediksi churn.
- **Tabel prioritas retensi** diurutkan dari probabilitas tertinggi dengan filter bawaan DataTable, siap dipakai tim retensi sebagai daftar kerja.
- **Tab simulasi memakai `State` + tombol** sehingga model hanya dipanggil saat pengguna siap, dan gauge berwarna sesuai kategori risiko.
- `server = app.server` disiapkan untuk deployment dengan gunicorn: `gunicorn sesi8.app_final_dashboard:server`.

8.5 Pemenuhan Dashboard Minimum Requirements

Rinciannya disajikan pada Tabel 8.2.

Tabel 8.2. *Pemenuhan Dashboard Minimum Requirements*

Requirement silabus	Implementasi di dashboard contoh	Status
Minimal 4 KPI cards	Pelanggan, Churn rate aktual, Rata-rata prob. churn, Risiko tinggi (+ tagihan berisiko)	Terpenuhi
Minimal 3 visualisasi utama, termasuk interaktif	6 chart Plotly: donut komposisi risiko, bar risiko tinggi per region, tren aktual vs prediksi, komposisi risiko per kontrak, distribusi probabilitas, region aktual vs prediksi; plus gauge	Terpenuhi
Minimal 2 filter/input	5 filter: region, segmen, kontrak, kategori risiko, tenure; 9 input simulasi	Terpenuhi
Minimal 1 tampilan prediction output	Filter risiko, histogram probabilitas, tabel prioritas, gauge simulasi	Terpenuhi
Minimal 1 tabel detail	DataTable prioritas retensi dengan sort, filter, paging	Terpenuhi
Layout jelas dan mendukung alur interpretasi	Header → KPI → chart → tabel; filter di sidebar; tab terpisah untuk simulasi	Terpenuhi
Judul, unit, periode, label jelas	Judul chart deskriptif, format %, Rupiah, periode 2023–2025 di header	Terpenuhi
Menjawab pertanyaan bisnis	Pertanyaan bisnis tertulis di header; dijawab oleh KPI risiko dan tabel prioritas	Terpenuhi

8.6 Checkpoint

1. Tanpa filter, KPI menampilkan 3,000 pelanggan, churn rate aktual 34.9%, rata-rata probabilitas 34.7%, dan 624 pelanggan risiko tinggi dengan tagihan Rp288.289.000/bln.
2. Hapus centang Rendah dan Sedang pada filter risiko: semua chart dan tabel hanya menampilkan pelanggan risiko tinggi.
3. Kosongkan semua jenis kontrak: muncul "Tidak ada data — longgarkan filter" tanpa error.
4. Buka tab Simulasi Prediksi dan klik **Jalankan prediksi** dengan nilai default: hasil 71.1%, Prediksi Churn, Risiko Tinggi, dan gauge merah.

8.7 Contoh insight dan rekomendasi

Angka berikut dihitung dari dashboard contoh. Gunakan format yang sama untuk proyek Anda.

1. **624 pelanggan (20,8%) berada pada kategori risiko tinggi**, dengan total tagihan Rp288,3 juta per bulan.
2. **96% pelanggan risiko tinggi berkontrak Bulanan.** Rekomendasi: program migrasi ke kontrak Tahunan dengan insentif diskon untuk pelanggan Bulanan berisiko tinggi.
3. **Rata-rata tenure pelanggan risiko tinggi 14,0 bulan**, hampir separuh rata-rata seluruh pelanggan (26,5 bulan). Rekomendasi: perkuat onboarding dan kontak proaktif pada 12 bulan pertama.
4. **Rata-rata keluhan pelanggan risiko tinggi 1,78**, dibanding 1,13 untuk seluruh pelanggan. Rekomendasi: SLA penyelesaian keluhan maksimal 48 jam untuk pelanggan berisiko tinggi.
5. **Proporsi risiko tinggi pada metode Transfer Bank 27,4%**, tertinggi dibanding E-Wallet 19,6%, Kartu Kredit 17,0%, dan Virtual Account 16,6%. Rekomendasi: dorong autodebit atau e-wallet untuk mengurangi friksi pembayaran.
6. **Sumatera memiliki proporsi risiko tinggi terbesar (24,3%)**, sedangkan Bali-Nusa Tenggara terendah (18,3%). Rekomendasi: prioritaskan kampanye retensi regional di Sumatera.
7. **Kategori risiko konsisten dengan data historis:** churn aktual 78,5% pada kelompok Tinggi, 45,4% pada Sedang, dan 9,7% pada Rendah. Catatan: sebagian data ini juga dipakai saat model dilatih, sehingga angka

tersebut cenderung lebih baik daripada kinerja pada pelanggan baru; acuan kinerja yang adil adalah metrik holdout di `model_metadata.json`.

8. **134 pelanggan yang masih aktif (belum churn) masuk kategori risiko tinggi.** Rekomendasi: jadikan 134 pelanggan ini daftar kerja pertama tim retensi minggu ini.

8.8 Struktur presentasi 5–7 menit

1. **Masalah bisnis (30 detik)** — pertanyaan yang dijawab dashboard.
2. **Data dan model (45 detik)** — sumber data, jumlah baris, model final yang dipakai, arti probabilitas dan kategori risiko.
3. **Tur dashboard (2 menit)** — KPI, chart utama, filter, tabel.
4. **Demo interaksi (1 menit)** — terapkan satu filter dan satu simulasi prediksi secara langsung.
5. **Insight (1,5 menit)** — 3 sampai 5 temuan terpenting dengan angka.
6. **Rekomendasi dan keterbatasan (1 menit)** — tindakan yang diusulkan dan batasan data/model.

8.9 Template final project

Isi template berikut sebagai dokumen pendamping dashboard.

Rinciannya disajikan pada Tabel 8.3.

Tabel 8.3. *Template final project*

Bagian	Isi minimum	Isian peserta
1. Business Problem	Definisi masalah dan pertanyaan bisnis	
2. Data Understanding	Sumber data, struktur, target/label, kualitas data	
3. EDA	Eksplorasi data, statistik, pola, temuan awal	
4. Visualization	Visualisasi utama dan alasan pemilihan chart	
5. Final ML Model	Model final yang dipakai; tanpa training ulang	
6. Model Loading & Inference	Load model, siapkan input, jalankan prediksi	
7. Prediction Output	Predicted value/class, probability/score, kategori	
8. Interactive Components	Filter/input pengguna dan komponen interaktif	
9. Callback / App Logic	Hubungan input dengan output dashboard	
10. Dashboard	KPI, visualisasi, filter, tampilan prediksi, layout	
11. Insight & Recommendation	Temuan utama dan tindakan yang disarankan	

8.10 Checklist final submission

- ☐ Source code Streamlit/Dash
- ☐ Dataset / analytical data
- ☐ EDA workflow
- ☐ Interactive Plotly visualizations
- ☐ Final trained / selected best-performing ML model
- ☐ Model loading & inference workflow

- ☐ Prediction result/output
- ☐ Interactive dashboard
- ☐ Filter/input components
- ☐ Callback implementation (untuk Dash project)
- ☐ Business insight
- ☐ Recommendation
- ☐ Presentasi final project

8.11 Rubrik penilaian

Kriteria kelulusan minimal: skor rata-rata 70. Pre-test/post-test tidak termasuk bobot kelulusan.

Rinciannya disajikan pada Tabel 8.4.

Tabel 8.4. *Rubrik penilaian*

Komponen	Bobot	Skor 90–100	Skor 70–89	Skor < 70
Hands-on Practice	40%	Semua praktikum Sesi 1–7 berjalan, latihan mandiri dikerjakan	Sebagian besar praktikum berjalan	Banyak praktikum tidak selesai
Final Dashboard Project	40%	Semua requirement minimum terpenuhi, callback dan prediksi berjalan tanpa error, layout rapi	Requirement terpenuhi dengan kekurangan kecil	Requirement minimum tidak terpenuhi atau prediksi tidak terintegrasi
Presentation & Business Insight	20%	5–10 insight berbasis angka, rekomendasi jelas, interpretasi prediksi tepat	Insight ada tetapi kurang spesifik	Hanya mendeskripsikan chart tanpa insight

8.12 Variasi studi kasus

Pola yang sama dapat dipakai untuk studi kasus lain di silabus. Yang berubah hanya data, model, dan bentuk prediction output.

Rinciannya disajikan pada Tabel 8.5.

Tabel 8.5. *Variasi studi kasus*

Kasus	Pertanyaan bisnis	Prediction output di dashboard
Sales Dashboard	Bagaimana kondisi penjualan dan berapa estimasi nilai penjualan?	KPI, tren, actual vs predicted
Customer Churn Dashboard	Siapa pelanggan yang berisiko churn?	Churn probability, risk category
Fraud / Risk Dashboard	Transaksi atau entitas mana yang berisiko tinggi?	Risk score/category, detail transaksi
Demand Forecasting Dashboard	Bagaimana estimasi kebutuhan periode mendatang?	Forecast dan tren
Customer / Sentiment Dashboard	Bagaimana hasil klasifikasi pelanggan/review?	Predicted class dan probability/score

Untuk model regresi (Sales, Demand), ganti `predict_proba` dengan `predict` dan tampilkan nilai prediksi berdampingan dengan nilai aktual, misalnya line chart actual vs predicted.

Lampiran

A. Kunci jawaban latihan mandiri

Kunci jawaban lengkap tersedia di folder `kunci_jawaban/` pada paket praktikum dan sudah diuji. Bagian ini merangkum inti setiap jawaban.

Sesi 1

No. 1, 2, dan 4 — tambahkan "Satelit" ke dictionary, slider promo, dan penghitung klik dengan session state (`kunci_jawaban/sesi1_kalkulator_plus.py`):

```python
HARGA = {"DSL": 260_000, "Fiber": 420_000, "Mobile Broadband": 310_000, "Satelit": 550_000}
promo = st.slider("Diskon promosi tambahan (%)", 0, 20, 0)

if "jumlah_klik" not in st.session_state:
    st.session_state.jumlah_klik = 0

if st.button("Hitung", type="primary"):
    st.session_state.jumlah_klik += 1
    per_bulan = HARGA[layanan] * (1 - DISKON[kontrak]) * (1 - promo / 100)
    ...
st.caption(f"Tombol Hitung sudah diklik {st.session_state.jumlah_klik} kali.")
```

No. 3 — profil peserta (`kunci_jawaban/sesi1_profil.py`):

```python
nama = st.text_input("Nama")
divisi = st.selectbox("Divisi", ["Finance", "Operations", "Marketing", "IT", "HR"])
pengalaman = st.slider("Pengalaman (tahun)", 0, 30, 2)
if nama:
    st.info(f"{nama} dari divisi {divisi} dengan pengalaman {pengalaman} tahun.")
```

Sesi 2 (`kunci_jawaban/sesi2_latihan.py`)

```python
# 1. Pie layanan internet, donut region, lalu bar horizontal terurut sebagai pembanding
s = df["internet_service"].value_counts()
ax.pie(s, labels=s.index, autopct="%1.1f%%", colors=WARNA[:3], startangle=90)
r = df["region"].value_counts()
ax.pie(r, labels=r.index, colors=WARNA, wedgeprops=dict(width=0.4, edgecolor="white"))
ax.barh(r.sort_values().index, r.sort_values().values, color=WARNA[0])

# 2. Stacked bar 100% status per layanan internet
ct = pd.crosstab(df["internet_service"], df["status"], normalize="index") * 100
ct.plot.barh(stacked=True, color=["#2a78d6", "#eb6834"])

# 3. Box plot support_calls per status
ax.boxplot([df.loc[df.status == s, "support_calls"] for s in ["Bertahan", "Churn"]],
           tick_labels=["Bertahan", "Churn"])

# 4. Churn rate per kuartal pendaftaran
q = df.groupby(df.signup_date.dt.to_period("Q"))["churn"].mean() * 100
ax.plot(q.index.astype(str), q.values, marker="o")
```

Untuk 5 region, bar horizontal terurut lebih mudah dibaca daripada donut karena perbedaan panjang batang langsung terlihat. Hasil no. 2: porsi churn DSL 34,9%, Fiber 34,7%, Mobile Broadband 35,5%; layanan internet bukan pembeda churn. No. 5 cukup menambah satu nama tab dan blok `with tab[6]:` berisi `px.bar(df.groupby("num_complaints", as_index=False)["churn"].mean(), x="num_complaints", y="churn")`. No. 6: `st.download_button("Unduh", rk.to_csv(index=False).encode("utf-8"), "ringkasan.csv", "text/csv")`.

Sesi 3

```python
# 1. Filter layanan internet
service = st.sidebar.multiselect("Layanan", sorted(df.internet_service.unique()),
                                 default=sorted(df.internet_service.unique()))
mask &= df["internet_service"].isin(service)

# 2. KPI ke-5
k1, k2, k3, k4, k5 = st.columns(5)
k5.metric("Keluhan ≥ 3", f"{(dff.num_complaints >= 3).mean():.1%}")

# 3. Churn rate per kuartal
q = (dff.assign(kuartal=dff.signup_date.dt.to_period("Q").astype(str))
       .groupby("kuartal", as_index=False)["churn"].mean())
st.plotly_chart(px.line(q, x="kuartal", y="churn", markers=True), width="stretch")

# 4. Kartu dengan border
with st.container(border=True):
    st.plotly_chart(fig_tren, width="stretch")
```

No. 5 memakai pola session state pada subbab 3.4.

Sesi 4 (`kunci_jawaban/sesi4_threshold.py`)

```python
meta = dict(load_metadata())
threshold = st.sidebar.slider("Threshold keputusan", 0.3, 0.7, 0.5, 0.05)
meta["decision_threshold"] = threshold          # salinan meta, file asli tidak berubah
hasil = predict(get_model(), data, meta)

st.dataframe(pd.crosstab(hasil.region, hasil.risk_category))                  # no. 3
berisiko = hasil.loc[hasil.risk_category == "Tinggi", "monthly_charges"].sum()  # no. 4
```

Dengan threshold 0,5 dan `new_customers.csv`, hasilnya 65 prediksi churn dan pendapatan bulanan berisiko Rp25.137.000. Untuk no. 2, `validate_input()` akan menampilkan pesan `Nilai tidak dikenal pada 'contract_type': ['Mingguan']`.

Sesi 5

```python
# 1. KPI kelima
kartu("Rata-rata keluhan", f"{df.num_complaints.mean():.2f}")

# 2. Dua grafik berdampingan
html.Div(style={"display": "grid", "gridTemplateColumns": "1fr 1fr", "gap": "12px"},
         children=[dcc.Graph(figure=fig_kontrak), dcc.Graph(figure=fig_region)])

# 3. DataTable dengan sorting dan paging
dash_table.DataTable(..., sort_action="native", page_size=5)

# 4. assets/style.css berisi .kartu { flex: 1; padding: 16px; border-radius: 10px; }
html.Div([...], className="kartu")
```

Sesi 6 (`kunci_jawaban/sesi6_latihan.py`)

```python
# 1. Bar horizontal + garis rata-rata
f1 = px.bar(pm, x="churn", y="payment_method", orientation="h")
f1.add_vline(x=df.churn.mean(), line_dash="dot", annotation_text="rata-rata")

# 2. Bubble chart
px.scatter(df, x="tenure_months", y="monthly_charges", color="status", size="support_calls")

# 3. Sunburst: lingkar dalam = region, tengah = segmen, luar = kontrak
cnt = df.groupby(["region", "segment", "contract_type"]).size().reset_index(name="pelanggan")
px.sunburst(cnt, path=["region", "segment", "contract_type"], values="pelanggan")
```

```
# 4. Gauge
go.Figure(go.Indicator(mode="gauge+number", value=df.churn.mean() * 100,
                       number={"suffix": "%"}, gauge={"axis": {"range": [0, 100]}}))
```

Cara membaca sunburst: luas setiap irisan sebanding dengan jumlah pelanggan; klik irisan untuk memperbesar cabang tersebut.

Sesi 7

No. 3 — klik titik scatter untuk melihat detail (`kunci_jawaban/sesi7_klik_scatter.py`). Kuncinya adalah `custom_data` pada figure dan `clickData` pada callback:

```
PYTHON
dcc.Graph(id="sc", figure=px.scatter(df, x="tenure_months", y="monthly_charges",
                                     custom_data=["customer_id"]))

@callback(Output("detail", "children"), Input("sc", "clickData"))
def tampil_detail(click):
    if not click:
        raise PreventUpdate
    cid = click["points"][0]["customdata"][0]
    r = df.loc[df.customer_id == cid].iloc[0]
    return html.Ul([html.Li(f"ID: {r.customer_id}"), html.Li(f"Region: {r.region}"),
                    html.Li(f"Kontrak: {r.contract_type}"), html.Li(f"Keluhan: {r.num_complaints}")])
```

No. 1, 2, 4 — tambahkan parameter baru ke `filter_df()`, tambahkan `Output("g-hist", "figure")` beserta nilai return-nya, dan simpan `d.to_dict("records")` ke `dcc.Store` lalu baca kembali dengan `pd.DataFrame(data)`. No. 5 memakai `Input("g-scatter", "selectedData")` dan mengambil `[p["customdata"][0] for p in selected["points"]]` sebagai daftar ID pelanggan (scatter dibuat dengan custom_data=["customer_id"]).

B. Cheat sheet

Streamlit

Rinciannya disajikan pada Tabel L.1.

Tabel L.1. *Cheat sheet (bagian 1)*

Kebutuhan	Kode
Menjalankan aplikasi	`streamlit run app.py`
Layout lebar	`st.set_page_config(layout="wide")`
Sidebar	`st.sidebar.selectbox(...)`
Kolom	`a, b = st.columns([2, 1])`
Tab	`t1, t2 = st.tabs(["A", "B"])`
KPI	`st.metric("Label", nilai, delta)`
Cache data	`@st.cache_data`
Cache model	`@st.cache_resource`
Form	`with st.form("f"): ... st.form_submit_button()`
Simpan nilai antar-rerun	`st.session_state["x"] = ...`
Hentikan script	`st.stop()`
Chart Plotly	`st.plotly_chart(fig)`

Dash

Rinciannya disajikan pada Tabel L.2.

Tabel L.2. *Cheat sheet (bagian 2)*

Kebutuhan	Kode
Menjalankan aplikasi	`python app.py` (berisi `app.run(debug=True)`)
Layout	`app.layout = html.Div([...])`
Grafik	`dcc.Graph(id="g", figure=fig)`
Callback	`@callback(Output(id, prop), Input(id, prop), State(id, prop))`
Pemicu callback	`ctx.triggered_id`
Batalkan update	`raise PreventUpdate`
Lewati satu output	`return no_update`
Jangan jalan saat awal	`prevent_initial_call=True`
Simpan data di browser	`dcc.Store(id="s", data=...)`
Interaksi chart	`clickData`, `hoverData`, `selectedData`
Deployment	`server = app.server` lalu `gunicorn app:server`

Plotly

Rinciannya disajikan pada Tabel L.3.

Tabel L.3. *Cheat sheet (bagian 3)*

Chart	Plotly Express
Bar	`px.bar(df, x, y, color, barmode="group")`
Line	`px.line(df, x, y, color, markers=True)`
Scatter	`px.scatter(df, x, y, color, size, hover_name, hover_data)`
Histogram	`px.histogram(df, x, color, nbins)`
Box	`px.box(df, x, y, color)`
Heatmap	`px.imshow(pivot, text_auto=True)`
Sunburst	`px.sunburst(df, path=[...], values=...)`
Gauge	`go.Figure(go.Indicator(mode="gauge+number", value=...))`
Garis referensi	`fig.add_hline(y=...)`, `fig.add_vline(x=...)`
Format persen	`fig.update_yaxes(tickformat=".0%")`

C. Troubleshooting umum

Rinciannya disajikan pada Tabel L.4.

Tabel L.4. *Troubleshooting umum*

Masalah	Langkah penyelesaian
`ModuleNotFoundError` untuk streamlit/dash/plotly	Aktifkan virtual environment, lalu `pip install -r requirements.txt`
`pip` lambat atau gagal karena jaringan kantor	Gunakan `pip install --proxy http://alamat-proxy:port -r requirements.txt` atau minta akses ke fasilitator
Aplikasi tidak terbuka di browser	Salin alamat `localhost` dari terminal; pastikan port tidak dipakai aplikasi lain
Perubahan kode tidak terlihat	Streamlit: klik **Rerun** atau aktifkan **Always rerun**. Dash: pastikan `debug=True`, lalu refresh browser
Model gagal dimuat	Periksa versi scikit-learn sama dengan `sklearn_version` di `model_metadata.json`

Masalah	Langkah penyelesaian
Angka berformat titik/koma tidak sesuai	Gunakan f-string `f"{x:,.0f}".replace(",", ".")` untuk format Rupiah
Windows: `Activate.ps1 cannot be loaded`	Jalankan `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` di PowerShell, atau gunakan CMD

D. Isi paket praktikum

Rinciannya disajikan pada Tabel L.5.

Tabel L.5. *Isi paket praktikum*

Folder/file	Isi
`data/`	`customer_churn.csv` (3.000 baris), `new_customers.csv` (200 baris), `generate_data.py`
`models/`	`churn_model.joblib`, `model_metadata.json`, `prepare_final_model.py` (dokumentasi fasilitator)
`utils/model_utils.py`	Fungsi load, validasi, persiapan input, dan prediksi
`sesi1/`	`app_hello.py`, `app_komponen.py`, `app_kalkulator_tagihan.py`
`sesi2/`	`eda_churn.py` (16 grafik EDA), `app_eda.py` (EDA interaktif 6 tab)
`sesi3/`	`app_layout_dashboard.py`
`sesi4/`	`cek_inference.py`, `app_prediksi.py`
`sesi5/`	`app_halo.py`, `app_dash_dasar.py`
`sesi6/`	`plotly_galeri.py` (13 chart), `app_dash_plotly.py`
`sesi7/`	`app_dash_callback.py`
`sesi8/`	`app_final_dashboard.py`
`kunci_jawaban/`	Solusi latihan mandiri yang sudah diuji
`requirements.txt`	Daftar library dan versi

E. Referensi

- Streamlit. *Caching overview*. docs.streamlit.io/develop/concepts/architecture/caching
- Streamlit. *Documentation*. docs.streamlit.io
- Plotly. *Adding Interactivity with Callbacks* (Dash). dash.plotly.com/basic-callbacks
- Plotly. *Basic Callbacks with State* (Dash). dash.plotly.com/state
- Plotly. *Plotly Express in Python*. plotly.com/python/plotly-express
- scikit-learn. *Model persistence*. scikit-learn.org/stable/model_persistence.html
- pandas. *User Guide*. pandas.pydata.org/docs/user_guide